AI Agent 개발 입문

- AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발
  - 과정 소개
  - 사전 준비 사항
  - 학습 경로 (Learning Path)
  - 모듈 목록 (전체 교육 시간: 약 30시간)
  - 환경 설정 가이드
  - 학습 자료 사용 방법
  - 전체 과정 일정표 (권장)
  - Part별 상세 개요
  - 자주 묻는 질문 (FAQ)
  - 참고 자료 모음
  - 버전 정보
- Module 01: AI 에이전트 개요와 핵심 개념
  - 학습 목표
  - 사전 지식
  - 1. 개념 설명
  - 2. 단계별 실습
  - 3. 실전 예제
  - 4. 연습 문제
  - 5. 핵심 정리
  - 6. 참고 자료
  - 다음 단계
- Module 02: LangGraph 기초 - 그래프, 노드, 엣지, 상태
  - 학습 목표
  - 사전 지식
  - 1. 개념 설명
  - 2. 단계별 실습
  - 3. 실전 예제
  - 4. 연습 문제
  - 5. 핵심 정리
  - 6. 참고 자료
  - 다음 단계

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

- Module 03: Jupyter 노트북 기반 에이전트 개발 환경 구축
  - 학습 목표
  - 사전 지식
  - 1. 개념 설명
  - 2. 단계별 실습
  - 3. 실전 예제
  - 4. 연습 문제
  - 5. 핵심 정리
  - 6. 참고 자료
  - 다음 단계
- Module 04: 나의 첫 LangGraph 에이전트 만들기
  - 학습 목표
  - 사전 지식
  - 1. 개념 설명
  - 2. 단계별 실습
  - 3. 실전 예제
  - 4. 연습 문제
  - 5. 핵심 정리
  - 6. 참고 자료
  - 다음 단계
- Module 05: 프롬프트 엔지니어링 - LLM의 성능을 결정하는 핵심 기술
  - 학습 목표
  - 사전 지식
  - 1. 개념 설명
  - 2. 단계별 실습
  - 3. 실전 예제
  - 4. 연습 문제
  - 5. 핵심 정리
  - 6. 참고 자료
  - 다음 단계
- Module 06: 구조화된 출력 - LLM 응답을 데이터로 변환하기
  - 학습 목표
  - 사전 지식
  - 1. 개념 설명
  - 2. 단계별 실습
  - 3. 실전 예제

2

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

  - 4. 연습 문제
  - 5. 핵심 정리
  - 6. 참고 자료
  - 다음 단계
- Module 07: LLM 호출 최적화 - 비용, 속도, 안정성
  - 학습 목표
  - 사전 지식
  - 1. 개념 설명
  - 2. 단계별 실습
  - 3. 실전 예제
  - 4. 연습 문제
  - 5. 핵심 정리
  - 6. 참고 자료
  - 다음 단계
- Module 08: 에러 처리와 회복 탄력성
  - 학습 목표
  - 사전 지식
  - 1. 개념 설명
  - 2. 단계별 실습
  - 3. 실전 예제
  - 4. 연습 문제
  - 5. 핵심 정리
  - 6. 참고 자료
  - 다음 단계
- Module 09: 외부 시스템 연동
  - 학습 목표
  - 사전 지식
  - 1. 개념 설명
  - 2. 단계별 실습
  - 3. 실전 예제
  - 4. 연습 문제
  - 5. 핵심 정리
  - 6. 참고 자료
  - 다음 단계
- Module 10: 리소스 최적화
  - 학습 목표

3

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

  - 사전 지식
  - 1. 개념 설명
  - 2. 단계별 실습
  - 3. 실전 예제
  - 4. 연습 문제
  - 5. 핵심 정리
  - 6. 참고 자료
  - 다음 단계
- Module 11: LLM 출력 품질 보장 - 신뢰할 수 있는 에이전트 만들기
  - 학습 목표
  - 사전 지식
  - 1. 개념 설명
  - 2. 단계별 실습
  - 3. 실전 예제: 6-레이어 품질 보장이 적용된 “코드 리뷰 에이전트”
  - 4. 연습 문제
  - 5. 핵심 정리
  - 6. 참고 자료
  - 다음 단계
- Module 12: LangGraph 고급 패턴 - 프로덕션 수준의 에이전트 아키텍처
  - 학습 목표
  - 사전 지식
  - 1. 개념 설명
  - 2. 단계별 실습
  - 3. 실전 예제: 멀티 문서 분석 에이전트
  - 4. 연습 문제
  - 5. 핵심 정리
  - 6. 참고 자료
  - 다음 단계

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

4

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

> **비개발자와 주니어 개발자를 위한 실습 중심 AI 에이전트 개발 교육 과정**

과정 소개

이 교육 과정은 AI 에이전트 개발을 처음 접하는 분들을 위해 설계되었습니다. LangGraph 프레임워크를 사용하여 단순한 그래프 구조부터 프로덕션 수준의 에이전트까지 단계적으로 학습합니다.

과정 목표

이 과정을 완료하면 다음을 할 수 있습니다:

1. AI 에이전트의 핵심 개념과 아키텍처를 이해하고 설명할 수 있다
2. LangGraph로 상태 기반 그래프를 설계하고 구현할 수 있다
3. LLM(대규모 언어 모델)을 에이전트에 연결하고 프롬프트를 설계할 수 있다
4. 도구(Tool)를 만들어 에이전트에 연결할 수 있다
5. 에러 처리, 테스트, 배포 등 프로덕션 고려사항을 적용할 수 있다
6. 멀티 에이전트 시스템을 설계하고 구현할 수 있다

대상 수강생

대상	설명
비개발자	AI 에이전트에 관심 있고 프로그래밍 기초를 배우고 싶은 분
주니어 개발자	Python 기초는 있지만 AI/LLM 개발 경험이 없는 분
QA 엔지니어	AI 기반 테스트 자동화에 관심 있는 분
기획자/PM	AI 에이전트 프로젝트를 이해하고 관리하고 싶은 분

사전 준비 사항

필수 환경

5

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

항목	최소 요구사항	권장사항
Python	3.10 이상	3.12
pip	최신 버전	최신 버전
OS	Windows 10, macOS 12, Ubuntu 20.04	macOS 또는 Linux
메모리	4GB RAM	8GB RAM 이상
에디터	아무 텍스트 에디터	VS Code

필수 사전 지식

- **Python 기초**: 변수, 함수, 조건문, 반복문, 딕셔너리
- **pip 사용법**: `pip install 패키지명` 으로 패키지 설치 가능
- **가상환경(venv)**: `python -m venv .venv` 로 가상환경 생성/활성화 가능
- **터미널 기초**: cd, ls, mkdir 등 기본 명령어 사용 가능

> **참고**: Python이 처음이시라면 아래 무료 강좌를 먼저 수강하세요. - 점프 투 파이썬 - Python 공식 튜토리얼 (한국어)

학습 경로 (Learning Path)

6

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
┌──────────────────────────────────────────────────────────────────────┐
│                          학습 경로 다이어그램                          │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  Part 1: 기초 다지기 (Foundation)                                     │
│  ┌──────────┐    ┌──────────────┐    ┌──────────────┐               │
│  │ Module 01 │──►│  Module 02   │──►│  Module 03   │               │
│  │ AI 에이전트│    │  LangGraph   │    │  State 관리  │               │
│  │ 개요      │    │  기초        │    │  심화        │               │
│  └──────────┘    └──────────────┘    └──────────────┘               │
│       │                                      │                       │
│       ▼                                      ▼                       │
│  Part 2: 첫 에이전트 만들기 (First Agent)                             │
│  ┌──────────┐    ┌──────────────┐    ┌──────────────┐               │
│  │ Module 04 │──►│  Module 05   │──►│  Module 06   │               │
│  │ LLM 연동  │    │  도구(Tool)  │    │  첫 에이전트 │               │
│  │           │    │  만들기      │    │  완성        │               │
│  └──────────┘    └──────────────┘    └──────────────┘               │
│       │                                      │                       │
│       ▼                                      ▼                       │
│  Part 3: 프롬프트와 LLM 심화 (Prompt & LLM)                           │
│  ┌──────────┐    ┌──────────────┐                                    │
│  │ Module 07 │──►│  Module 08   │                                    │
│  │ 프롬프트  │    │  LLM 라우팅  │                                    │
│  │ 엔지니어링│    │  & 최적화    │                                    │
│  └──────────┘    └──────────────┘                                    │
│       │                                                              │
│       ▼                                                              │
│  Part 4: 프로덕션 (Production)                                        │
│  ┌──────────┐    ┌──────────────┐                                    │
│  │ Module 09 │──►│  Module 10   │                                    │
│  │ 에러 처리 │    │  테스트 &    │                                    │
│  │ & 복구    │    │  배포        │                                    │
│  └──────────┘    └──────────────┘                                    │
│       │                                                              │
│       ▼                                                              │
│  Part 5: 고급 패턴 (Advanced)                                         │
│  ┌──────────┐    ┌──────────────┐                                    │
│  │ Module 11 │──►│  Module 12   │                                    │
│  │ 멀티      │    │  실전        │                                    │
│  │ 에이전트  │    │  프로젝트    │                                    │
│  └──────────┘    └──────────────┘                                    │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

모듈 목록 (전체 교육 시간: 약 30시간)

Part 1: 기초 다지기 (Foundation) - 약 7시간

7

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

모듈	제목	학습 시간	핵심 내용
01	AI 에이전트 개요와 핵심 개념	2시간	에이전트란?, LLM 기초, 아키텍처 패턴
02	LangGraph 기초 - 그래프, 노드, 엣지, 상태	2.5시간	StateGraph, TypedDict, 노드, 엣지, 조건부 분기
03	State 관리 심화 - Reducer와 체크포인트	2.5시간	Reducer 패턴, Annotation, 체크포인트, 상태 복구

Part 2: 첫 에이전트 만들기 (First Agent) - 약 8시간

모듈	제목	학습 시간	핵심 내용
04	LLM 연동 - ChatAnthropic과 메시지 구조	2.5시간	ChatAnthropic, 메시지 타입, 스트리밍
05	도구(Tool) 만들기 - 에이전트의 손과 발	2.5시간	@tool 데코레이터, ToolNode, 스키마 정의
06	첫 ReAct 에이전트 완성	3시간	ReAct 패턴 구현, 도구 호출 루프, 실행과 디버깅

Part 3: 프롬프트와 LLM 심화 (Prompt & LLM) - 약 5시간

모듈	제목	학습 시간	핵심 내용
07	프롬프트 엔지니어링 실전	2.5시간	시스템 프롬프트, Few-shot, 구조화된 출력
08	LLM 라우팅과 비용 최적화	2.5시간	모델 선택, 토큰 관리, 비용 제한, 캐싱

Part 4: 프로덕션 (Production) - 약 5시간

8

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

모듈	제목	학습 시간	핵심 내용
09	에러 처리와 복구 전략	2.5시간	재시도, 폴백, 타임아웃, 그레이스풀 디그레이드
10	테스트와 배포	2.5시간	단위 테스트, 통합 테스트, Docker, CI/CD 기초

Part 5: 고급 패턴 (Advanced) - 약 5시간

모듈	제목	학습 시간	핵심 내용
11	멀티 에이전트 시스템	2.5시간	서브 그래프, 에이전트 간 통신, 오케스트레이션
12	실전 프로젝트 - 문서 분석 에이전트	2.5시간	종합 프로젝트, 설계부터 배포까지

환경 설정 가이드

Step 1: Python 설치 확인

```
# Python 버전 확인 (3.10 이상이어야 합니다)
python --version
# 또는
python3 --version
```

출력 예시:

```
Python 3.12.0
```

Step 2: 프로젝트 디렉토리 생성

```
# 학습용 디렉토리 생성
mkdir ai-agent-study
cd ai-agent-study
```

Step 3: 가상환경 생성 및 활성화

9

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
# 가상환경 생성
python -m venv .venv

# 가상환경 활성화 (macOS/Linux)
source .venv/bin/activate

# 가상환경 활성화 (Windows)
# .venv\Scripts\activate
```

> **확인**: 터미널 프롬프트 앞에 `(.venv)` 가 표시되면 성공입니다.

Step 4: 패키지 설치

```
# 핵심 패키지 설치
pip install langgraph langchain-anthropic langchain-core pydantic

# 설치 확인
pip list | grep -E "(langgraph|langchain|pydantic)"
```

출력 예시:

```
langchain-anthropic    0.3.x
langchain-core         0.3.x
langgraph              0.3.x
pydantic               2.x.x
```

Step 5: API 키 설정

```
# Anthropic API 키 설정 (macOS/Linux)
export ANTHROPIC_API_KEY="your-api-key-here"

# Windows (PowerShell)
# $env:ANTHROPIC_API_KEY="your-api-key-here"
```

> **중요**: API 키는 절대 코드에 직접 작성하지 마세요! 환경변수 또는 `.env` 파일을 사용하세요.

Step 6: 설치 검증

다음 Python 코드를 실행하여 설치를 확인합니다:

10

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# verify_setup.py
from langgraph.graph import StateGraph, END
from typing import TypedDict

class TestState(TypedDict):
    message: str

def hello(state: TestState) -> dict:
    return {"message": "설치 완료! 학습을 시작하세요!"}

graph = StateGraph(TestState)
graph.add_node("hello", hello)
graph.set_entry_point("hello")
graph.add_edge("hello", END)

app = graph.compile()
result = app.invoke({"message": ""})
print(result["message"])
```

```
python verify_setup.py
# 출력: 설치 완료! 학습을 시작하세요!
```

학습 자료 사용 방법

각 모듈의 구성

모든 모듈은 동일한 구조를 따릅니다:

```
┌──────────────────────────────────────────┐
│  학습 목표          ← 이번 모듈에서 배울 것   │
│  사전 지식          ← 필요한 선수 지식         │
│  1. 개념 설명       ← 이론과 다이어그램        │
│  2. 단계별 실습     ← 코드를 따라 치며 학습    │
│  3. 실전 예제       ← 완성된 동작 코드         │
│  4. 연습 문제       ← 스스로 풀어보기          │
│  5. 핵심 정리       ← 요약과 체크리스트        │
│  6. 참고 자료       ← 더 깊이 공부할 링크      │
│  다음 단계          ← 다음 모듈 안내           │
└──────────────────────────────────────────┘
```

권장 학습 방법

1. **순서대로 학습하세요**: Module 01부터 순서대로 진행합니다
2. **코드를 직접 타이핑하세요**: 복사-붙여넣기 대신 직접 입력하면 더 잘 기억됩니다

11

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

3. **연습 문제를 꼭 풀어보세요**: 이해도를 확인하는 가장 좋은 방법입니다
4. **모르는 부분은 참고 자료를 확인하세요**: 각 모듈 끝에 추가 학습 링크가 있습니다
5. **하루 1모듈씩 진행하세요**: 2~3시간씩 집중해서 학습하는 것이 효과적입니다

코드 실행 방법

```
# 각 모듈의 코드 파일을 실행하는 방법
python 파일이름.py

# 예시
python 01_first_graph.py
```

전체 과정 일정표 (권장)

```
┌──────────┬──────────────────────────────┬──────────┐
│  주차    │  학습 내용                    │ 예상 시간 │
├──────────┼──────────────────────────────┼──────────┤
│  1주차   │  Module 01 ~ 03 (기초 다지기)  │  7시간   │
│  2주차   │  Module 04 ~ 06 (첫 에이전트)  │  8시간   │
│  3주차   │  Module 07 ~ 08 (프롬프트/LLM) │  5시간   │
│  4주차   │  Module 09 ~ 10 (프로덕션)     │  5시간   │
│  5주차   │  Module 11 ~ 12 (고급/프로젝트) │  5시간   │
├──────────┼──────────────────────────────┼──────────┤
│  합계    │  12개 모듈                    │ 약 30시간 │
└──────────┴──────────────────────────────┴──────────┘
```

Part별 상세 개요

Part 1: 기초 다지기 (Foundation)

AI 에이전트의 개념을 이해하고 LangGraph의 핵심 구성 요소를 학습합니다. 코드 없이 개념을 먼저 이해한 후, 간단한 그래프를 만들며 실습합니다.

핵심 질문: "AI 에이전트란 무엇이고, 어떻게 구조화하는가?"

Part 2: 첫 에이전트 만들기 (First Agent)

LLM을 연동하고, 도구를 만들어 연결하여 실제로 동작하는 에이전트를 완성합니다. 이 Part를 마치면 스스로 간단한 에이전트를 만들 수 있게 됩니다.

12

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

핵심 질문: "에이전트가 실제로 어떻게 생각하고 행동하는가?"

Part 3: 프롬프트와 LLM 심화 (Prompt & LLM)

프롬프트 엔지니어링의 실전 기법과 LLM 비용 최적화 전략을 학습합니다. 같은 에이전트라도 프롬프트에 따라 성능이 크게 달라짐을 체험합니다.

핵심 질문: "에이전트가 더 정확하고 효율적으로 동작하게 하려면?"

Part 4: 프로덕션 (Production)

실제 서비스에서 필요한 에러 처리, 테스트, 배포 방법을 학습합니다. 안정적으로 운영 가능한 에이전트 시스템을 만드는 방법을 익힙니다.

핵심 질문: "에이전트를 실제 서비스에 안전하게 배포하려면?"

Part 5: 고급 패턴 (Advanced)

여러 에이전트가 협력하는 시스템을 설계하고, 종합 프로젝트를 통해 전체 과정을 복습합니다.

핵심 질문: "여러 에이전트가 어떻게 함께 일하는가?"

자주 묻는 질문 (FAQ)

Q: 프로그래밍을 해본 적이 없어도 수강할 수 있나요?

A: Python 기초 문법(변수, 함수, 조건문)은 알아야 합니다. 완전 초보라면 먼저 점프 투 파이썬을 1~3장까지 학습한 후 시작하세요.

Q: API 키가 꼭 필요한가요?

A: Part 1 (Module 01~03)은 API 키 없이 학습할 수 있습니다. Part 2부터는 Anthropic API 키가 필요합니다. Anthropic Console에서 발급받을 수 있습니다.

Q: 예상 비용은 어느 정도인가요?

A: 전체 과정을 학습하는 데 약 $5~10 정도의 API 비용이 발생합니다. Anthropic은 신규 가입 시 무료 크레딧을 제공하기도 합니다.

Q: 오프라인에서도 학습할 수 있나요?

13

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

A: Part 1의 Module 01~03은 오프라인으로 학습 가능합니다. LLM을 사용하는 Module 04부터는 인터넷 연결이 필요합니다.

참고 자료 모음

공식 문서

- LangGraph 공식 문서
- LangChain 공식 문서
- Anthropic Claude 문서
- Pydantic 공식 문서

학습 자료

- LangGraph 튜토리얼
- LangChain Academy
- Anthropic Cookbook

커뮤니티

- LangChain Discord
- LangChain GitHub Discussions

버전 정보

항목	버전
교육 과정 버전	1.0
최종 업데이트	2026-03
LangGraph 기준 버전	0.3.x
Python 기준 버전	3.12

14

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

학습을 시작하려면 Module 01: AI 에이전트 개요와 핵심 개념으로 이동하세요.

15

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

⬆ 목차로 돌아가기

Module 01: AI 에이전트 개요와 핵심 개념

> **학습 시간**: 약 2시간 **난이도**: ★☆☆☆☆ (입문) **코드 실습**: 없음 (개념 학습)

학습 목표

이 모듈을 완료하면 다음을 할 수 있습니다:

- ☐ AI 에이전트가 무엇인지 자신만의 말로 설명할 수 있다
- ☐ AI 에이전트와 전통 소프트웨어의 차이를 3가지 이상 설명할 수 있다
- ☐ LLM(대규모 언어 모델)의 기본 동작 원리를 이해할 수 있다
- ☐ 에이전트 아키텍처 패턴 3가지를 구분할 수 있다
- ☐ 에이전트의 5대 구성 요소를 나열하고 설명할 수 있다

사전 지식

이 모듈은 **프로그래밍 경험이 없어도** 학습할 수 있습니다. 다음 개념에 익숙하면 더 쉽게 이해할 수 있습니다:

- 컴퓨터 프로그램이 무엇인지 대략적으로 알고 있음
- ChatGPT, Claude 등 AI 챗봇을 사용해본 경험이 있음
- "입력 → 처리 → 출력"의 개념을 이해하고 있음

---

1. 개념 설명

1.1 AI 에이전트란 무엇인가?

일상의 비유로 이해하기

AI 에이전트를 이해하는 가장 쉬운 방법은 **"디지털 직원"** 으로 생각하는 것입니다.

16

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
┌──────────────────────────────────────────────────────┐
│                    사람(직원)의 업무 처리                    │
│                                                      │
│   □ 인지(Perceive)     : 이메일을 읽고, 상황을 파악한다     │
│        ↓                                             │
│   □ 추론(Reason)       : 무엇을 해야 할지 판단한다         │
│        ↓                                             │
│   □ 행동(Act)          : 문서를 작성하고, 회신한다          │
│        ↓                                             │
│   □ 반복(Loop)         : 결과를 확인하고, 추가 작업한다     │
│                                                      │
└──────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────┐
│                  AI 에이전트의 업무 처리                    │
│                                                      │
│   □ 인지(Perceive)     : 입력 데이터를 받아 분석한다        │
│        ↓                                             │
│   □ 추론(Reason)       : LLM이 다음 행동을 결정한다        │
│        ↓                                             │
│   □ 행동(Act)          : 도구를 호출해 작업을 수행한다      │
│        ↓                                             │
│   □ 반복(Loop)         : 결과를 확인하고, 필요시 반복한다   │
│                                                      │
└──────────────────────────────────────────────────────┘
```

정의

> **AI 에이전트(AI Agent)** 란, 주어진 목표를 달성하기 위해 **스스로 인지하고, 추론하고, 행동하는** 자율적 소프트웨어 시스템입니다.

핵심 키워드 세 가지: 1. **자율성(Autonomy)**: 사람의 매 단계 개입 없이 스스로 판단 2. **목표 지향(Goal-oriented)**: 주어진 목표를 달성하기 위해 행동 3. **도구 사용(Tool-use)**: 외부 도구(API, 데이터베이스, 파일 등)를 활용

챗봇 vs 에이전트

많은 분들이 ChatGPT 같은 챗봇과 에이전트를 혼동합니다. 차이를 알아봅시다:

17

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
┌──────────────────────────────────────────────────────────┐
│                        챗봇 (Chatbot)                     │
│                                                           │
│   사용자: "날씨 알려줘"                                    │
│        ↓                                                  │
│   챗봇: "저는 실시간 날씨를 확인할 수 없습니다."             │
│                                                           │
│   → 한 번의 질문 / 한 번의 응답 (단발성)                    │
│   → 외부 도구 사용 불가                                     │
│   → 텍스트 생성만 가능                                      │
│                                                           │
└──────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────┐
│                       에이전트 (Agent)                    │
│                                                           │
│   사용자: "날씨 알려줘"                                    │
│        ↓                                                  │
│   에이전트: (생각) "날씨 API를 호출해야겠다"                 │
│        ↓                                                  │
│   에이전트: (행동) weather_api.get("서울") 호출             │
│        ↓                                                  │
│   에이전트: (응답) "서울은 현재 맑음, 22°C입니다"            │
│                                                           │
│   → 여러 단계의 추론과 행동 (연쇄적)                        │
│   → 외부 도구 사용 가능                                     │
│   → 실제 작업 수행 가능                                     │
│                                                           │
└──────────────────────────────────────────────────────────┘
```

1.2 AI 에이전트 vs 전통 소프트웨어 비교

18

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

비교 항목	전통 소프트웨어	AI 에이전트
동작 방식	미리 정해진 규칙(if-else)	LLM이 상황에 따라 판단
유연성	정해진 시나리오만 처리	예상하지 못한 상황도 대응 가능
입력 형태	구조화된 데이터 (JSON, DB)	자연어, 비정형 데이터 포함
실행 흐름	고정된 순서 (A→B→C)	동적으로 결정 (A→C or A→B→A→C)
에러 처리	예외 코드로 사전 정의	상황 판단 후 대안 탐색 가능
학습	코드 수정 필요	프롬프트 변경으로 행동 조정
비용	서버 비용만 발생	서버 + LLM API 비용 발생
속도	매우 빠름 (밀리초)	상대적으로 느림 (초 단위)
정확성	100% 결정적(Deterministic)	확률적(Probabilistic)

> **핵심 포인트**: AI 에이전트는 "유연하지만 느리고 비용이 드는" 특성을 가집니다. 따라서 모든 것을 에이전트로 만들 필요는 없습니다. **에이전트가 적합한 문제**를 선별하는 것이 중요합니다.

에이전트가 적합한 경우

- 입력이 비정형(자연어, 다양한 형식)인 경우
- 처리 과정에 판단이 필요한 경우
- 다양한 도구를 상황에 따라 선택해야 하는 경우
- 사람이 수동으로 반복하던 작업을 자동화하려는 경우

에이전트가 부적합한 경우

- 단순 반복 작업 (예: 파일 복사, 정해진 형식 변환)
- 100% 정확성이 요구되는 경우 (예: 금융 거래)
- 실시간 응답이 필수인 경우 (예: 밀리초 단위 API)
- 비용이 극도로 중요한 경우

1.3 LLM(대규모 언어 모델) 기초

LLM이란?

19

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

> **LLM(Large Language Model)** 은 대규모 텍스트 데이터를 학습하여 **다음에 올 단어를 예측**하는 AI 모델입니다.

```
┌──────────────────────────────────────────────────────────┐
│                      LLM의 동작 원리                      │
│                                                          │
│   프롬프트(입력)          LLM 모델          응답(출력)     │
│                                                          │
│   "한국의 수도는"  ──▶  [신경망 처리]  ──▶  "서울입니다"   │
│                                                          │
│   내부적으로는 "한국의 수도는" 다음에 올 가장 적절한         │
│   단어들을 확률적으로 예측하여 생성합니다.                   │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

프롬프트(Prompt)와 응답(Response)

LLM과의 대화는 항상 **프롬프트 → 응답** 구조입니다:

```
프롬프트 (사람이 작성)          응답 (LLM이 생성)
──────────────────          ──────────────────

"이 코드의 버그를 찾아줘:      "3번째 줄에서 인덱스
 x = list[5]                  범위를 벗어날 수 있습니다.
 print(x[10])"                list에 11개 이상의 요소가
                               있는지 확인하세요."
```

토큰(Token)이란?

LLM은 텍스트를 **토큰(Token)** 이라는 단위로 처리합니다.

```
┌──────────────────────────────────────────────────────────┐
│                      토큰 분해 예시                       │
│                                                          │
│   영어:  "Hello, world!"                                  │
│          → ["Hello", ",", " world", "!"]  (4 토큰)        │
│                                                          │
│   한국어: "안녕하세요"                                     │
│          → ["안녕", "하", "세요"]  (3 토큰, 모델마다 다름)   │
│                                                          │
│   코드:  "print('hi')"                                    │
│          → ["print", "('", "hi", "')"]  (4 토큰)          │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

> **왜 토큰이 중요한가요?** - LLM API는 **토큰 수에 따라 비용**이 부과됩니다 - 한 번에 처리할 수 있는 토큰 수에 **제한(Context Window)** 이 있습니다 - 한국어는 영어보다 같은 의미를 표현하는 데 더 많은 토큰을 소비하는 경향이

20

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

> 있습니다

Temperature (온도)

Temperature는 LLM 응답의 **창의성(무작위성)** 을 조절하는 파라미터입니다.

```
Temperature 0.0 (결정적)          Temperature 1.0 (창의적)
────────────────────────          ────────────────────────
항상 같은 답변 생성               매번 다른 답변 생성
정확하고 일관됨                     다양하고 창의적
사실 기반 작업에 적합             글쓰기, 브레인스토밍에 적합

예: "1+1은?"                        예: "시를 써줘"
답: "2" (항상 동일)                 답: (매번 다른 시)
```

```
확률 분포 시각화

Temperature = 0 (가장 높은 확률의 단어만 선택)
┌──────────────────────────────────────┐
│  ▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇  "서울"  (95%)  │
│  ▇▇                    "부산"  (3%)   │
│  ▇                     "대구"  (2%)   │
└──────────────────────────────────────┘

Temperature = 1.0 (확률 분포를 더 균등하게)
┌──────────────────────────────────────┐
│  ▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇  "서울"  (50%)  │
│  ▇▇▇▇▇▇                "부산"  (25%)  │
│  ▇▇▇▇▇                 "대구"  (25%)  │
└──────────────────────────────────────┘
```

주요 LLM 모델 비교

모델	제공사	특징	용도 예시
Claude (Sonnet/Haiku)	Anthropic	안전성 강조, 긴 컨텍스트	문서 분석, 코드 리뷰
GPT-4o	OpenAI	범용 고성능	복잡한 추론, 코드 생성
Gemini	Google	멀티모달 강점	이미지+텍스트 분석
Llama	Meta	오픈소스	자체 서버 배포

1.4 에이전트 아키텍처 패턴

21

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

에이전트를 설계하는 대표적인 패턴 3가지를 살펴봅니다.

패턴 1: ReAct (Reasoning + Acting)

"생각하고, 행동하고, 관찰하고, 반복한다"

가장 널리 사용되는 패턴입니다. 사람의 사고 과정과 유사합니다.

```
┌──────────────────────────────────────────────────────┐
│                    ReAct 패턴 흐름도                  │
│                                                       │
│     ┌──────────┐                                      │
│     │  입력    │                                      │
│     │ (질문)   │                                      │
│     └──────────┘                                      │
│          │                                            │
│          ▼                                            │
│     ┌──────────┐     ┌──────────┐     ┌──────────┐   │
│     │ Thought  │──▶│  Action  │──▶│Observation│   │
│     │ (추론)   │     │ (행동)   │     │ (관찰)   │   │
│     └──────────┘     └──────────┘     └──────────┘   │
│          ▲                                  │         │
│          │          아직 미완료?            │         │
│          └──────────────────────────────────┘         │
│                          │                            │
│                     완료됨!                           │
│                          ▼                            │
│                    ┌──────────┐                       │
│                    │ 최종 응답 │                       │
│                    └──────────┘                       │
└──────────────────────────────────────────────────────┘
```

예시: "서울 날씨에 맞는 옷 추천해줘"

단계	구분	내용
1	Thought	"먼저 서울 현재 날씨를 확인해야겠다"
2	Action	weather_api("서울") 호출
3	Observation	"서울: 22°C, 맑음"
4	Thought	"22°C 맑은 날씨에 맞는 옷을 추천하자"
5	Action	최종 답변 생성
6	응답	"가벼운 긴팔 셔츠나 얇은 가디건을 추천합니다"

패턴 2: Plan-and-Execute

"먼저 계획을 세우고, 단계별로 실행한다"

22

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

복잡한 작업에 적합합니다. 사람이 프로젝트를 계획하는 것과 유사합니다.

```
┌──────────────────────────────────────────────────────────┐
│              Plan-and-Execute 패턴 흐름도                 │
│                                                          │
│     ┌──────────┐                                         │
│     │  입력    │                                         │
│     │ (복잡한  │                                         │
│     │  작업)   │                                         │
│     └──────────┘                                         │
│         │                                                │
│         ▼                                                │
│     ┌──────────────────────────┐                         │
│     │  Planner (계획 수립)     │                         │
│     │  Step 1: 데이터 수집     │                         │
│     │  Step 2: 데이터 분석     │                         │
│     │  Step 3: 보고서 작성     │                         │
│     └──────────────────────────┘                         │
│         │                                                │
│         ▼                                                │
│     ┌──────────┐   ┌──────────┐   ┌──────────┐         │
│     │ Execute  │→  │ Execute  │→  │ Execute  │         │
│     │ Step 1   │   │ Step 2   │   │ Step 3   │         │
│     └──────────┘   └──────────┘   └──────────┘         │
│                                        │                 │
│                                        ▼                 │
│                                   ┌──────────┐           │
│                                   │ 최종 결과 │           │
│                                   └──────────┘           │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

패턴 3: Tool-use (도구 사용)

"필요한 도구를 선택하여 사용한다"

LLM이 상황에 맞는 도구를 골라 실행하는 패턴입니다.

23

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
┌─────────────────────────────────────────────────────┐
│                Tool-use 패턴 흐름도                  │
│                                                     │
│     ┌──────────┐                                    │
│     │  입력    │                                    │
│     └──────────┘                                    │
│          │                                          │
│          ▼                                          │
│     ┌──────────┐      어떤 도구를                   │
│     │   LLM    │      사용할까?                     │
│     │  (판단)  │──────────────┐                     │
│     └──────────┘              │                     │
│                               ▼                     │
│           ┌──────────────────────────┐              │
│           │      도구 상자 (Tool Box)  │              │
│           │                          │              │
│           │  🔍 검색     📧 이메일     │              │
│           │  📊 분석     💾 저장       │              │
│           │  🌐 API      📝 문서 작성  │              │
│           │                          │              │
│           └──────────────────────────┘              │
│                               │                     │
│                               ▼                     │
│                        ┌──────────┐                 │
│                        │ 도구 실행 │                 │
│                        │ 결과 반환 │                 │
│                        └──────────┘                 │
└─────────────────────────────────────────────────────┘
```

패턴 비교 요약

패턴	장점	단점	적합한 경우
ReAct	유연, 적응적	비효율적일 수 있음	탐색적 작업, 대화형
Plan-and-Execute	체계적, 예측 가능	계획 수정이 어려움	복잡하고 정형화된 작업
Tool-use	확장성, 모듈화	도구 선택 실수 가능	다양한 도구가 필요한 경우

> **실무에서는** 이 패턴들을 조합하여 사용합니다. 예를 들어, ReAct + Tool-use 패턴이 가장 일반적입니다.

1.5 에이전트 구성 요소

LangGraph에서 에이전트는 다음 5가지 요소로 구성됩니다:

24

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
┌──────────────────────────────────────────────────────────┐
│                  에이전트 구성 요소 전체 그림                │
│                                                          │
│   ┌──────────────────────────────────────────┐           │
│   │              그래프 (Graph)                 │          │
│   │                                          │           │
│   │   ┌──────────────────────────────────┐   │           │
│   │   │          상태 (State)              │   │           │
│   │   │   messages: [...]                 │   │           │
│   │   │   current_step: "분석중"           │   │           │
│   │   │   result: null                    │   │           │
│   │   └──────────────────────────────────┘   │           │
│   │                                          │           │
│   │   ┌──────┐   엣지(Edge)   ┌──────┐        │          │
│   │   │노드 A │──────────────▶│노드 B │        │          │
│   │   │(분석) │               │(생성) │        │          │
│   │   └──────┘               └──────┘        │          │
│   │       │                      │           │           │
│   │       ▼                      ▼           │           │
│   │   ┌──────┐               ┌──────┐        │          │
│   │   │도구 1 │               │도구 2 │        │          │
│   │   │(검색) │               │(저장) │        │          │
│   │   └──────┘               └──────┘        │          │
│   │                                          │           │
│   └──────────────────────────────────────────┘           │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

(1) 노드 (Node)

> **노드**는 에이전트가 수행하는 **개별 작업 단위**입니다.

사람으로 비유하면 "하나의 업무 단계"에 해당합니다.

노드 예시	설명
분석 노드	입력 데이터를 분석하는 단계
LLM 호출 노드	AI 모델에게 질문하는 단계
도구 실행 노드	외부 API를 호출하는 단계
결과 정리 노드	결과를 포맷팅하는 단계

(2) 엣지 (Edge)

25

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

> **엣지**는 노드와 노드를 연결하는 **흐름의 방향**입니다.

두 가지 종류가 있습니다:

```
일반 엣지 (항상 같은 방향):
   노드 A ──────────▶ 노드 B
   (무조건 A 다음에 B 실행)

조건부 엣지 (조건에 따라 분기):
                      ┌──▶ 노드 B (조건 1)
   노드 A ──(판단)──┤
                      └──▶ 노드 C (조건 2)
   (A의 결과에 따라 B 또는 C 실행)
```

(3) 상태 (State)

> **상태**는 에이전트가 작업하면서 **공유하는 데이터 공간**입니다.

사무실의 공유 화이트보드와 같습니다. 모든 노드가 상태를 읽고 업데이트합니다.

```
┌──────────────── 상태 (State) ────────────────┐
│                                              │
│   messages: ["안녕하세요", "무엇을 도와..."]  │
│   user_intent: "날씨 질문"                    │
│   tool_results: {"temp": 22}                 │
│   final_answer: null                         │
│                                              │
│   ← 노드 A가 messages를 읽음                 │
│   ← 노드 B가 user_intent를 업데이트          │
│   ← 노드 C가 tool_results를 업데이트         │
│   ← 노드 D가 final_answer를 작성             │
│                                              │
└──────────────────────────────────────────────┘
```

(4) 그래프 (Graph)

> **그래프**는 노드와 엣지를 연결한 **전체 워크플로우**입니다.

에이전트의 전체 로직을 하나의 그래프로 표현합니다.

26

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
그래프 = 노드 + 엣지 + 상태의 조합

┌─────┐    ┌─────┐    ┌─────┐    ┌─────┐
│START│──▶│분석 │──▶│생성 │──▶│ END │
└─────┘    └─────┘    └─────┘    └─────┘
```

(5) 도구 (Tool)

> **도구**는 에이전트가 외부 세계와 **상호작용하는 수단**입니다.

사람이 컴퓨터, 전화, 이메일을 사용하듯, 에이전트도 도구를 사용합니다.

도구 종류	예시
검색 도구	웹 검색, 데이터베이스 조회
계산 도구	수학 계산, 통계 분석
통신 도구	이메일 발송, 슬랙 메시지
파일 도구	파일 읽기/쓰기, PDF 생성
API 도구	외부 서비스 호출 (날씨, 지도 등)

1.6 핵심 용어 사전

27

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

한국어	영어	설명
에이전트	Agent	자율적으로 작업을 수행하는 AI 시스템
대규모 언어 모델	LLM (Large Language Model)	텍스트를 이해하고 생성하는 AI 모델
프롬프트	Prompt	LLM에 전달하는 입력 텍스트
토큰	Token	LLM이 텍스트를 처리하는 최소 단위
온도	Temperature	응답의 창의성/무작위성 조절 파라미터
컨텍스트 윈도우	Context Window	LLM이 한 번에 처리할 수 있는 최대 토큰 수
노드	Node	그래프에서 하나의 작업 단위
엣지	Edge	노드 간의 연결 (흐름 방향)
상태	State	에이전트가 공유하는 데이터 공간
그래프	Graph	노드와 엣지로 구성된 전체 워크플로우
도구	Tool	에이전트가 사용하는 외부 기능
추론	Reasoning	상황을 분석하고 판단하는 과정
행동	Action	도구 호출 등 실제 작업 수행
관찰	Observation	행동의 결과를 확인하는 과정
오케스트레이션	Orchestration	여러 에이전트의 협력을 조율하는 것
할루시네이션	Hallucination	LLM이 사실이 아닌 내용을 생성하는 현상
그라운딩	Grounding	LLM 응답을 사실에 근거하게 만드는 기법
프롬프트 엔지니어링	Prompt Engineering	효과적인 프롬프트를 설계하는 기술
체인	Chain	여러 LLM 호출을 순서대로 연결한 것
파이프라인	Pipeline	데이터 처리의 연속적인 단계들

1.7 에이전트 활용 사례

28

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

AI 에이전트는 다양한 분야에서 활용되고 있습니다:

소프트웨어 개발

```
┌──────────────────────────────────────────────────┐
│  코드 분석 에이전트                                 │
│                                                   │
│  입력: "이 Python 코드를 리뷰해줘"                  │
│    ↓                                              │
│  [코드 파싱] → [패턴 분석] → [보안 검사] → [리뷰 작성]  │
│    ↓                                              │
│  출력: "3건의 버그, 2건의 보안 이슈, 5건의 개선 제안"    │
└──────────────────────────────────────────────────┘
```

활용 분야	에이전트 역할	예시
코드 리뷰	코드 품질 분석 및 개선 제안	버그 탐지, 보안 취약점 발견
테스트 생성	테스트 케이스 자동 생성	단위 테스트, 시나리오 작성
문서 작성	코드 기반 문서 자동 생성	API 문서, README 작성
디버깅	에러 원인 분석 및 해결책 제시	로그 분석, 스택 트레이스 해석

비즈니스/일반

활용 분야	에이전트 역할	예시
고객 지원	질문 분류 및 자동 응답	FAQ 자동 응답, 티켓 분류
문서 요약	긴 문서를 핵심만 추출	논문 요약, 회의록 정리
데이터 분석	데이터 수집 및 인사이트 도출	매출 분석, 트렌드 파악
콘텐츠 생성	다양한 형식의 콘텐츠 작성	마케팅 카피, SNS 게시물
일정 관리	일정 조율 및 알림	미팅 스케줄링, 마감 관리
번역	다국어 문서 번역	웹사이트 현지화, 기술 문서 번역

29

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

2. 단계별 실습

이 모듈에서는 코드를 작성하지 않습니다. 대신 에이전트의 동작을 **직접 설계해보는** 사고 실습을 진행합니다.

실습 1: 에이전트 동작 흐름 따라가기

다음 시나리오를 읽고, 에이전트가 어떤 순서로 동작할지 생각해보세요.

시나리오: 사용자가 "오늘 서울 날씨를 확인하고, 비가 오면 실내 데이트 코스를 추천해줘" 라고 입력했습니다.

에이전트가 사용할 수 있는 도구: - `get_weather(city)` - 도시의 현재 날씨 조회 - `search_places(query, category)` - 장소 검색 - `get_reviews(place_id)` - 장소 리뷰 조회

풀어볼 질문: 1. 에이전트의 첫 번째 Thought(추론)는 무엇일까요? 2. 몇 번의 도구 호출이 필요할까요? 3. 날씨가 맑다면 에이전트의 행동이 어떻게 달라질까요?

▶ 정답 예시 (클릭하여 펼치기)

실습 2: 에이전트 구성 요소 매핑

다음 "이메일 자동 분류 에이전트"의 각 부분이 어떤 구성 요소에 해당하는지 연결해보세요.

번호	에이전트의 부분	구성 요소는?
1	이메일 내용을 읽는 단계	_____
2	"이 이메일은 업무/스팸/개인" 판단	_____
3	이메일 목록, 분류 결과 데이터	_____
4	읽기 → 분류 → 폴더 이동 순서	_____
5	스팸 필터 API, 이메일 API	_____

▶ 정답 (클릭하여 펼치기)

3. 실전 예제

예제: "고객 문의 처리 에이전트" 설계

실제 비즈니스에서 사용할 수 있는 에이전트를 개념적으로 설계해봅시다.

30

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

요구사항

- 고객의 문의 이메일을 자동으로 처리
- 문의 유형을 분류 (환불, 기술 지원, 일반 문의, 불만)
- 유형에 따라 적절한 부서로 전달
- 간단한 문의는 자동 응답

설계 (그래프 구조)

31

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
┌──────────────────────────────────────────────────────────────────────┐
│                 고객 문의 처리 에이전트 설계                          │
│                                                                      │
│    ┌──────────┐                                                      │
│    │  START   │                                                      │
│    │ (문의수신)│                                                      │
│    └──────────┘                                                      │
│         │                                                            │
│         ▼                                                            │
│    ┌──────────┐                                                      │
│    │ 문의 분류 │  ← LLM이 문의 내용을 분석하여 유형 판단              │
│    │  (노드1) │                                                      │
│    └──────────┘                                                      │
│         │                                                            │
│         ├── 환불 요청 ──────▶ ┌──────────┐                           │
│         │                     │ 환불 처리 │ ← 환불 정책 확인          │
│         │                     │  (노드2a) │   & 자동 처리             │
│         │                     └──────────┘                           │
│         │                                                            │
│         ├── 기술 지원 ──────▶ ┌──────────┐                           │
│         │                     │ 기술 지원 │ ← FAQ 검색                │
│         │                     │  (노드2b) │   & 해결책 제시           │
│         │                     └──────────┘                           │
│         │                                                            │
│         ├── 일반 문의 ──────▶ ┌──────────┐                           │
│         │                     │ 자동 응답 │ ← 템플릿 기반             │
│         │                     │  (노드2c) │   응답 생성               │
│         │                     └──────────┘                           │
│         │                                                            │
│         └── 불만 접수 ──────▶ ┌──────────┐                           │
│                               │ 담당자    │ ← 사람에게                │
│                               │ 에스컬레이션│  즉시 전달               │
│                               │  (노드2d) │                           │
│                               └──────────┘                           │
│                                    │                                 │
│                                    ▼                                 │
│                               ┌──────────┐                           │
│                               │ 결과 기록 │ ← DB에 처리 결과          │
│                               │  (노드3) │   저장                    │
│                               └──────────┘                           │
│                                    ▼                                 │
│                               ┌──────────┐                           │
│                               │   END    │                           │
│                               └──────────┘                           │
│                                                                      │
│   [State 구성]                                                       │
│   - email_content: 원본 이메일 내용                                  │
│   - category: 분류 결과 (환불/기술/일반/불만)                        │
│   - response: 생성된 응답                                            │
│   - escalated: 에스컬레이션 여부                                     │
│                                                                      │
│   [사용 도구]                                                        │
│   - 이메일 API: 이메일 수신/발신                                     │
│   - DB API: 고객 정보 조회, 처리 결과 저장                           │
│   - FAQ 검색: 기술 문서 검색                                         │
```


32

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
│                                                                │
└────────────────────────────────────────────────────────────────┘
```

구성 요소 정리

구성 요소	이 에이전트에서의 역할
노드	문의 분류, 환불 처리, 기술 지원, 자동 응답, 에스컬레이션, 결과 기록
엣지	노드 간 연결 (분류 결과에 따른 조건부 엣지 포함)
상태	email_content, category, response, escalated
그래프	위의 전체 흐름도
도구	이메일 API, DB API, FAQ 검색

---

4. 연습 문제

문제 1: 개념 확인

다음 빈칸을 채우세요:

1. AI 에이전트의 3대 특성은 _____, _____, _____ 이다.
2. LLM은 _____을 예측하여 텍스트를 생성한다.
3. Temperature가 0에 가까울수록 응답은 _____ 해지고, 1에 가까울수록 _____ 해진다.
4. ReAct 패턴은 _____ → _____ → _____ 을 반복한다.
5. 에이전트가 외부 API를 호출하는 수단을 _____ 이라고 한다.

▶ 정답

문제 2: 에이전트 설계

다음 시나리오에서 에이전트를 설계해보세요. (종이에 그려도 좋습니다)

시나리오: "회의록 정리 에이전트" - 회의 녹음을 텍스트로 변환 (음성 → 텍스트) - 핵심 내용 요약 - 액션 아이템(할일) 추출 - 참석자별 발언 정리 - 결과를 슬랙으로 전송

설계해볼 것: 1. 필요한 노드 목록 2. 노드 간 연결 (엣지) 순서 3. 상태에 포함될 데이터 4. 필요한 도구 목록

▶ 정답 예시

문제 3: 적합성 판단

33

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

다음 각 상황에서 AI 에이전트가 적합한지(O) 부적합한지(X) 판단하고 이유를 적어보세요:

1. ( ) 매일 오전 9시에 같은 형식의 보고서를 생성하는 작업
2. ( ) 다양한 형식의 이력서를 분석하여 핵심 역량을 추출하는 작업
3. ( ) 주식 거래를 밀리초 단위로 실행하는 고빈도 트레이딩
4. ( ) 고객의 자유 형식 피드백을 분류하고 요약하는 작업
5. ( ) CSV 파일의 A열과 B열을 합산하는 작업

▶ 정답

---

5. 핵심 정리

이 모듈에서 배운 것

```
┌──────────────────────────────────────────────────────────┐
│                      Module 01 핵심 정리                  │
│                                                           │
│  1. AI 에이전트 = 인지 + 추론 + 행동 하는 자율 시스템       │
│                                                           │
│  2. LLM = 다음 토큰을 예측하는 대규모 언어 모델             │
│     - 프롬프트 → LLM → 응답                                │
│     - Temperature로 창의성 조절                            │
│                                                           │
│  3. 아키텍처 패턴:                                         │
│     - ReAct: 생각→행동→관찰 반복                           │
│     - Plan-and-Execute: 계획→순차 실행                     │
│     - Tool-use: 필요한 도구 선택 사용                      │
│                                                           │
│  4. 구성 요소 5가지:                                       │
│     - 노드(Node): 작업 단위                                │
│     - 엣지(Edge): 노드 간 연결                             │
│     - 상태(State): 공유 데이터                             │
│     - 그래프(Graph): 전체 워크플로우                       │
│     - 도구(Tool): 외부 상호작용 수단                       │
│                                                           │
│  5. 에이전트는 만능이 아니다                               │
│     → 적합한 문제를 선별하는 것이 중요                     │
│                                                           │
└──────────────────────────────────────────────────────────┘
```

체크리스트

다음 항목을 모두 설명할 수 있다면 이 모듈을 완료한 것입니다:

- ☐ AI 에이전트의 정의와 3대 특성

34

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

- ☐ 챗봇과 에이전트의 차이
- ☐ LLM의 기본 동작 원리 (프롬프트 → 토큰 예측 → 응답)
- ☐ Temperature의 역할
- ☐ ReAct, Plan-and-Execute, Tool-use 패턴의 차이
- ☐ 노드, 엣지, 상태, 그래프, 도구의 역할
- ☐ 에이전트가 적합한/부적합한 상황 구분

---

6. 참고 자료

필수 읽기

자료	링크	설명
LangGraph 공식 문서	langgraph docs	LangGraph 프레임워크 공식 가이드
LangChain 개념 가이드	langchain concepts	LangChain 핵심 개념 설명
Anthropic Claude 문서	anthropic docs	Claude 모델 사용법
OpenAI 토큰 설명	openai tokenizer	토큰이 무엇인지 직접 체험

추가 학습

자료	링크	설명
ReAct 논문 (원문)	arxiv.org/abs/2210.03629	ReAct 패턴의 원본 논문
LangChain Academy	academy.langchain.com	LangChain 공식 교육 플랫폼
Anthropic Cookbook	github.com/anthropics/anthropic-cookbook	Anthropic의 실전 예제 모음
Prompt Engineering Guide	promptingguide.ai	프롬프트 엔지니어링 종합 가이드 (한국어)

영상 자료

자료	설명
LangGraph Quickstart (YouTube)	LangGraph 빠른 시작 영상
What are AI Agents? (YouTube)	AI 에이전트 개념 설명 영상

35

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

다음 단계

축하합니다! Module 01을 완료했습니다.

다음 모듈에서는 이 개념들을 **실제 코드**로 구현합니다:

> **Module 02: LangGraph 기초 - 그래프, 노드, 엣지, 상태**
>
> - LangGraph 설치 및 첫 그래프 작성
> - TypedDict로 상태 정의하기
> - 노드 함수 작성 및 엣지 연결
> - 조건부 엣지로 분기 처리
> - 실행 결과 시각화

이론은 충분합니다. 이제 코드를 작성할 시간입니다!

36

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

⬆ 목차로 돌아가기

Module 02: LangGraph 기초 - 그래프, 노드, 엣지, 상태

학습 시간: 약 2.5시간 **난이도**: ★★☆☆☆ (초급) **코드 실습**: 있음 (Python 코드 직접 작성)

학습 목표

이 모듈을 완료하면 다음을 할 수 있습니다:

- ☐ LangGraph의 역할과 장점을 설명할 수 있다
- ☐ TypedDict로 상태(State)를 정의할 수 있다
- ☐ 노드 함수를 작성하고 그래프에 추가할 수 있다
- ☐ 일반 엣지와 조건부 엣지를 사용할 수 있다
- ☐ StateGraph를 컴파일하고 실행할 수 있다
- ☐ Mermaid 다이어그램으로 그래프를 시각화할 수 있다

사전 지식

이 모듈을 시작하기 전에 다음을 확인하세요:

- ☐ Module 01 완료 (에이전트 개념 이해)
- ☐ Python 기초 (함수, 딕셔너리, 타입 힌트)
- ☐ 환경 설정 완료 (`pip install langgraph` 설치됨)
- ☐ 코드 에디터 준비 (VS Code 권장)

Python 딕셔너리가 낯설다면?

37

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
# 딕셔너리 = 키:값 쌍의 모음
person = {"name": "홍길동", "age": 30}
print(person["name"])  # "홍길동"
```

---

1. 개념 설명

1.1 LangGraph란?

> **LangGraph**는 **코드로 워크플로우를 그래프로 표현하는 프레임워크**입니다.

Module 01에서 배운 노드, 엣지, 상태, 그래프 개념을 실제 Python 코드로 구현할 수 있게 해주는 도구입니다.

```
┌──────────────────────────────────────────────────────────┐
│                LangGraph의 역할                          │
│                                                          │
│   "이런 에이전트를 만들고 싶다"  (아이디어)                │
│        ↓                                                 │
│   ┌──────────────────────────────────────┐               │
│   │            LangGraph                 │               │
│   │                                      │               │
│   │  - 상태(State) 정의 도구              │               │
│   │  - 노드(Node) 등록 도구               │               │
│   │  - 엣지(Edge) 연결 도구               │               │
│   │  - 그래프 컴파일 & 실행 엔진          │               │
│   │  - 시각화 도구                        │               │
│   │                                      │               │
│   └──────────────────────────────────────┘               │
│        ↓                                                 │
│   "실제로 동작하는 에이전트"  (코드)                       │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

LangGraph를 사용하는 이유

38

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

직접 구현	LangGraph 사용
상태 관리를 직접 코딩	TypedDict로 간단히 정의
분기 로직을 if-else로 복잡하게 작성	조건부 엣지로 깔끔하게 표현
실행 흐름 추적이 어려움	자동 시각화 (Mermaid)
에러 시 어디서 멈췄는지 파악 어려움	체크포인트로 자동 추적
테스트가 어려움	노드 단위 테스트 가능

1.2 핵심 개념 상세 설명

(1) StateGraph - 상태를 중심으로 동작하는 그래프

StateGraph는 LangGraph의 핵심 클래스입니다. 모든 에이전트는 StateGraph로 시작합니다.

```
┌──────────────────────────────────────────────────────────┐
│                    StateGraph 구조                       │
│                                                          │
│   StateGraph(상태 타입)                                   │
│        │                                                 │
│        ├── add_node("이름", 함수)      ← 노드 추가        │
│        ├── add_edge("A", "B")          ← 엣지 추가        │
│        ├── set_entry_point("시작")     ← 시작점 설정      │
│        ├── add_conditional_edges(...) ← 조건부 엣지       │
│        │                                                 │
│        └── compile()                   ← 실행 가능하게 컴파일│
│              │                                           │
│              └── invoke(초기상태)      ← 실행!            │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

(2) TypedDict - 상태의 타입 정의

TypedDict는 Python의 내장 기능으로, "이 딕셔너리에는 이런 키와 타입이 있다"를 선언하는 방법입니다.

```python
from typing import TypedDict

# 일반 딕셔너리 (타입 정보 없음)
state = {"message": "안녕", "count": 0}

# TypedDict (타입 정보 있음) - LangGraph에서 사용
class MyState(TypedDict):
    message: str      # message는 문자열
    count: int        # count는 정수
```


39

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

> **왜 TypedDict를 사용하나요?** - 상태에 어떤 데이터가 있는지 한눈에 파악 - 에디터에서 자동완성 지원 - 타입 실수를 미리 발견

```
┌──────────────────────────────────────────────────────────┐
│              TypedDict 시각적 이해                        │
│                                                          │
│   class AgentState(TypedDict):                           │
│       messages: list[str]    ← 대화 내역 (문자열 목록)    │
│       current_step: str      ← 현재 단계 (문자열)        │
│       result: str            ← 최종 결과 (문자열)        │
│                                                          │
│   이것은 마치 "양식지"와 같습니다:                         │
│                                                          │
│   ┌──────────────────────────────────┐                   │
│   │ [양식지]                          │                   │
│   │                                  │                   │
│   │ messages:     [_____________]    │                   │
│   │ current_step: [_____________]    │                   │
│   │ result:       [_____________]    │                   │
│   │                                  │                   │
│   │ → 각 노드가 이 양식지를 읽고 채움  │                   │
│   └──────────────────────────────────┘                   │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

(3) Node - 상태를 받아 변환하는 함수

노드는 **상태를 입력받아, 변경할 부분만 반환하는 함수**입니다.

```python
# 노드 함수의 기본 형태
def my_node(state: MyState) -> dict:
    # 1. 상태에서 필요한 데이터를 읽음
    current_message = state["message"]

    # 2. 처리 로직 수행
    new_message = current_message + " (처리됨)"

    # 3. 변경할 부분만 딕셔너리로 반환
    return {"message": new_message}
```


40

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
┌──────────────────────────────────────────────────────────┐
│                    노드의 동작 원리                       │
│                                                          │
│   상태 (입력)              노드 함수              상태 (갱신) │
│   ┌──────────────┐     ┌──────────────────┐     ┌──────────────┐│
│   │message: "안녕"│──▶│  처리 로직       │──▶│message:      ||
│   │count: 0      │     │  (변환, 계산)    │     | "안녕 (처리됨)"||
│   └──────────────┘     └──────────────────┘     |count: 0      ||
│                                                 └──────────────┘│
│                                                                 │
│   ※ 노드가 반환하지 않은 필드(count)는 그대로 유지됩니다      |
│                                                                 │
└──────────────────────────────────────────────────────────┘
```

> **핵심 규칙**: 노드 함수는 **변경할 필드만** 딕셔너리로 반환합니다. 반환하지 않은 필드는 이전 값이 그대로 유지됩니다.

(4) Edge - 노드 간 연결

엣지는 두 가지 종류가 있습니다:

일반 엣지: 항상 같은 다음 노드로 이동

```
# A 노드 다음에는 항상 B 노드 실행
graph.add_edge("A", "B")
```

조건부 엣지: 조건에 따라 다른 노드로 이동

```
# A 노드 후 routing_function의 결과에 따라 분기
graph.add_conditional_edges(
    "A",                        # 출발 노드
    routing_function,           # 분기 조건 함수
    {
        "positive": "B",        # 결과가 "positive"이면 B로
        "negative": "C",        # 결과가 "negative"이면 C로
    }
)
```


41

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
┌──────────────────────────────────────────────────────────┐
│                 엣지 종류 비교                              │
│                                                           │
│   일반 엣지:                                               │
│                                                           │
│   [노드A] ──────────────────▶ [노드B]                     │
│             (항상 B로 이동)                                 │
│                                                           │
│                                                           │
│   조건부 엣지:                                             │
│                                                           │
│                     ┌── "양성" ──▶ [노드B]                  │
│   [노드A] ──(판단)──┤                                      │
│                     └── "음성" ──▶ [노드C]                  │
│               (조건에 따라 분기)                            │
│                                                           │
└──────────────────────────────────────────────────────────┘
```

(5) END - 종료 노드

`END` 는 그래프의 실행을 종료하는 특별한 노드입니다.

```
from langgraph.graph import StateGraph, END

# 마지막 노드 다음에 종료
graph.add_edge("last_node", END)
```

---

2. 단계별 실습

첫 번째 그래프 만들기: "인사 처리기"

가장 간단한 2노드 그래프를 만들어봅시다.

```
    목표: 이름을 입력하면 인사말을 생성하는 그래프

    [START] → [인사생성] → [포맷팅] → [END]
```

Step 1: State 정의

파일을 생성합니다: `01_first_graph.py`

42

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
from typing import TypedDict
from langgraph.graph import StateGraph, END


# Step 1: 상태(State) 정의
# - 그래프 전체에서 공유할 데이터의 구조를 선언합니다
class GreetingState(TypedDict):
    name: str           # 사용자 이름
    greeting: str       # 생성된 인사말
    formatted: str      # 최종 포맷된 결과
```

> **이해 포인트**: `GreetingState` 는 "이 그래프에서 사용할 데이터는 name, greeting, formatted 세 가지입니다" 라고 선언하는 것입니다.

Step 2: 노드 함수 작성

```python
# Step 2: 노드 함수 작성
# - 각 노드는 state를 받아서, 변경할 부분만 딕셔너리로 반환합니다

def generate_greeting(state: GreetingState) -> dict:
    """인사말을 생성하는 노드"""
    name = state["name"]
    greeting = f"안녕하세요, {name}님!"
    print(f"  [인사생성 노드] '{name}' → '{greeting}'")
    return {"greeting": greeting}


def format_output(state: GreetingState) -> dict:
    """결과를 포맷팅하는 노드"""
    greeting = state["greeting"]
    formatted = f"=== {greeting} 좋은 하루 되세요! ==="
    print(f"  [포맷팅 노드] '{greeting}' → '{formatted}'")
    return {"formatted": formatted}
```

> **이해 포인트**: 각 함수는 `state` 에서 필요한 데이터를 읽고, 처리 결과를 딕셔너리로 반환합니다. 반환된 값만 상태가 업데이트됩니다.

Step 3: StateGraph 생성, 노드/엣지 추가

43

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
# Step 3: 그래프 생성 및 구성
# - StateGraph에 상태 타입을 전달하여 생성
# - 노드와 엣지를 추가하여 워크플로우를 정의

# 3-1: 그래프 생성 (상태 타입 지정)
graph = StateGraph(GreetingState)

# 3-2: 노드 추가 (이름, 함수)
graph.add_node("generate", generate_greeting)
graph.add_node("format", format_output)

# 3-3: 시작점 설정 (가장 먼저 실행할 노드)
graph.set_entry_point("generate")

# 3-4: 엣지 추가 (노드 간 연결)
graph.add_edge("generate", "format")    # generate 다음에 format 실행
graph.add_edge("format", END)           # format 다음에 종료
```

> **이해 포인트**: 마치 레고 블록을 조립하듯 노드를 추가하고 연결합니다. `set_entry_point` 는 "여기서 시작해!"를 지정하는 것입니다.

Step 4: 컴파일 및 실행

```
# Step 4: 컴파일 및 실행

# 4-1: 그래프 컴파일 (실행 가능한 형태로 변환)
app = graph.compile()

# 4-2: 초기 상태를 전달하여 실행
print("=== 그래프 실행 시작 ===")
result = app.invoke({
    "name": "홍길동",
    "greeting": "",
    "formatted": ""
})

# 4-3: 결과 확인
print("\n=== 실행 결과 ===")
print(f"최종 결과: {result['formatted']}")
```

Step 5: 결과 확인

전체 코드를 실행합니다:

```
python 01_first_graph.py
```

예상 출력:

44

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
=== 그래프 실행 시작 ===
  [인사생성 노드] '홍길동' → '안녕하세요, 홍길동님!'
  [포맷팅 노드] '안녕하세요, 홍길동님!' → '=== 안녕하세요, 홍길동님! 좋은 하루 되세요! ==='

=== 실행 결과 ===
최종 결과: === 안녕하세요, 홍길동님! 좋은 하루 되세요! ===
```

전체 코드 (복사용)

45

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
"""
01_first_graph.py - 첫 번째 LangGraph 그래프
목표: 이름을 입력하면 인사말을 생성하는 간단한 2노드 그래프
"""
from typing import TypedDict
from langgraph.graph import StateGraph, END


# 1. 상태 정의
class GreetingState(TypedDict):
    name: str
    greeting: str
    formatted: str


# 2. 노드 함수 작성
def generate_greeting(state: GreetingState) -> dict:
    """인사말을 생성하는 노드"""
    name = state["name"]
    greeting = f"안녕하세요, {name}님!"
    print(f"  [인사생성 노드] '{name}' → '{greeting}'")
    return {"greeting": greeting}


def format_output(state: GreetingState) -> dict:
    """결과를 포맷팅하는 노드"""
    greeting = state["greeting"]
    formatted = f"=== {greeting} 좋은 하루 되세요! ==="
    print(f"  [포맷팅 노드] '{greeting}' → '{formatted}'")
    return {"formatted": formatted}


# 3. 그래프 구성
graph = StateGraph(GreetingState)
graph.add_node("generate", generate_greeting)
graph.add_node("format", format_output)
graph.set_entry_point("generate")
graph.add_edge("generate", "format")
graph.add_edge("format", END)

# 4. 컴파일 및 실행
app = graph.compile()

print("=== 그래프 실행 시작 ===")
result = app.invoke({
    "name": "홍길동",
    "greeting": "",
    "formatted": ""
})

print("\n=== 실행 결과 ===")
print(f"최종 결과: {result['formatted']}")
```


46

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

그래프 실행 흐름 추적

위 코드가 실행될 때 내부에서 일어나는 일을 단계별로 살펴봅시다:

```
┌──────────────────────────────────────────────────────────┐
│                  실행 흐름 상세 추적                      │
│                                                          │
│  [1] invoke() 호출                                       │
│      초기 상태: {name: "홍길동", greeting: "", formatted: ""}│
│                                                          │
│  [2] entry_point → "generate" 노드 실행                  │
│      입력: {name: "홍길동", greeting: "", formatted: ""}  │
│      반환: {greeting: "안녕하세요, 홍길동님!"}            │
│      상태: {name: "홍길동",                              │
│             greeting: "안녕하세요, 홍길동님!",            │
│             formatted: ""}                               │
│                                                          │
│  [3] edge → "format" 노드 실행                           │
│      입력: {name: "홍길동",                              │
│             greeting: "안녕하세요, 홍길동님!",            │
│             formatted: ""}                               │
│      반환: {formatted: "=== 안녕하세요, ... ===="}        │
│      상태: {name: "홍길동",                              │
│             greeting: "안녕하세요, 홍길동님!",            │
│             formatted: "=== 안녕하세요, ... ===="}        │
│                                                          │
│  [4] edge → END (종료)                                   │
│      최종 상태 반환                                      │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

조건부 엣지 (add_conditional_edges)

이제 조건에 따라 다른 경로로 분기하는 그래프를 만들어봅시다.

파일: `02_conditional_graph.py`

```
목표: 점수에 따라 다른 메시지를 생성하는 그래프

                        ┌── 점수 >= 80 ──▶ [축하 메시지] ──┐
[START] → [점수확인] ──┤                                  ├──▶ [END]
                        └── 점수 <  80 ──▶ [격려 메시지] ──┘
```


47

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
"""
02_conditional_graph.py - 조건부 엣지 실습
목표: 점수에 따라 다른 메시지를 생성하는 그래프
"""
from typing import TypedDict
from langgraph.graph import StateGraph, END


# 1. 상태 정의
class ScoreState(TypedDict):
    student_name: str       # 학생 이름
    score: int              # 점수
    grade: str              # 등급 (pass/fail)
    message: str            # 결과 메시지


# 2. 노드 함수들
def check_score(state: ScoreState) -> dict:
    """점수를 확인하고 등급을 판정하는 노드"""
    score = state["score"]
    grade = "pass" if score >= 80 else "fail"
    print(f"  [점수확인] {state['student_name']}: {score}점 → {grade}")
    return {"grade": grade}


def congratulate(state: ScoreState) -> dict:
    """합격 축하 메시지를 생성하는 노드"""
    msg = f"축하합니다, {state['student_name']}님! {state['score']}점으로 합격입니다!"
    print(f"  [축하] {msg}")
    return {"message": msg}


def encourage(state: ScoreState) -> dict:
    """격려 메시지를 생성하는 노드"""
    msg = f"아쉽지만 {state['student_name']}님, {state['score']}점입니다. 다음에 화이팅!"
    print(f"  [격려] {msg}")
    return {"message": msg}


# 3. 라우팅 함수 (조건부 엣지에서 사용)
def route_by_grade(state: ScoreState) -> str:
    """등급에 따라 다음 노드를 결정하는 함수"""
    if state["grade"] == "pass":
        return "pass"
    else:
        return "fail"


# 4. 그래프 구성
graph = StateGraph(ScoreState)

# 노드 추가
graph.add_node("check", check_score)
graph.add_node("congratulate", congratulate)
```


48

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
graph.add_node("encourage", encourage)

# 시작점 설정
graph.set_entry_point("check")

# 조건부 엣지: check 노드 후 grade에 따라 분기
graph.add_conditional_edges(
    "check",            # 출발 노드
    route_by_grade,     # 라우팅 함수
    {
        "pass": "congratulate",     # "pass" → 축하 노드로
        "fail": "encourage",        # "fail" → 격려 노드로
    }
)

# 축하/격려 노드 → 종료
graph.add_edge("congratulate", END)
graph.add_edge("encourage", END)

# 5. 컴파일 및 실행
app = graph.compile()

# 테스트 1: 높은 점수
print("=== 테스트 1: 높은 점수 ===")
result1 = app.invoke({
    "student_name": "김철수",
    "score": 95,
    "grade": "",
    "message": ""
})
print(f"결과: {result1['message']}\n")

# 테스트 2: 낮은 점수
print("=== 테스트 2: 낮은 점수 ===")
result2 = app.invoke({
    "student_name": "이영희",
    "score": 65,
    "grade": "",
    "message": ""
})
print(f"결과: {result2['message']}")
```

실행 결과:

```
=== 테스트 1: 높은 점수 ===
  [점수확인] 김철수: 95점 → pass
  [축하] 축하합니다, 김철수님! 95점으로 합격입니다!
결과: 축하합니다, 김철수님! 95점으로 합격입니다!

=== 테스트 2: 낮은 점수 ===
  [점수확인] 이영희: 65점 → fail
  [격려] 아쉽지만 이영희님, 65점입니다. 다음에 화이팅!
결과: 아쉽지만 이영희님, 65점입니다. 다음에 화이팅!
```


49

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

라우팅 함수의 동작 원리

```
┌──────────────────────────────────────────────────────────┐
│                 라우팅 함수의 역할                          │
│                                                            │
│   check 노드 실행 후:                                       │
│   상태 = {student_name: "김철수", score: 95, grade: "pass"}│
│                                                            │
│        ↓                                                   │
│   route_by_grade(state) 호출                                │
│        ↓                                                   │
│   state["grade"] == "pass"  →  "pass" 반환                  │
│        ↓                                                   │
│   매핑: {"pass": "congratulate", "fail": "encourage"}       │
│        ↓                                                   │
│   "pass" → "congratulate" 노드 실행                         │
│                                                            │
│                                                            │
│   ※ 라우팅 함수는 반드시 매핑의 키 중 하나를 반환해야 합니다 │
│                                                            │
└──────────────────────────────────────────────────────────┘
```

Mermaid 시각화

LangGraph는 그래프를 Mermaid 다이어그램으로 자동 변환할 수 있습니다.

50

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
"""
03_visualize_graph.py - 그래프 시각화
"""
from typing import TypedDict
from langgraph.graph import StateGraph, END


class ScoreState(TypedDict):
    student_name: str
    score: int
    grade: str
    message: str


def check_score(state: ScoreState) -> dict:
    grade = "pass" if state["score"] >= 80 else "fail"
    return {"grade": grade}

def congratulate(state: ScoreState) -> dict:
    return {"message": "합격!"}

def encourage(state: ScoreState) -> dict:
    return {"message": "다음 기회에!"}

def route_by_grade(state: ScoreState) -> str:
    return "pass" if state["grade"] == "pass" else "fail"


graph = StateGraph(ScoreState)
graph.add_node("check", check_score)
graph.add_node("congratulate", congratulate)
graph.add_node("encourage", encourage)
graph.set_entry_point("check")
graph.add_conditional_edges("check", route_by_grade, {
    "pass": "congratulate",
    "fail": "encourage",
})
graph.add_edge("congratulate", END)
graph.add_edge("encourage", END)

app = graph.compile()

# Mermaid 다이어그램 출력
mermaid_code = app.get_graph().draw_mermaid()
print("=== Mermaid 다이어그램 ===")
print(mermaid_code)
```

출력되는 Mermaid 코드를 Mermaid Live Editor에 붙여넣으면 시각적인 다이어그램을 확인할 수 있습니다.

51

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
┌──────────────────────────────────────────────────────────┐
│                  시각화 활용 방법                          │
│                                                          │
│   1. 위 코드 실행하여 Mermaid 코드 복사                    │
│   2. https://mermaid.live/ 접속                           │
│   3. 왼쪽 에디터에 Mermaid 코드 붙여넣기                   │
│   4. 오른쪽에서 그래프 다이어그램 확인                      │
│                                                          │
│   ※ VS Code 사용자는 "Mermaid Preview" 확장을 설치하면     │
│     에디터에서 바로 확인할 수 있습니다                      │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

---

3. 실전 예제

3-노드 "텍스트 분류기"

실전에 가까운 예제를 만들어봅시다. 입력된 텍스트의 주제를 분류하는 그래프입니다.

```
전체 흐름:
[START] → [입력처리] → [분류] → [결과포맷] → [END]
```

파일: `04_text_classifier.py`

52

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
"""
04_text_classifier.py - 텍스트 분류기 (3노드 그래프)
목표: 입력 텍스트를 분석하여 카테고리를 분류하고 결과를 포맷팅
"""
from typing import TypedDict
from langgraph.graph import StateGraph, END


# ========================================
# 1. 상태 정의
# ========================================
class ClassifierState(TypedDict):
    raw_text: str           # 원본 입력 텍스트
    cleaned_text: str       # 전처리된 텍스트
    category: str           # 분류 결과
    confidence: float       # 신뢰도 (0.0 ~ 1.0)
    result_report: str      # 최종 결과 리포트


# ========================================
# 2. 카테고리 키워드 사전
# ========================================
# 실제 프로덕션에서는 LLM이 이 역할을 합니다.
# 여기서는 학습 목적으로 키워드 매칭을 사용합니다.
CATEGORY_KEYWORDS = {
    "기술": ["프로그래밍", "코드", "개발", "서버", "API", "데이터베이스",
             "Python", "JavaScript", "버그", "배포"],
    "비즈니스": ["매출", "수익", "마케팅", "전략", "투자", "시장",
                 "고객", "영업", "경영", "계약"],
    "건강": ["운동", "식단", "다이어트", "병원", "건강", "수면",
            "스트레스", "영양", "체중", "의사"],
    "교육": ["학습", "강의", "시험", "학교", "교수", "수업",
            "과제", "학생", "교육", "공부"],
}


# ========================================
# 3. 노드 함수들
# ========================================
def preprocess(state: ClassifierState) -> dict:
    """입력 텍스트를 전처리하는 노드"""
    raw = state["raw_text"]

    # 공백 정리 및 소문자 변환 (한국어는 소문자 개념 없음)
    cleaned = raw.strip()
    # 불필요한 연속 공백 제거
    cleaned = " ".join(cleaned.split())

    print(f"  [전처리] 원본 길이: {len(raw)} → 정리 후: {len(cleaned)}")
    return {"cleaned_text": cleaned}


def classify(state: ClassifierState) -> dict:
    """텍스트를 카테고리로 분류하는 노드"""
```


53

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
    text = state["cleaned_text"]

    # 각 카테고리의 키워드 매칭 점수 계산
    scores = {}
    for category, keywords in CATEGORY_KEYWORDS.items():
        # 텍스트에 포함된 키워드 수 카운트
        match_count = sum(1 for kw in keywords if kw in text)
        scores[category] = match_count

    # 최고 점수 카테고리 선택
    if max(scores.values()) == 0:
        best_category = "기타"
        confidence = 0.0
    else:
        best_category = max(scores, key=scores.get)
        total_keywords = len(CATEGORY_KEYWORDS[best_category])
        confidence = round(scores[best_category] / total_keywords, 2)

    print(f"  [분류] 점수: {scores}")
    print(f"  [분류] 결과: {best_category} (신뢰도: {confidence})")
    return {
        "category": best_category,
        "confidence": confidence
    }


def format_result(state: ClassifierState) -> dict:
    """결과를 보기 좋게 포맷팅하는 노드"""
    report = (
        f"┌{'─' * 40}┐\n"
        f"│ 텍스트 분류 결과                          │\n"
        f"├{'─' * 40}┤\n"
        f"│ 입력: {state['cleaned_text'][:30]:30s}  │\n"
        f"│ 카테고리: {state['category']:28s}  │\n"
        f"│ 신뢰도: {state['confidence']:<30} │\n"
        f"└{'─' * 40}┘"
    )
    print(f"  [포맷] 리포트 생성 완료")
    return {"result_report": report}


# ========================================
# 4. 그래프 구성
# ========================================
graph = StateGraph(ClassifierState)

# 노드 추가
graph.add_node("preprocess", preprocess)
graph.add_node("classify", classify)
graph.add_node("format", format_result)

# 시작점 설정
graph.set_entry_point("preprocess")

# 엣지 연결: 전처리 → 분류 → 포맷 → 종료
```


54

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
graph.add_edge("preprocess", "classify")
graph.add_edge("classify", "format")
graph.add_edge("format", END)

# 컴파일
app = graph.compile()


# ========================================
# 5. 테스트 실행
# ========================================
test_texts = [
    "Python 프로그래밍으로 API 서버를 개발하고 배포했습니다",
    "올해 매출이 전년 대비 20% 증가하여 마케팅 전략을 수정합니다",
    "매일 30분 운동과 균형 잡힌 식단으로 건강을 관리하세요",
    "학생들의 학습 효과를 높이기 위해 새로운 교육 방법을 도입합니다",
]

for i, text in enumerate(test_texts, 1):
    print(f"\n{'='*50}")
    print(f"테스트 {i}: '{text[:40]}...'")
    print(f"{'='*50}")

    result = app.invoke({
        "raw_text": text,
        "cleaned_text": "",
        "category": "",
        "confidence": 0.0,
        "result_report": ""
    })

    print(f"\n{result['result_report']}")
```

실행:

```
python 04_text_classifier.py
```

---

4. 연습 문제

연습 1: 기본 그래프 수정

위의 "인사 처리기" (`01_first_graph.py`)를 수정하여 다음 기능을 추가해보세요:

1. 상태에 `language` 필드를 추가 (값: "ko" 또는 "en")
2. 새로운 노드 `detect_language` 를 추가하여 이름에 한글이 포함되면 "ko", 아니면 "en"으로 설정
3. 실행 순서: `detect_language` → `generate` → `format` → END

55

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

4. `generate` 노드에서 language에 따라 한국어/영어 인사말 생성

힌트:

```
# 한글 포함 여부 확인
def has_korean(text):
    for char in text:
        if '가' <= char <= '힣':
            return True
    return False
```

▶ 정답 코드 (클릭하여 펼치기)

연습 2: 조건부 엣지 추가

다음 요구사항을 가진 그래프를 처음부터 만들어보세요:

"주문 처리 그래프"

상태: - `item_name`: 상품명 (str) - `quantity`: 수량 (int) - `in_stock`: 재고 여부 (bool) - `order_status`: 주문 상태 (str) - `message`: 결과 메시지 (str)

노드: - `check_stock`: 수량이 10 이하면 재고 있음(True), 10 초과면 재고 없음(False) 설정 - `process_order`: 주문 처리 (order_status = "완료") - `backorder`: 입고 대기 처리 (order_status = "입고대기")

흐름:

```
[START] → [check_stock] ──(재고있음)──▶ [process_order] ──▶ [END]
                        └─(재고없음)──▶ [backorder] ────▶ [END]
```

▶ 정답 코드 (클릭하여 펼치기)

연습 3: 4-노드 "감정 분석기" 만들기 (도전!)

다음 요구사항으로 4노드 그래프를 만들어보세요:

"텍스트 감정 분석기"

상태: - `raw_text`: 원본 텍스트 - `cleaned_text`: 전처리된 텍스트 - `sentiment`: 감정 ("긍정", "부정", "중립") - `keywords`: 감지된 키워드 목록 - `report`: 최종 분석 리포트

노드: 1. `preprocess`: 텍스트 전처리 (공백 정리, 소문자 변환) 2. `analyze_sentiment`: 키워드 기반 감정 분석 - 긍정 키워드: 좋다, 훌륭, 최고, 행복, 감사, 사랑, 만족, 추천, 완벽, 대박 - 부정 키워드: 나쁘다, 최악, 불만, 화남, 실망, 싫다, 후회, 별로, 짜증, 불편 3. `generate_report`: 분석 리포트 생성 4. `add_recommendation`: 감정에 따른 추천 메시지 추가 (조건부 엣지 사용)

흐름:

56

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
[START] → [preprocess] → [analyze_sentiment] → [generate_report]
                                                        │
                                        ┌── 긍정 ──▶ [감사 메시지] ──┐
                                        ├── 부정 ──▶ [개선 안내] ────┼──▶ [END]
                                        └── 중립 ──▶ [기본 안내] ────┘
```

힌트: `add_recommendation` 대신 3개의 별도 노드(`thank` , `improve` , `default_msg`)를 만들고 조건부 엣지로 분기하세요.

▶ 정답 코드 (클릭하여 펼치기)

---

5. 핵심 정리

이 모듈에서 배운 것

```
┌──────────────────────────────────────────────────────────────┐
│                    Module 02 핵심 정리                       │
│                                                              │
│  1. LangGraph = 워크플로우를 그래프로 표현하는 프레임워크     │
│                                                              │
│  2. 핵심 요소:                                               │
│     - StateGraph: 그래프 객체 생성                           │
│     - TypedDict: 상태 타입 선언                              │
│     - Node: state → dict 형태의 함수                         │
│     - Edge: 노드 간 연결 (일반/조건부)                       │
│     - END: 그래프 종료 지점                                  │
│                                                              │
│  3. 그래프 생성 5단계:                                       │
│     Step 1: TypedDict로 State 정의                           │
│     Step 2: 노드 함수 작성                                   │
│     Step 3: StateGraph 생성 + 노드/엣지 추가                 │
│     Step 4: compile()로 컴파일                               │
│     Step 5: invoke()로 실행                                  │
│                                                              │
│  4. 조건부 엣지:                                             │
│     - add_conditional_edges(출발, 라우팅함수, 매핑)         │
│     - 라우팅 함수는 매핑의 키 중 하나를 반환                 │
│                                                              │
│  5. 시각화:                                                  │
│     - app.get_graph().draw_mermaid()                         │
│     - Mermaid Live Editor에서 확인                           │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

자주 하는 실수

57

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

실수	해결 방법
노드 함수에서 전체 상태를 반환	변경할 필드만 딕셔너리로 반환
set_entry_point를 빼먹음	반드시 시작 노드를 지정해야 함
END를 연결하지 않음	마지막 노드에 END 엣지 필수
조건부 엣지 매핑에 없는 키 반환	라우팅 함수가 매핑 키 중 하나를 반환하는지 확인
compile() 전에 invoke() 호출	반드시 compile() 후 invoke()

코드 패턴 요약

```python
# === LangGraph 기본 패턴 ===
from typing import TypedDict
from langgraph.graph import StateGraph, END

# 1. 상태 정의
class MyState(TypedDict):
    field1: str
    field2: int

# 2. 노드 함수
def my_node(state: MyState) -> dict:
    return {"field1": "새 값"}

# 3. 라우팅 함수 (조건부 엣지용)
def my_router(state: MyState) -> str:
    return "option_a" if state["field2"] > 0 else "option_b"

# 4. 그래프 구성
graph = StateGraph(MyState)
graph.add_node("node_name", my_node)
graph.set_entry_point("node_name")
graph.add_edge("node_name", END)
# 또는 조건부: graph.add_conditional_edges("node", my_router, {...})

# 5. 컴파일 및 실행
app = graph.compile()
result = app.invoke({"field1": "", "field2": 0})
```

---

6. 참고 자료

필수 읽기

58

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

자료	링크	설명
LangGraph 빠른 시작	langgraph tutorials	공식 튜토리얼로 시작하기
LangGraph 핵심 개념	langgraph concepts	StateGraph, Node, Edge 공식 설명
Python TypedDict	python docs	TypedDict 공식 문서

추가 학습

자료	링크	설명
LangGraph How-to Guides	langgraph how-to	상황별 구현 가이드
Mermaid Live Editor	mermaid.live	그래프 시각화 온라인 도구
LangGraph GitHub	github.com/langchain-ai/langgraph	소스 코드 및 예제
LangChain Academy - LangGraph	academy.langchain.com	공식 교육 과정 (영문)

API 레퍼런스

클래스/함수	설명
`StateGraph(state_type)`	상태 그래프 생성
`graph.add_node(name, func)`	노드 추가
`graph.add_edge(from, to)`	일반 엣지 추가
`graph.add_conditional_edges(from, router, mapping)`	조건부 엣지 추가
`graph.set_entry_point(name)`	시작 노드 설정
`graph.compile()`	그래프 컴파일
`app.invoke(initial_state)`	그래프 실행
`app.get_graph().draw_mermaid()`	Mermaid 다이어그램 생성

다음 단계

축하합니다! Module 02를 완료했습니다.

59

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

이제 LangGraph의 기본 구조를 이해하고 직접 그래프를 만들 수 있습니다. 다음 모듈에서는 상태 관리를 더 깊이 배웁니다:

> **Module 03: State 관리 심화 - Reducer와 체크포인트** (예정)
>
> - Annotation과 Reducer 패턴
> - 리스트 상태에 값 누적하기
> - 체크포인트로 상태 저장/복구
> - 실전: 대화 히스토리 관리

기초가 탄탄해야 응용이 가능합니다. 연습 문제를 모두 풀어보셨나요?

60

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

⬆ 목차로 돌아가기

Module 03: Jupyter 노트북 기반 에이전트 개발 환경 구축

학습 목표

이 모듈을 완료하면 다음을 할 수 있습니다:

- Jupyter 노트북 환경에서 LangGraph 에이전트를 대화형으로 개발하고 테스트할 수 있다
- FakeLLM을 만들어 API 키 없이, 비용 없이 에이전트를 실행할 수 있다
- Mermaid 다이어그램으로 에이전트 그래프를 시각적으로 확인할 수 있다
- 개별 노드를 독립적으로 호출하여 디버깅할 수 있다
- Mock 객체로 외부 API를 대체하는 테스트 패턴을 적용할 수 있다

사전 지식

항목	수준	비고
Python 기초	필수	함수, 클래스, 딕셔너리 사용법
Module 01	필수	LangGraph 기본 개념 (노드, 엣지, State)
Module 02	권장	LangGraph 설치 및 첫 그래프 실행 경험
Jupyter Notebook	선택	사용 경험이 없어도 이 모듈에서 배울 수 있음

---

1. 개념 설명

1.1 왜 Jupyter 노트북으로 에이전트를 개발하나?

일반적인 에이전트 개발 과정을 생각해 봅시다:

61

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
[기존 방식]
코드 수정 → 터미널에서 python main.py 실행 → 전체 파이프라인 실행 → 로그 확인
                                                ↑ 오래 걸림          ↑ 읽기 어려움
```

```
[Jupyter 방식]
셀에서 코드 수정 → Shift+Enter로 실행 → 즉시 결과 확인 → 그래프 시각화
                    ↑ 수 초 이내          ↑ 인라인 출력    ↑ 다이어그램 렌더링
```

Jupyter 노트북이 에이전트 개발에 적합한 4가지 이유:

장점	설명
빠른 피드백 루프	코드 변경 후 해당 셀만 실행하면 됨. 전체 프로그램을 다시 시작할 필요 없음
시각적 디버깅	그래프 다이어그램, state 변화, LLM 응답을 인라인으로 바로 확인
재현 가능한 테스트	FakeLLM과 fixture를 사용하면 동일 입력 → 동일 출력이 항상 보장됨
외부 의존성 제거	실제 LLM API, 데이터베이스, 외부 서비스 없이 로컬에서 완전히 동작

1.2 Jupyter 노트북이란?

Jupyter 노트북은 코드, 실행 결과, 설명 텍스트를 하나의 문서에 담는 대화형 개발 환경입니다. 코드를 "셀(cell)" 단위로 나누어 실행하고, 각 셀의 결과를 바로 아래에서 확인할 수 있습니다.

```
┌──────────────────────────────────┐
│ [Cell 1] Python 코드             │  ← 코드 셀
│ x = 10                           │
│ print(x * 2)                     │
├──────────────────────────────────┤
│ 출력: 20                         │  ← 실행 결과
├──────────────────────────────────┤
│ [Cell 2] 마크다운 설명           │  ← 마크다운 셀
│ ## 다음 단계                     │
│ 아래에서 그래프를 만듭니다.      │
├──────────────────────────────────┤
│ [Cell 3] Python 코드             │  ← 코드 셀
│ graph = build_graph()            │
│ graph.invoke(...)                │
├──────────────────────────────────┤
│ 출력: {'result': '분석 완료'}    │  ← 실행 결과
└──────────────────────────────────┘
```


62

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

1.3 FakeLLM이란?

FakeLLM은 실제 LLM(ChatGPT, Claude 등) 대신 사용하는 **가짜 LLM 객체**입니다. 미리 정해둔 규칙에 따라 응답을 반환합니다.

```
[실제 LLM]
질문 → 인터넷 → OpenAI/Anthropic 서버 → 응답 (비용 발생, API 키 필요)

[FakeLLM]
질문 → 패턴 매칭 → 미리 준비한 응답 반환 (무료, API 키 불필요)
```

왜 FakeLLM을 사용하나요?

- **비용 절감**: 개발 중에 수십~수백 번 실행해도 비용이 0원
- **API 키 불필요**: 환경 설정 없이 바로 시작 가능
- **일관된 결과**: 같은 입력에 항상 같은 출력 (디버깅이 쉬움)
- **오프라인 가능**: 인터넷 연결 없이도 개발 가능

---

2. 단계별 실습

2.1 Python 가상환경 생성 및 활성화

프로젝트별로 독립적인 Python 환경을 만들어야 패키지 충돌을 방지할 수 있습니다.

```python
# 1. 프로젝트 디렉토리 생성
mkdir my-agent-notebook
cd my-agent-notebook

# 2. 가상환경 생성 (Python 3.11 이상 권장)
python -m venv .venv

# 3. 가상환경 활성화
# macOS / Linux:
source .venv/bin/activate
# Windows:
# .venv\Scripts\activate

# 활성화 확인 (가상환경 경로가 출력되어야 함)
which python
# 출력 예시: /Users/yourname/my-agent-notebook/.venv/bin/python
```

> **팁**: 터미널 프롬프트 앞에 `(.venv)` 가 표시되면 가상환경이 활성화된 것입니다.

63

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

2.2 Jupyter + ipykernel 설치

```
# Jupyter 노트북과 커널 패키지 설치
pip install jupyter ipykernel ipython

# 설치 확인
jupyter --version
```

2.3 LangGraph 관련 패키지 설치

```
# LangGraph 핵심 패키지
pip install langgraph langchain-core

# 시각화 및 유틸리티
pip install ipython

# (선택) state diff 비교용
pip install deepdiff
```

2.4 Jupyter 커널 등록

가상환경을 Jupyter 커널로 등록하면, 노트북에서 이 환경의 패키지를 사용할 수 있습니다.

```
# 커널 등록
python -m ipykernel install --user \
    --name my-agent-dev \
    --display-name "My Agent Dev (Python 3.12)"

# 등록된 커널 목록 확인
jupyter kernelspec list
```

2.5 Jupyter 노트북 시작

```
# Jupyter 노트북 서버 실행
jupyter notebook

# 또는 JupyterLab (더 현대적인 인터페이스)
# pip install jupyterlab
# jupyter lab
```

브라우저가 자동으로 열립니다. 오른쪽 위 “New” 버튼을 클릭하고, 방금 등록한 커널 “My Agent Dev (Python 3.12)”을 선택하여 새 노트북을 만듭니다.

64

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

3. 실전 예제

3.1 FakeLLM 클래스 만들기

아래 코드를 노트북의 첫 번째 셀에 입력하고 실행합니다. 이것이 우리가 사용할 FakeLLM입니다.

65

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# ── Cell 1: FakeLLM 정의 ──

from langchain_core.language_models import BaseChatModel
from langchain_core.messages import AIMessage, BaseMessage
from typing import Any, Optional


class FakeLLM(BaseChatModel):
    """패턴 매칭 기반 가짜 LLM.

    미리 정의한 패턴-응답 매핑에 따라 응답을 반환합니다.
    실제 LLM API를 호출하지 않으므로 비용이 발생하지 않습니다.
    """

    responses: dict[str, str] = {}
    default_response: str = "이것은 FakeLLM의 기본 응답입니다."

    @property
    def _llm_type(self) -> str:
        return "fake-llm"

    def _generate(self, messages: list[BaseMessage], **kwargs) -> Any:
        """메시지 내용을 패턴 매칭하여 적절한 응답을 반환합니다."""
        from langchain_core.outputs import ChatGeneration, ChatResult

        # 마지막 메시지의 내용을 확인
        last_message = messages[-1].content if messages else ""

        # 패턴 매칭: responses 딕셔너리에서 키워드 검색
        matched_response = self.default_response
        for pattern, response in self.responses.items():
            if pattern.lower() in last_message.lower():
                matched_response = response
                break

        generation = ChatGeneration(
            message=AIMessage(content=matched_response)
        )
        return ChatResult(generations=[generation])


# FakeLLM 인스턴스 생성
fake_llm = FakeLLM(
    responses={
        "분석": "분석 결과: 이 문서는 Python 웹 프레임워크에 관한 기술 문서입니다.",
        "요약": "요약: 본 문서는 FastAPI를 활용한 REST API 구축 방법을 설명합니다.",
        "번역": "Translation: This document explains how to build REST APIs.",
    },
    default_response="처리되었습니다. 추가 정보가 필요하면 알려주세요.",
)

# 테스트
response = fake_llm.invoke("이 문서를 분석해주세요")
```


66

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
print(f"응답: {response.content}")
# 출력: 응답: 분석 결과: 이 문서는 Python 웹 프레임워크에 관한 기술 문서입니다.
```

3.2 Fixture 파일로 테스트 데이터 관리

패턴-응답 매핑이 많아지면 코드에 직접 넣기 어렵습니다. JSON 파일로 분리하여 관리합시다.

67

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# ── Cell 2: Fixture 파일 생성 및 로드 ──

import json
from pathlib import Path

# fixture 디렉토리 생성
fixtures_dir = Path("fixtures")
fixtures_dir.mkdir(exist_ok=True)

# fixture 파일 작성
fixture_data = {
    "analyze": {
        "pattern": "분석",
        "response": "분석 결과: 주요 키워드는 [Python, FastAPI, REST]입니다. 기술 문서로
        분류됩니다."
    },
    "summarize": {
        "pattern": "요약",
        "response": "요약: 본 문서는 3개 섹션으로 구성되며, 핵심 내용은 API 설계 원칙입니다."
    },
    "classify": {
        "pattern": "분류",
        "response": "분류 결과: 카테고리=기술문서, 난이도=중급, 대상=백엔드 개발자"
    }
}

fixture_path = fixtures_dir / "llm_responses.json"
with open(fixture_path, "w", encoding="utf-8") as f:
    json.dump(fixture_data, f, ensure_ascii=False, indent=2)

print(f"Fixture 파일 생성 완료: {fixture_path}")


def load_fixtures(fixture_file: str = "fixtures/llm_responses.json") -> dict[str, str]:
    """Fixture JSON 파일을 로드하여 패턴-응답 딕셔너리로 변환합니다."""
    with open(fixture_file, "r", encoding="utf-8") as f:
        data = json.load(f)

    return {
        item["pattern"]: item["response"]
        for item in data.values()
    }


# fixture 기반 FakeLLM 생성
responses = load_fixtures()
fake_llm = FakeLLM(responses=responses)

# 테스트
result = fake_llm.invoke("이 텍스트를 분류해주세요")
print(f"응답: {result.content}")
# 출력: 응답: 분류 결과: 카테고리=기술문서, 난이도=중급, 대상=백엔드 개발자
```


68

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

3.3 노트북에서 LangGraph 그래프 만들고 실행하기

이제 FakeLLM을 활용하여 실제 LangGraph 그래프를 노트북에서 실행해 봅시다.

69

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# ── Cell 3: 간단한 3-노드 그래프 정의 ──

from typing import TypedDict
from langgraph.graph import StateGraph, END


# 1. State 정의
class TextAnalysisState(TypedDict):
    """텍스트 분석 에이전트의 상태"""
    input_text: str           # 입력 텍스트
    analysis: str | None      # 분석 결과
    summary: str | None       # 요약 결과
    current_step: str         # 현재 단계


# 2. 노드 함수 정의
def analyze_text(state: TextAnalysisState) -> dict:
    """텍스트를 분석하는 노드"""
    response = fake_llm.invoke(f"다음 텍스트를 분석해주세요: {state['input_text']}")
    return {
        "analysis": response.content,
        "current_step": "analyzed",
    }


def summarize_text(state: TextAnalysisState) -> dict:
    """분석 결과를 요약하는 노드"""
    response = fake_llm.invoke(f"다음 분석 결과를 요약해주세요: {state['analysis']}")
    return {
        "summary": response.content,
        "current_step": "summarized",
    }


def format_result(state: TextAnalysisState) -> dict:
    """최종 결과를 포맷하는 노드"""
    formatted = f"[분석 결과]\n{state['analysis']}\n\n[요약]\n{state['summary']}"
    return {
        "summary": formatted,
        "current_step": "completed",
    }


# 3. 그래프 조립
graph_builder = StateGraph(TextAnalysisState)

# 노드 추가
graph_builder.add_node("analyze", analyze_text)
graph_builder.add_node("summarize", summarize_text)
graph_builder.add_node("format", format_result)

# 엣지 연결
graph_builder.set_entry_point("analyze")
graph_builder.add_edge("analyze", "summarize")
```


70

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
graph_builder.add_edge("summarize", "format")
graph_builder.add_edge("format", END)

# 컴파일
graph = graph_builder.compile()

print("그래프 빌드 완료!")
```

```
# ── Cell 4: 그래프 실행 ──

import json

# 초기 상태 설정
initial_state: TextAnalysisState = {
    "input_text": "FastAPI는 Python으로 만든 고성능 웹 프레임워크입니다.",
    "analysis": None,
    "summary": None,
    "current_step": "start",
}

# stream 모드로 실행 (각 노드의 출력을 순차적으로 확인)
print("=" * 60)
print("그래프 실행 시작")
print("=" * 60)

for event in graph.stream(initial_state):
    node_name = list(event.keys())[0]
    node_output = event[node_name]
    print(f"\n>> 노드: {node_name}")
    print(f"   현재 단계: {node_output.get('current_step', 'N/A')}")
    for key, value in node_output.items():
        if key != "current_step" and value is not None:
            print(f"   {key}: {str(value)[:100]}")
    print("-" * 60)

print("\n실행 완료!")
```

3.4 그래프 시각화: Mermaid 다이어그램

LangGraph는 그래프 구조를 Mermaid 다이어그램으로 출력할 수 있습니다. 이를 통해 에이전트의 흐름을 시각적으로 파악할 수 있습니다.

71

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
# ── Cell 5: Mermaid 텍스트 출력 ──

# 방법 1: Mermaid 텍스트 직접 출력
mermaid_text = graph.get_graph().draw_mermaid()
print(mermaid_text)

# 출력 예시:
# %%{init: {'flowchart': {'curve': 'linear'}}}%%
# graph TD;
#     __start__([start]):::first
#     analyze(analyze)
#     summarize(summarize)
#     format(format)
#     __end__([end]):::last
#     __start__ --> analyze;
#     analyze --> summarize;
#     summarize --> format;
#     format --> __end__;
```

```
# ── Cell 6: Mermaid 다이어그램 렌더링 ──

from IPython.display import display, Image

# 방법 2: PNG 이미지로 렌더링 (노트북에 인라인 표시)
try:
    png_bytes = graph.get_graph().draw_mermaid_png()
    display(Image(png_bytes))
    print("그래프 다이어그램이 위에 표시됩니다.")
except Exception as e:
    # mermaid-cli가 없는 경우 Mermaid Live Editor 링크 제공
    import base64
    encoded = base64.urlsafe_b64encode(mermaid_text.encode()).decode()
    print(f"PNG 렌더링 실패: {e}")
    print(f"\n아래 링크를 브라우저에서 열면 다이어그램을 볼 수 있습니다:")
    print(f"https://mermaid.live/edit#base64:{encoded}")
```

```
# ── Cell 7: 노트북에서 Mermaid 마크다운으로 표시 ──

from IPython.display import display, Markdown

# 방법 3: Jupyter에서 마크다운으로 렌더링
# (JupyterLab에서는 Mermaid를 기본 지원합니다)
display(Markdown(f"```mermaid\n{mermaid_text}\n```"))
```

3.5 노드별 디버깅: 개별 노드 독립 호출

전체 그래프를 실행하지 않고, 특정 노드만 따로 호출하여 테스트할 수 있습니다. LangGraph의 노드는 본질적으로 `(state) -> dict` 형태의 일반 Python 함수이기 때문입니다.

72

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# ── Cell 8: 개별 노드 독립 테스트 ──

# analyze_text 노드만 따로 실행
test_state: TextAnalysisState = {
    "input_text": "머신러닝은 데이터에서 패턴을 학습하는 AI 기술입니다.",
    "analysis": None,
    "summary": None,
    "current_step": "start",
}

# 노드 함수를 직접 호출
result = analyze_text(test_state)
print("=== analyze_text 노드 출력 ===")
print(f"analysis: {result['analysis']}")
print(f"current_step: {result['current_step']}")

# summarize_text 노드를 테스트하려면 analysis가 채워진 state가 필요
test_state_2: TextAnalysisState = {
    "input_text": "머신러닝은 데이터에서 패턴을 학습하는 AI 기술입니다.",
    "analysis": "분석 결과: AI/ML 관련 기술 문서",
    "summary": None,
    "current_step": "analyzed",
}

result_2 = summarize_text(test_state_2)
print("\n=== summarize_text 노드 출력 ===")
print(f"summary: {result_2['summary']}")
```

3.6 State Diff 시각화 (Before/After 비교)

노드 실행 전후의 상태 변화를 한눈에 보여주는 유틸리티를 만들어 봅시다.

73

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# ── Cell 9: State Diff 유틸리티 ──

def show_state_diff(before: dict, node_output: dict, node_name: str = ""):
    """노드 실행 전후의 상태 변경을 시각적으로 출력합니다.

    Args:
        before: 노드 실행 전 state
        node_output: 노드가 반환한 dict (변경된 필드만 포함)
        node_name: 노드 이름 (표시용)
    """
    header = f"State Diff: {node_name}" if node_name else "State Diff"
    print(f"\n{'=' * 60}")
    print(f"  {header}")
    print(f"{'=' * 60}")

    changes_found = False

    for key, new_value in node_output.items():
        old_value = before.get(key)
        if old_value != new_value:
            changes_found = True
            old_display = _truncate(str(old_value))
            new_display = _truncate(str(new_value))
            print(f"\n  [변경] {key}:")
            print(f"    이전: {old_display}")
            print(f"    이후: {new_display}")

    if not changes_found:
        print("\n  (변경사항 없음)")

    print(f"\n{'=' * 60}\n")


def _truncate(text: str, max_len: int = 80) -> str:
    """긴 텍스트를 잘라서 보기 좋게 표시합니다."""
    return text[:max_len] + "..." if len(text) > max_len else text


# 사용 예시
before_state: TextAnalysisState = {
    "input_text": "FastAPI는 Python 웹 프레임워크입니다.",
    "analysis": None,
    "summary": None,
    "current_step": "start",
}

node_result = analyze_text(before_state)
show_state_diff(before_state, node_result, "analyze_text")

# 출력 예시:
# ============================================================
#   State Diff: analyze_text
# ============================================================
#
```


74

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
#   [변경] analysis:
#     이전: None
#     이후: 분석 결과: 주요 키워드는 [Python, FastAPI, REST]입니다. ...
#
#   [변경] current_step:
#     이전: start
#     이후: analyzed
#
# ============================================================
```

3.7 Mock 객체로 외부 시스템 대체하기

에이전트가 외부 API(데이터베이스, REST API 등)를 호출할 때, Mock 객체로 대체하면 외부 시스템 없이 테스트할 수 있습니다.

```python
# ── Cell 10: unittest.mock 기초 ──

from unittest.mock import MagicMock, patch

# 예시: 외부 API를 호출하는 함수
def fetch_document_from_api(doc_id: str) -> dict:
    """외부 API에서 문서를 가져오는 함수 (원래는 HTTP 요청 필요)"""
    import httpx
    response = httpx.get(f"https://api.example.com/docs/{doc_id}")
    return response.json()


# Mock으로 대체: 실제 HTTP 요청 없이 테스트
mock_api = MagicMock()
mock_api.return_value = {
    "id": "doc-001",
    "title": "Python 가이드",
    "content": "Python은 배우기 쉬운 프로그래밍 언어입니다.",
    "author": "홍길동",
}

# Mock 함수 호출
result = mock_api("doc-001")
print(f"Mock API 응답: {result['title']}")
print(f"호출 횟수: {mock_api.call_count}")
print(f"호출 인자: {mock_api.call_args}")
```


75

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# ── Cell 11: 에이전트 노드에서 Mock 활용 ──

from functools import partial
from typing import TypedDict

# 외부 도구에 의존하는 노드 함수
def fetch_document(state: dict, api_client=None) -> dict:
    """문서를 가져오는 노드.

    api_client를 외부에서 주입받으므로, 테스트 시 Mock으로 교체 가능합니다.
    """
    doc = api_client(state["doc_id"])
    return {
        "document": doc,
        "current_step": "fetched",
    }


# 실제 사용 시: 진짜 API 클라이언트 주입
# real_fetch = partial(fetch_document, api_client=real_api_client)

# 테스트 시: Mock 주입
mock_client = MagicMock(return_value={
    "id": "doc-001",
    "title": "테스트 문서",
    "content": "이것은 테스트용 문서입니다.",
})

test_fetch = partial(fetch_document, api_client=mock_client)

# 노드 실행
result = test_fetch({"doc_id": "doc-001", "document": None, "current_step": "start"})
print(f"가져온 문서: {result['document']['title']}")

# Mock 호출 검증
mock_client.assert_called_once_with("doc-001")
print("Mock 검증 통과! (정확히 1번 호출됨)")
```

3.8 노트북 표준 템플릿

매번 노트북을 만들 때마다 같은 설정을 반복하지 않도록, 표준 템플릿을 정해 둡시다.

76

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# ── 표준 템플릿: Cell 1 - 환경 초기화 ──

# 필수 import
from typing import TypedDict
from langgraph.graph import StateGraph, END
from langchain_core.language_models import BaseChatModel
from langchain_core.messages import AIMessage, BaseMessage
from langchain_core.outputs import ChatGeneration, ChatResult
from IPython.display import display, Image, Markdown
from functools import partial
from unittest.mock import MagicMock
import json

print("환경 초기화 완료!")
```

```python
# ── 표준 템플릿: Cell 2 - FakeLLM 설정 ──

class FakeLLM(BaseChatModel):
    """표준 FakeLLM - 모든 노트북에서 재사용"""
    responses: dict[str, str] = {}
    default_response: str = "처리 완료."

    @property
    def _llm_type(self) -> str:
        return "fake-llm"

    def _generate(self, messages: list[BaseMessage], **kwargs):
        last_msg = messages[-1].content if messages else ""
        matched = self.default_response
        for pattern, response in self.responses.items():
            if pattern.lower() in last_msg.lower():
                matched = response
                break
        return ChatResult(
            generations=[ChatGeneration(message=AIMessage(content=matched))]
        )

# FakeLLM 인스턴스 (프로젝트에 맞게 responses를 수정하세요)
fake_llm = FakeLLM(responses={})
print("FakeLLM 준비 완료!")
```


77

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# ── 표준 템플릿: Cell 3 - 유틸리티 함수 ──

def show_graph(graph):
    """그래프를 시각화합니다."""
    try:
        png = graph.get_graph().draw_mermaid_png()
        display(Image(png))
    except Exception:
        mermaid = graph.get_graph().draw_mermaid()
        display(Markdown(f"```mermaid\n{mermaid}\n```"))

def run_and_trace(graph, initial_state: dict):
    """그래프를 실행하고 각 노드의 출력을 추적합니다."""
    print("=" * 60)
    events = []
    for event in graph.stream(initial_state):
        events.append(event)
        node_name = list(event.keys())[0]
        print(f"\n>> 노드 실행: {node_name}")
        for key, val in event[node_name].items():
            print(f"   {key}: {_truncate(str(val))}")
        print("-" * 40)
    print("=" * 60)
    print("실행 완료!")
    return events

def _truncate(s: str, n: int = 80) -> str:
    return s[:n] + "..." if len(s) > n else s

print("유틸리티 함수 준비 완료!")
```

---

4. 연습 문제

연습 4-1: FakeLLM 확장하기

아래 요구사항에 맞게 FakeLLM을 만들어 보세요:

1. "감정" 키워드가 포함된 질문에는 `"감정 분석: 긍정적(75%), 부정적(15%), 중립(10%)"` 응답
2. "번역" 키워드가 포함된 질문에는 `"Translation: Hello, this is a test."` 응답
3. "코드" 키워드가 포함된 질문에는 `"코드 리뷰: 변수명이 명확하고, 함수 구조가 좋습니다."` 응답
4. 위 어느 것에도 해당하지 않으면 `"잘 모르겠습니다. 다시 질문해 주세요."` 응답

78

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# 여기에 코드를 작성하세요
fake_llm = FakeLLM(
    responses={
        # TODO: 패턴-응답 매핑을 채우세요
    },
    default_response="TODO: 기본 응답을 설정하세요",
)

# 테스트
assert "긍정" in fake_llm.invoke("이 리뷰의 감정을 분석해줘").content
assert "Translation" in fake_llm.invoke("이 문장을 번역해줘").content
assert "코드 리뷰" in fake_llm.invoke("이 코드를 검토해줘").content
assert "잘 모르겠습니다" in fake_llm.invoke("날씨가 좋다").content
print("모든 테스트 통과!")
```

연습 4-2: 3-노드 그래프 만들고 시각화하기

다음 3개 노드를 가진 "텍스트 처리 에이전트"를 만들어 보세요:

1. **clean_text**: 입력 텍스트에서 공백 정리 (FakeLLM 불필요)
2. **classify_text**: FakeLLM으로 텍스트 분류
3. **generate_report**: FakeLLM으로 분류 결과 기반 보고서 생성

79

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# 여기에 코드를 작성하세요

# 1. State 정의
class TextProcessState(TypedDict):
    raw_text: str
    cleaned_text: str | None
    classification: str | None
    report: str | None
    current_step: str

# 2. 노드 함수 정의
def clean_text(state: TextProcessState) -> dict:
    # TODO: 구현하세요
    pass

def classify_text(state: TextProcessState) -> dict:
    # TODO: FakeLLM을 사용하여 구현하세요
    pass

def generate_report(state: TextProcessState) -> dict:
    # TODO: FakeLLM을 사용하여 구현하세요
    pass

# 3. 그래프 조립
# TODO: StateGraph를 만들고 노드와 엣지를 연결하세요

# 4. 실행 및 시각화
# TODO: 그래프를 시각화하고, 샘플 입력으로 실행하세요
```

연습 4-3: Mock으로 외부 API 대체하기

다음 시나리오를 구현하세요:

- `search_web` 노드: 웹 검색 API를 호출하여 결과를 가져옴 (Mock으로 대체)
- `extract_info` 노드: 검색 결과에서 핵심 정보 추출 (FakeLLM 사용)
- Mock을 사용하여 `search_web` 이 3개의 검색 결과를 반환하도록 설정

```python
# 여기에 코드를 작성하세요

# 힌트: functools.partial로 의존성을 주입하세요
# search_fn = partial(search_web, search_client=mock_search)
```

---

5. 핵심 정리

80

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

개념	설명
Jupyter 노트북	코드를 셀 단위로 실행하고 결과를 즉시 확인하는 대화형 개발 환경
ipykernel	Python 가상환경을 Jupyter 커널로 등록하여, 노트북에서 특정 환경의 패키지를 사용
FakeLLM	실제 LLM API 없이 패턴 매칭으로 응답을 반환하는 가짜 LLM 객체
Fixture	테스트용 데이터를 JSON 파일로 외부 관리. FakeLLM의 응답 데이터 소스
Mermaid 다이어그램	LangGraph 그래프 구조를 시각적으로 표현하는 다이어그램 포맷
노드 독립 호출	전체 그래프를 실행하지 않고 개별 노드 함수를 직접 호출하여 테스트
State Diff	노드 실행 전후의 상태 변화를 비교하여 보여주는 디버깅 기법
Mock 객체	외부 시스템(API, DB)을 가짜 객체로 대체. `unittest.mock` 표준 라이브러리 사용
의존성 주입	`functools.partial` 로 노드 함수에 외부 도구(LLM, API 클라이언트)를 주입. 테스트 시 교체 용이
표준 템플릿	모든 노트북에서 공통으로 사용하는 초기화 셀 패턴. 환경 설정의 일관성 보장

---

6. 참고 자료

공식 문서

- **Jupyter 공식 문서**: https://jupyter.org/documentation
  - Jupyter 노트북의 설치, 사용법, 설정에 대한 종합 가이드
- **IPython 커널 설치 가이드**: https://ipython.readthedocs.io/en/stable/install/kernel_install.html
  - 가상환경을 Jupyter 커널로 등록하는 방법
- **LangGraph 시각화**: https://langchain-ai.github.io/langgraph/how-tos/visualization/
  - Mermaid 다이어그램과 PNG 렌더링 방법
- **Python unittest.mock**: https://docs.python.org/3/library/unittest.mock.html
  - Mock, MagicMock, patch 등 테스트 도구 공식 레퍼런스

81

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

추가 학습 자료

- **LangChain FakeListLLM**: https://python.langchain.com/docs/modules/model_io/llms/fake_llm
  - LangChain에서 제공하는 기본 FakeLLM 구현
- **functools.partial 문서**: https://docs.python.org/3/library/functools.html#functools.partial
  - 의존성 주입에 사용하는 partial 함수 상세 설명
- **Mermaid 공식 사이트**: https://mermaid.js.org/
  - Mermaid 다이어그램 문법과 라이브 에디터
- **JupyterLab**: https://jupyterlab.readthedocs.io/
  - Jupyter 노트북의 차세대 인터페이스

다음 단계

이 모듈에서 Jupyter 기반 개발 환경을 구축했습니다. 다음 **Module 04: 나의 첫 LangGraph 에이전트 만들기**에서는 이 환경을 활용하여 완전한 에이전트를 처음부터 끝까지 만들어 봅니다.

다음 모듈에서 다루는 내용: - 실전 에이전트 프로젝트 설계 (문서 요약 에이전트) - 4개 노드로 구성된 완전한 그래프 구현 - 조건부 분기와 에러 처리 - 의존성 주입을 활용한 테스트 가능한 에이전트 설계 - FakeLLM 기반 E2E 테스트

82

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

⬆ 목차로 돌아가기

Module 04: 나의 첫 LangGraph 에이전트 만들기

학습 목표

이 모듈을 완료하면 다음을 할 수 있습니다:

- LangGraph로 완전한 에이전트를 처음부터 설계하고 구현할 수 있다
- TypedDict로 에이전트 상태(State)를 정의하고 각 필드의 역할을 설명할 수 있다
- 4개 노드를 가진 그래프를 조립하고, 일반 엣지와 조건부 엣지를 사용할 수 있다
- functools.partial로 의존성을 주입하여 테스트 가능한 에이전트를 만들 수 있다
- FakeLLM으로 에이전트를 E2E 테스트할 수 있다

사전 지식

항목	수준	비고
Module 01-02	필수	LangGraph 기본 개념 (노드, 엣지, State)
Module 03	필수	Jupyter 환경, FakeLLM, 그래프 시각화
Python 함수/클래스	필수	def, class, TypedDict, dict 다루기
Python 타입 힌트	권장	str, list, dict, Optional 등

---

1. 개념 설명

1.1 프로젝트 설계: "문서 요약 에이전트"

이번 모듈에서 만들 에이전트는 **Document Summarizer Agent(문서 요약 에이전트)**입니다. 문서를 입력받아 내용을 분석하고 요약하여 깔끔한 결과물을 출력합니다.

요구사항 정의

83

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

항목	내용
에이전트 이름	문서 요약 에이전트 (Document Summarizer)
입력	문서 텍스트 (문자열)
출력	구조화된 요약 결과 (제목, 핵심 내용, 키워드, 난이도)
노드 수	4개
LLM 호출	2회 (분석 1회 + 요약 1회)
에러 처리	빈 문서 또는 분석 실패 시 에러 경로로 분기

워크플로우 다이어그램

```
[시작]
  |
  v
[fetch_document] ── 문서 로드 및 유효성 검사
  |
  ├─ (문서가 비어있음) ──> [END] (에러 경로)
  |
  v
[analyze_content] ── LLM으로 핵심 내용 분석
  |
  ├─ (분석 실패) ──> [END] (에러 경로)
  |
  v
[generate_summary] ── LLM으로 요약 생성
  |
  v
[format_output] ── 결과 포맷팅 후 완료
  |
  v
[END]
```

1.2 왜 이렇게 설계하나?

에이전트 설계에는 몇 가지 원칙이 있습니다:

1. **단일 책임 원칙**: 각 노드는 하나의 작업만 수행
   - `fetch_document`: 문서를 가져오는 것만 담당
   - `analyze_content`: 분석만 담당
   - 하나의 노드가 "가져오기 + 분석 + 요약"을 모두 하지 않음
2. **의존성 주입**: LLM, 외부 API 등을 노드 함수 내부에서 직접 생성하지 않고, 외부에서 주입
   - 테스트 시 FakeLLM으로 쉽게 교체 가능
   - 실제 운영 시 진짜 LLM으로 교체 가능

84

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

3. **에러 경로 분리**: 정상 경로와 에러 경로를 명확히 분리
   - 어디서 실패했는지 즉시 파악 가능
   - 에러 시 불필요한 노드 실행을 건너뜀

---

2. 단계별 실습

2.1 State 설계

State는 에이전트의 "기억"입니다. 모든 노드가 이 State를 통해 데이터를 주고받습니다.

85

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# ── Cell 1: State 정의 ──

from typing import TypedDict


class DocumentSummaryState(TypedDict):
    """문서 요약 에이전트의 상태 정의.

    각 노드는 이 State에서 필요한 데이터를 읽고, 결과를 State에 기록합니다.
    """

    # ─── 입력 필드 ───
    source_text: str
    """원본 문서 텍스트. 사용자가 처음에 제공하는 입력값."""

    # ─── 중간 처리 필드 ───
    document: dict | None
    """로드된 문서 정보. fetch_document 노드가 채움.
    예: {"title": "...", "content": "...", "word_count": 150}
    """

    analysis: dict | None
    """분석 결과. analyze_content 노드가 채움.
    예: {"keywords": [...], "topic": "...", "difficulty": "..."}
    """

    # ─── 출력 필드 ───
    summary: str | None
    """생성된 요약문. generate_summary 노드가 채움."""

    final_output: str | None
    """최종 포맷팅된 결과. format_output 노드가 채움."""

    # ─── 제어 필드 ───
    current_step: str
    """현재 진행 단계. 디버깅 및 추적 용도.
    값: "start" | "fetched" | "analyzed" | "summarized" | "completed" | "error"
    """

    error: str | None
    """에러 메시지. 에러 발생 시 채워지며, 조건 분기에서 에러 경로로 라우팅."""


print("State 정의 완료!")
print("필드 목록:", list(DocumentSummaryState.__annotations__.keys()))
```

State 필드 역할 요약

86

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
[source_text] ──fetch──> [document] ──analyze──> [analysis] ──summarize──> [summary]
──format──> [final_output]

|
[current_step]: "start" → "fetched" → "analyzed" → "summarized" → "completed"
[error]: None (정상) 또는 "에러 메시지" (실패 시)
```

필드	타입	채우는 노드	읽는 노드	설명
`source_text`	`str`	(사용자 입력)	fetch_document	원본 텍스트
`document`	`dict | None`	fetch_document	analyze_content	파싱된 문서 정보
`analysis`	`dict | None`	analyze_content	generate_summary	분석 결과
`summary`	`str | None`	generate_summary	format_output	요약문
`final_output`	`str | None`	format_output	(최종 출력)	포맷된 결과
`current_step`	`str`	모든 노드	조건 분기	진행 상태
`error`	`str | None`	에러 발생 노드	조건 분기	에러 메시지

2.2 노드 함수 구현

노드 1: fetch_document (문서 로드)

87

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# ── Cell 2: fetch_document 노드 ──

def fetch_document(state: DocumentSummaryState) -> dict:
    """문서를 로드하고 기본 정보를 추출하는 노드.

    Args:
        state: 현재 에이전트 상태. source_text 필드를 읽음.

    Returns:
        업데이트할 필드들. document, current_step, (에러 시) error.
    """
    source_text = state["source_text"]

    # 유효성 검사: 빈 문서 체크
    if not source_text or not source_text.strip():
        return {
            "error": "문서가 비어있습니다. 텍스트를 입력해주세요.",
            "current_step": "error",
        }

    # 문서 정보 추출
    lines = source_text.strip().split("\n")
    title = lines[0] if lines else "제목 없음"
    content = "\n".join(lines[1:]).strip() if len(lines) > 1 else source_text
    word_count = len(source_text.split())

    document = {
        "title": title,
        "content": content,
        "word_count": word_count,
        "line_count": len(lines),
    }

    print(f"  [fetch_document] 문서 로드 완료: '{title}' ({word_count}단어)")

    return {
        "document": document,
        "current_step": "fetched",
    }


# 단독 테스트
test_result = fetch_document({
    "source_text": "Python 가이드\n파이썬은 간결하고 읽기 쉬운 언어입니다.",
    "document": None, "analysis": None, "summary": None,
    "final_output": None, "current_step": "start", "error": None,
})
print(f"결과: {test_result}")
```

노드 2: analyze_content (핵심 내용 분석)

88

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# ── Cell 3: analyze_content 노드 ──

import json


def analyze_content(state: DocumentSummaryState, llm=None) -> dict:
    """LLM을 사용하여 문서의 핵심 내용을 분석하는 노드.

    Args:
        state: 현재 에이전트 상태. document 필드를 읽음.
        llm: LLM 인스턴스 (의존성 주입). None이면 에러.

    Returns:
        업데이트할 필드들. analysis, current_step, (에러 시) error.
    """
    document = state.get("document")

    if not document:
        return {
            "error": "문서가 로드되지 않았습니다.",
            "current_step": "error",
        }

    if llm is None:
        return {
            "error": "LLM이 설정되지 않았습니다.",
            "current_step": "error",
        }

    # LLM에 분석 요청
    prompt = (
        f"다음 문서를 분석해주세요.\n\n"
        f"제목: {document['title']}\n"
        f"내용: {document['content']}\n\n"
        f"JSON 형식으로 응답해주세요: "
        f'{{"keywords": [...], "topic": "...", "difficulty": "초급|중급|고급"}}'
    )

    try:
        response = llm.invoke(prompt)
        content = response.content

        # JSON 파싱 시도
        try:
            analysis = json.loads(content)
        except json.JSONDecodeError:
            # JSON이 아닌 경우 텍스트 기반 분석 결과 구성
            analysis = {
                "keywords": [],
                "topic": content[:100],
                "difficulty": "중급",
                "raw_response": content,
            }
```


89

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
        print(f"  [analyze_content] 분석 완료: 주제='{analysis.get('topic', 'N/A')}'")

        return {
            "analysis": analysis,
            "current_step": "analyzed",
        }

    except Exception as e:
        return {
            "error": f"분석 중 오류 발생: {str(e)}",
            "current_step": "error",
        }
```

노드 3: generate_summary (요약 생성)

90

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# ── Cell 4: generate_summary 노드 ──

def generate_summary(state: DocumentSummaryState, llm=None) -> dict:
    """LLM을 사용하여 문서 요약을 생성하는 노드.

    Args:
        state: 현재 에이전트 상태. document와 analysis 필드를 읽음.
        llm: LLM 인스턴스 (의존성 주입).

    Returns:
        업데이트할 필드들. summary, current_step.
    """
    document = state.get("document", {})
    analysis = state.get("analysis", {})

    if llm is None:
        return {
            "error": "LLM이 설정되지 않았습니다.",
            "current_step": "error",
        }

    prompt = (
        f"다음 문서를 요약해주세요.\n\n"
        f"제목: {document.get('title', 'N/A')}\n"
        f"내용: {document.get('content', 'N/A')}\n"
        f"분석 결과: {json.dumps(analysis, ensure_ascii=False)}\n\n"
        f"3줄 이내로 요약해주세요."
    )

    try:
        response = llm.invoke(prompt)
        summary = response.content

        print(f"  [generate_summary] 요약 생성 완료 ({len(summary)}자)")

        return {
            "summary": summary,
            "current_step": "summarized",
        }

    except Exception as e:
        return {
            "error": f"요약 생성 중 오류 발생: {str(e)}",
            "current_step": "error",
        }
```

노드 4: format_output (결과 포맷팅)

91

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# ── Cell 5: format_output 노드 ──

def format_output(state: DocumentSummaryState) -> dict:
    """최종 결과를 보기 좋게 포맷하는 노드.

    Args:
        state: 현재 에이전트 상태. document, analysis, summary 필드를 읽음.

    Returns:
        업데이트할 필드들. final_output, current_step.
    """
    document = state.get("document", {})
    analysis = state.get("analysis", {})
    summary = state.get("summary", "요약 없음")

    # 키워드 목록 포맷
    keywords = analysis.get("keywords", [])
    keywords_str = ", ".join(keywords) if keywords else "추출된 키워드 없음"

    # 최종 출력 구성
    output_parts = [
        "=" * 50,
        "  문서 요약 결과",
        "=" * 50,
        "",
        f"  제목: {document.get('title', 'N/A')}",
        f"  단어 수: {document.get('word_count', 'N/A')}",
        f"  주제: {analysis.get('topic', 'N/A')}",
        f"  난이도: {analysis.get('difficulty', 'N/A')}",
        f"  키워드: {keywords_str}",
        "",
        "-" * 50,
        "  요약:",
        f"  {summary}",
        "-" * 50,
    ]

    final_output = "\n".join(output_parts)

    print(f"  [format_output] 포맷팅 완료")

    return {
        "final_output": final_output,
        "current_step": "completed",
    }
```

2.3 그래프 조립

이제 4개의 노드를 연결하여 완전한 그래프를 만듭니다.

92

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# ── Cell 6: 그래프 조립 ──

from langgraph.graph import StateGraph, END
from functools import partial


def build_summarizer_graph(llm=None):
    """문서 요약 에이전트 그래프를 빌드합니다.

    Args:
        llm: LLM 인스턴스. 테스트 시 FakeLLM, 운영 시 실제 LLM.

    Returns:
        컴파일된 LangGraph 그래프.
    """
    graph = StateGraph(DocumentSummaryState)

    # ─── 노드 등록 ───
    # fetch_document는 LLM이 필요 없으므로 그대로 등록
    graph.add_node("fetch_document", fetch_document)

    # analyze_content, generate_summary는 LLM이 필요하므로
    # functools.partial로 llm을 주입
    graph.add_node(
        "analyze_content",
        partial(analyze_content, llm=llm),
    )
    graph.add_node(
        "generate_summary",
        partial(generate_summary, llm=llm),
    )

    # format_output도 LLM 불필요
    graph.add_node("format_output", format_output)

    # ─── 진입점 설정 ───
    graph.set_entry_point("fetch_document")

    # ─── 엣지 연결 ───

    # 조건부 엣지 1: fetch_document 이후
    # - 에러가 있으면 END로 (에러 경로)
    # - 정상이면 analyze_content로
    graph.add_conditional_edges(
        "fetch_document",
        _route_after_fetch,
        {
            "analyze": "analyze_content",
            "error": END,
        },
    )

    # 조건부 엣지 2: analyze_content 이후
    # - 에러가 있으면 END로 (에러 경로)
```


93

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
    # - 정상이면 generate_summary로
    graph.add_conditional_edges(
        "analyze_content",
        _route_after_analyze,
        {
            "summarize": "generate_summary",
            "error": END,
        },
    )

    # 일반 엣지: generate_summary → format_output → END
    graph.add_edge("generate_summary", "format_output")
    graph.add_edge("format_output", END)

    return graph.compile()


# ── 조건 분기 함수 ──

def _route_after_fetch(state: DocumentSummaryState) -> str:
    """fetch_document 이후 분기 결정.

    Returns:
        "analyze": 정상 → 분석 진행
        "error": 에러 → 종료
    """
    if state.get("error"):
        return "error"
    return "analyze"


def _route_after_analyze(state: DocumentSummaryState) -> str:
    """analyze_content 이후 분기 결정.

    Returns:
        "summarize": 정상 → 요약 진행
        "error": 에러 → 종료
    """
    if state.get("error"):
        return "error"
    return "summarize"


print("그래프 빌드 함수 정의 완료!")
```

핵심 포인트: functools.partial

`partial(analyze_content, llm=fake_llm)` 은 `analyze_content` 함수에 `llm=fake_llm` 을 미리 채운 새 함수를 만듭니다. 이렇게 하면: - **테스트 시:** `partial(analyze_content, llm=fake_llm)` - FakeLLM 주입 - **운영 시:** `partial(analyze_content, llm=real_llm)` - 실제 LLM 주입

94

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

> 노드 함수 코드는 전혀 변경하지 않아도 됩니다.

2.4 FakeLLM 준비 및 그래프 빌드

95

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# ── Cell 7: FakeLLM 생성 및 그래프 빌드 ──

from langchain_core.language_models import BaseChatModel
from langchain_core.messages import AIMessage, BaseMessage
from langchain_core.outputs import ChatGeneration, ChatResult


class FakeLLM(BaseChatModel):
    """패턴 매칭 기반 가짜 LLM."""
    responses: dict[str, str] = {}
    default_response: str = "처리 완료."

    @property
    def _llm_type(self) -> str:
        return "fake-llm"

    def _generate(self, messages: list[BaseMessage], **kwargs):
        last_msg = messages[-1].content if messages else ""
        matched = self.default_response
        for pattern, response in self.responses.items():
            if pattern.lower() in last_msg.lower():
                matched = response
                break
        return ChatResult(
            generations=[ChatGeneration(message=AIMessage(content=matched))]
        )


# 문서 요약 에이전트용 FakeLLM 설정
fake_llm = FakeLLM(
    responses={
        "분석": json.dumps({
            "keywords": ["Python", "FastAPI", "웹 프레임워크"],
            "topic": "Python 웹 개발",
            "difficulty": "중급",
        }, ensure_ascii=False),
        "요약": (
            "본 문서는 FastAPI를 활용한 Python 웹 개발 방법을 설명합니다. "
            "REST API 설계 원칙과 비동기 처리 패턴을 중심으로 다루며, "
            "초보 개발자도 쉽게 따라할 수 있는 단계별 가이드를 제공합니다."
        ),
    },
    default_response="요청을 처리했습니다.",
)

# 그래프 빌드
graph = build_summarizer_graph(llm=fake_llm)
print("그래프 빌드 완료!")
```

2.5 그래프 시각화

96

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# ── Cell 8: 그래프 시각화 ──

from IPython.display import display, Image, Markdown

try:
    png = graph.get_graph().draw_mermaid_png()
    display(Image(png))
except Exception:
    mermaid = graph.get_graph().draw_mermaid()
    display(Markdown(f"```mermaid\n{mermaid}\n```"))

print("위 다이어그램에서 에이전트의 흐름을 확인하세요.")
print("- fetch_document에서 analyze_content 또는 END로 분기")
print("- analyze_content에서 generate_summary 또는 END로 분기")
```

---

3. 실전 예제

3.1 정상 경로 실행

97

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# ── Cell 9: 정상 케이스 실행 ──

# 초기 상태
initial_state: DocumentSummaryState = {
    "source_text": (
        "FastAPI 시작하기\n"
        "FastAPI는 Python 3.7+로 API를 구축하기 위한 고성능 웹 프레임워크입니다. "
        "Starlette과 Pydantic을 기반으로 하며, 자동 문서 생성과 타입 검사를 제공합니다. "
        "비동기 처리를 기본 지원하여 높은 처리량을 달성할 수 있습니다."
    ),
    "document": None,
    "analysis": None,
    "summary": None,
    "final_output": None,
    "current_step": "start",
    "error": None,
}

# stream 모드로 실행하여 각 노드의 실행을 추적
print("=" * 60)
print("  문서 요약 에이전트 실행")
print("=" * 60)

events = []
for event in graph.stream(initial_state):
    events.append(event)
    node_name = list(event.keys())[0]
    print(f"\n>> 노드 실행 완료: {node_name}")

# 최종 결과 확인
final_state = {}
for event in events:
    for node_output in event.values():
        final_state.update(node_output)

if final_state.get("final_output"):
    print("\n" + final_state["final_output"])
elif final_state.get("error"):
    print(f"\n[에러] {final_state['error']}")
```

3.2 에러 경로 실행

98

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# ── Cell 10: 에러 케이스 - 빈 문서 ──

error_state: DocumentSummaryState = {
    "source_text": "",  # 빈 문서!
    "document": None,
    "analysis": None,
    "summary": None,
    "final_output": None,
    "current_step": "start",
    "error": None,
}

print("=" * 60)
print("  에러 케이스: 빈 문서 테스트")
print("=" * 60)

for event in graph.stream(error_state):
    node_name = list(event.keys())[0]
    node_output = event[node_name]
    print(f"\n>> 노드: {node_name}")
    if node_output.get("error"):
        print(f"   에러 발생: {node_output['error']}")
    print(f"   현재 단계: {node_output.get('current_step', 'N/A')}")

# 출력:
# >> 노드: fetch_document
#    에러 발생: 문서가 비어있습니다. 텍스트를 입력해주세요.
#    현재 단계: error
# (analyze_content, generate_summary, format_output은 실행되지 않음)
```

3.3 각 노드 독립 테스트

전체 그래프를 실행하지 않고 개별 노드를 직접 테스트합니다.

99

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# ── Cell 11: 개별 노드 테스트 ──

print("=" * 60)
print("  개별 노드 테스트")
print("=" * 60)

# --- 테스트 1: fetch_document ---
print("\n[테스트 1] fetch_document - 정상 입력")
result_1 = fetch_document({
    "source_text": "테스트 제목\n테스트 본문 내용입니다.",
    "document": None, "analysis": None, "summary": None,
    "final_output": None, "current_step": "start", "error": None,
})
assert result_1["current_step"] == "fetched"
assert result_1["document"]["title"] == "테스트 제목"
print(f"  통과! document.title = '{result_1['document']['title']}'")

print("\n[테스트 1-b] fetch_document - 빈 입력")
result_1b = fetch_document({
    "source_text": "   ",
    "document": None, "analysis": None, "summary": None,
    "final_output": None, "current_step": "start", "error": None,
})
assert result_1b.get("error") is not None
print(f"  통과! error = '{result_1b['error']}'")

# --- 테스트 2: analyze_content ---
print("\n[테스트 2] analyze_content - FakeLLM 사용")
analyze_fn = partial(analyze_content, llm=fake_llm)
result_2 = analyze_fn({
    "source_text": "", "document": {"title": "테스트", "content": "분석 대상 텍스트"},
    "analysis": None, "summary": None, "final_output": None,
    "current_step": "fetched", "error": None,
})
assert result_2["current_step"] == "analyzed"
assert result_2["analysis"] is not None
print(f"  통과! analysis.topic = '{result_2['analysis'].get('topic', 'N/A')}'")

# --- 테스트 3: generate_summary ---
print("\n[테스트 3] generate_summary - FakeLLM 사용")
summary_fn = partial(generate_summary, llm=fake_llm)
result_3 = summary_fn({
    "source_text": "", "document": {"title": "테스트", "content": "요약 대상"},
    "analysis": {"keywords": ["테스트"], "topic": "테스트 주제", "difficulty": "초급"},
    "summary": None, "final_output": None,
    "current_step": "analyzed", "error": None,
})
assert result_3["current_step"] == "summarized"
assert result_3["summary"] is not None
print(f"  통과! summary 길이 = {len(result_3['summary'])}자")

# --- 테스트 4: format_output ---
print("\n[테스트 4] format_output")
result_4 = format_output({
```


100

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
    "source_text": "", "document": {"title": "테스트", "word_count": 50},
    "analysis": {"keywords": ["A", "B"], "topic": "테스트", "difficulty": "초급"},
    "summary": "테스트 요약입니다.",
    "final_output": None, "current_step": "summarized", "error": None,
})
assert result_4["current_step"] == "completed"
assert "테스트 요약입니다." in result_4["final_output"]
print(f"  통과! final_output 길이 = {len(result_4['final_output'])}자")

print("\n" + "=" * 60)
print("  모든 노드 테스트 통과!")
print("=" * 60)
```

---

4. 연습 문제

연습 4-1: 전체 코드를 한 파일로 정리

지금까지 작성한 코드를 하나의 Python 파일(`summarizer_agent.py`)로 정리해 보세요. 아래 뼈대를 완성하세요.

101

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
"""문서 요약 에이전트 - 완전한 구현.

사용법:
    from summarizer_agent import build_summarizer_graph, FakeLLM

    fake_llm = FakeLLM(responses={...})
    graph = build_summarizer_graph(llm=fake_llm)
    result = graph.invoke(initial_state)
"""

from typing import TypedDict
from langgraph.graph import StateGraph, END
from functools import partial
import json

# ── FakeLLM ──
# TODO: Module 03에서 만든 FakeLLM 클래스를 여기에 넣으세요

# ── State ──
# TODO: DocumentSummaryState를 정의하세요

# ── 노드 함수 ──
# TODO: 4개 노드 함수를 넣으세요

# ── 조건 분기 함수 ──
# TODO: _route_after_fetch, _route_after_analyze를 넣으세요

# ── 그래프 빌더 ──
# TODO: build_summarizer_graph 함수를 넣으세요

# ── 메인 실행 ──
if __name__ == "__main__":
    fake_llm = FakeLLM(responses={
        "분석": '{"keywords": ["Python"], "topic": "프로그래밍", "difficulty": "초급"}',
        "요약": "이 문서는 Python 프로그래밍에 대한 입문 가이드입니다.",
    })

    graph = build_summarizer_graph(llm=fake_llm)

    result = graph.invoke({
        "source_text": "Python 입문\nPython은 배우기 쉬운 언어입니다.",
        "document": None, "analysis": None, "summary": None,
        "final_output": None, "current_step": "start", "error": None,
    })

    if result.get("final_output"):
        print(result["final_output"])
    elif result.get("error"):
        print(f"에러: {result['error']}")
```

연습 4-2: "이메일 분류 에이전트" 직접 만들기

102

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

아래 요구사항에 맞는 에이전트를 직접 설계하고 구현하세요.

요구사항

항목	내용
에이전트 이름	이메일 분류 에이전트 (Email Classifier)
입력	이메일 텍스트 (제목 + 본문)
출력	분류 결과 (카테고리, 긴급도, 추천 조치)
노드 수	4개 이상

필요한 노드

1. **parse_email**: 이메일 텍스트에서 제목, 본문, 발신자를 추출
2. **classify_email**: LLM으로 카테고리 분류 (업무, 스팸, 마케팅, 개인)
3. **assess_urgency**: LLM으로 긴급도 판단 (높음, 중간, 낮음)
4. **generate_action**: 분류 결과를 바탕으로 추천 조치 생성

State 설계 힌트

```
class EmailClassifierState(TypedDict):
    raw_email: str              # 원본 이메일 텍스트
    parsed_email: dict | None   # 파싱된 이메일 정보
    category: str | None        # 분류 카테고리
    urgency: str | None         # 긴급도
    action: str | None          # 추천 조치
    final_report: str | None    # 최종 보고서
    current_step: str           # 현재 단계
    error: str | None           # 에러 메시지
```

FakeLLM 설정 힌트

```
fake_llm = FakeLLM(responses={
    "분류": '{"category": "업무", "confidence": 0.92}',
    "긴급": '{"urgency": "높음", "reason": "마감일이 내일입니다"}',
    "조치": "1. 즉시 회신 필요\n2. 관련 팀에 공유\n3. 일정에 등록",
})
```

도전 과제

- 조건부 분기 추가: "스팸" 카테고리인 경우 assess_urgency를 건너뛰고 바로 종료
- 모든 노드를 개별적으로 테스트하는 코드 작성
- 그래프를 시각화하고, 정상/에러/스팸 세 가지 경로를 각각 실행해 보기

103

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
# 여기에 코드를 작성하세요

# 1. State 정의

# 2. 노드 함수 구현 (4개)

# 3. 조건 분기 함수

# 4. 그래프 빌더

# 5. FakeLLM 설정 및 빌드

# 6. 테스트 케이스 3가지 실행:
#    - 정상 업무 이메일
#    - 스팸 이메일 (assess_urgency 건너뛰기)
#    - 빈 이메일 (에러 경로)
```

---

5. 핵심 정리

에이전트 개발 체크리스트

단계	핵심 질문	이 모듈에서 배운 답
State 설계	어떤 데이터가 노드 사이를 오가는가?	TypedDict로 명시적 필드 정의. 입력/중간/출력/제어 필드 분류
노드 구현	각 노드는 무엇을 하는가?	`(state) -> dict` 함수. 한 노드 = 한 책임
의존성 주입	LLM, API를 어떻게 교체하는가?	`functools.partial` 로 외부 주입. 테스트 시 FakeLLM/Mock 교체
엣지 연결	노드 실행 순서는?	일반 엣지(무조건 이동) + 조건부 엣지(분기 결정 함수)
에러 처리	실패하면 어떻게 되는가?	`state["error"]` 설정 + 조건부 엣지로 에러 경로 분기
테스트	어떻게 검증하는가?	개별 노드 독립 호출 + FakeLLM 기반 E2E 실행

핵심 패턴 요약

104

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# 패턴 1: 노드 함수 시그니처
def my_node(state: MyState, llm=None, tool=None) -> dict:
    """state를 읽고, 처리하고, 변경된 필드만 반환"""
    # state에서 데이터 읽기
    data = state["some_field"]
    # 처리
    result = llm.invoke(data)
    # 변경된 필드만 반환
    return {"output_field": result, "current_step": "done"}

# 패턴 2: 의존성 주입
from functools import partial
node_fn = partial(my_node, llm=fake_llm, tool=mock_tool)

# 패턴 3: 조건부 분기
def route_fn(state: MyState) -> str:
    if state.get("error"):
        return "error_path"
    return "next_path"

graph.add_conditional_edges("my_node", route_fn, {
    "next_path": "next_node",
    "error_path": END,
})

# 패턴 4: 그래프 빌드
def build_graph(llm=None):
    graph = StateGraph(MyState)
    graph.add_node("node_a", node_a)
    graph.add_node("node_b", partial(node_b, llm=llm))
    graph.set_entry_point("node_a")
    graph.add_edge("node_a", "node_b")
    graph.add_edge("node_b", END)
    return graph.compile()
```

---

6. 참고 자료

공식 문서

- **LangGraph How-to Guides**: https://langchain-ai.github.io/langgraph/how-tos/
  - LangGraph의 모든 기능에 대한 실전 가이드. 그래프 정의, 조건부 분기, 스트리밍 등
- **functools.partial**: https://docs.python.org/3/library/functools.html#functools.partial
  - 의존성 주입에 사용하는 partial 함수의 공식 레퍼런스
- **Python TypedDict**: https://docs.python.org/3/library/typing.html#typing.TypedDict
  - State 정의에 사용하는 TypedDict의 공식 문서

105

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

추가 학습 자료

- **LangGraph 시각화**: https://langchain-ai.github.io/langgraph/how-tos/visualization/
  - 그래프 다이어그램 렌더링 방법
- **LangGraph Conditional Edges**: https://langchain-ai.github.io/langgraph/how-tos/branching/
  - 조건부 분기 패턴 상세 설명
- **Python unittest.mock**: https://docs.python.org/3/library/unittest.mock.html
  - Mock 객체 활용 레퍼런스
- **LangGraph State Management**: https://langchain-ai.github.io/langgraph/concepts/low_level/#state
  - State 설계 원칙과 고급 패턴

흔한 실수와 해결책 (FAQ)

문제	원인	해결
`TypeError: 'NoneType' is not subscriptable`	이전 노드가 채워야 할 필드가 None인 상태에서 접근	`state.get("field")` 사용 또는 None 체크 추가
그래프 실행 시 무한 루프	조건부 분기 함수가 잘못된 키를 반환	분기 함수의 반환값이 `add_conditional_edges` 의 매핑 딕셔너리 키와 일치하는지 확인
`KeyError: 'field_name'`	State에 정의하지 않은 필드를 사용	TypedDict에 모든 필드를 정의하고, 초기 상태에서 모든 필드에 값을 설정
FakeLLM이 기본 응답만 반환	패턴 매칭 키워드가 프롬프트에 포함되지 않음	FakeLLM의 `responses` 키가 프롬프트 텍스트에 실제로 포함되는지 확인
`partial()` 로 주입한 인자가 전달 안 됨	노드 함수의 매개변수 이름 불일치	`partial(fn, llm=llm)` 에서 `llm` 이 함수 시그니처의 매개변수명과 정확히 일치해야 함
에러 경로로 가지 않고 예외 발생	노드 내부에서 `raise` 를 사용	`raise` 대신 `return {"error": "메시지", "current_step": "error"}` 패턴 사용
그래프 시각화가 안 됨	`draw_mermaid_png()` 실패 (외부 도구 미설치)	`draw_mermaid()` 텍스트 출력 사용 또는 Mermaid Live Editor 활용

다음 단계

106

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

이 모듈에서 첫 번째 완전한 LangGraph 에이전트를 만들었습니다. 축하합니다!

지금까지 배운 것: - Module 01-02: LangGraph 기본 개념과 설치 - Module 03: Jupyter 개발 환경, FakeLLM, 시각화 - **Module 04: 완전한 에이전트 설계, 구현, 테스트** (이번 모듈)

다음 모듈에서 다루는 내용: - **Module 05**: 에이전트 고급 패턴 - Annotated State, Reducer, 서브그래프 - **Module 06**: 에이전트 프로덕션 배포 - 체크포인트, 재시도, 모니터링

107

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

⬆ 목차로 돌아가기

Module 05: 프롬프트 엔지니어링 - LLM의 성능을 결정하는 핵심 기술

학습 목표

이 모듈을 완료하면 다음을 할 수 있습니다:

1. 프롬프트의 구조(역할, 맥락, 지시, 출력 형식)를 이해하고 설계할 수 있다
2. LangChain `ChatPromptTemplate`을 사용하여 재사용 가능한 프롬프트를 만들 수 있다
3. Zero-shot, One-shot, Few-shot 기법의 차이를 이해하고 적용할 수 있다
4. 프롬프트를 외부 YAML 파일로 분리하여 버전 관리할 수 있다
5. 도메인/언어별로 특화된 프롬프트를 설계할 수 있다

사전 지식

이 모듈을 시작하기 전에 다음 내용을 알고 있어야 합니다:

- **Python 기초**: 함수, 클래스, 딕셔너리, f-string 사용법
- **LLM 기본 개념**: LLM이 무엇이고, API를 통해 호출하는 방법 (Module 03 참고)
- **LangChain 기초**: LangChain이 무엇인지 기본 개념 (Module 04 참고)

> **용어 정리** - **프롬프트(Prompt)**: LLM에게 보내는 입력 텍스트. "질문"이라고 생각하면 됩니다. - **토큰(Token)**: LLM이 텍스트를 처리하는 최소 단위. 대략 한국어 1글자 = 1~2 토큰. - **System 메시지**: LLM에게 "역할"을 부여하는 특별한 메시지. - **Human 메시지**: 사용자가 LLM에게 보내는 실제 요청 메시지.

---

1. 개념 설명

1.1 프롬프트란? 왜 중요한가?

108

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

프롬프트는 LLM에게 보내는 **지시문**입니다. 같은 모델이라도 프롬프트에 따라 결과의 품질이 극적으로 달라집니다.

동일한 모델, 다른 프롬프트 = 완전히 다른 결과

다음 두 프롬프트의 결과를 비교해 봅시다:

나쁜 프롬프트:

```
이 코드를 분석해줘.

def calculate_price(items):
    total = 0
    for item in items:
        total += item.price
    return total
```

결과: "이 코드는 가격을 계산하는 함수입니다." (너무 일반적이고 유용하지 않음)

좋은 프롬프트:

```
당신은 시니어 Python 개발자입니다.
아래 함수의 버그, 성능 이슈, 개선점을 분석하세요.

## 분석 기준
- 예외 처리 여부
- 타입 안전성
- 엣지 케이스 (빈 리스트, None 등)

## 응답 형식
JSON으로 출력하세요:
{"bugs": [...], "improvements": [...], "severity": "HIGH|MEDIUM|LOW"}

## 코드
def calculate_price(items):
    total = 0
    for item in items:
        total += item.price
    return total
```

결과: items가 None일 때 TypeError 발생, item.price가 음수인 경우 검증 누락 등 구체적인 분석 결과가 나옵니다.

> **핵심 포인트**: 프롬프트 엔지니어링은 "LLM에게 정확히 무엇을 원하는지 명확하게 전달하는 기술"입니다.

1.2 프롬프트의 4가지 구성 요소

좋은 프롬프트는 다음 4가지 요소를 포함합니다:

109

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

구성 요소	설명	예시
역할(Role)	LLM에게 전문가 역할 부여	"당신은 시니어 QA 엔지니어입니다"
맥락(Context)	배경 정보, 도메인 지식	"Spring Boot 기반 REST API 프로젝트입니다"
지시(Instruction)	구체적으로 무엇을 해야 하는지	"코드의 버그를 찾아 수정 방안을 제시하세요"
출력 형식(Format)	원하는 응답 형태	"JSON 형식으로, bugs와 fixes 키를 포함하세요"

```
[역할] 당신은 시니어 소프트웨어 엔지니어입니다.

[맥락] 아래는 사용자 인증 모듈의 코드 변경사항(diff)입니다.
       이 프로젝트는 Java/Spring Boot 기반입니다.

[지시] 변경된 기능을 분석하고, 테스트가 필요한 시나리오를 식별하세요.

[형식] 다음 JSON 형식으로 응답하세요:
       { "summary": "...", "test_scenarios": [...] }
```

1.3 System 메시지 vs Human 메시지

LLM API는 보통 두 종류의 메시지를 구분합니다:

구분	System 메시지	Human 메시지
역할	LLM의 행동 규칙 설정	실제 요청/데이터 전달
포함 내용	역할 정의, 규칙, 출력 형식	입력 데이터, 구체적 요청
변경 빈도	거의 변경되지 않음	매 요청마다 다름
비유	직원에게 "업무 매뉴얼"을 주는 것	직원에게 "오늘 할 일"을 주는 것

System에 넣어야 할 것: 역할, 규칙, 금지 사항, 출력 형식 **Human에 넣어야 할 것**: 입력 데이터, 변수, 구체적 요청문

110

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
# System: 역할 + 규칙 (불변)
system_message = """
당신은 소프트웨어 테스트 전문가입니다.

## 평가 기준
- PASS: 소스코드가 기대 결과를 충족
- FAIL: 결함이 발견되어 기대 결과 미충족
- BLOCKED: 판단을 위한 정보 부족

## 응답 형식
반드시 유효한 JSON만 출력하세요.
"""

# Human: 데이터 + 요청 (매번 다름)
human_message = """
## TC 정보
- 제목: 로그인 API 정상 동작 검증
- 기대 결과: 200 OK, JWT 토큰 반환

## 관련 소스코드
{source_code}

위 정보를 기반으로 TC를 평가하세요.
"""
```

---

2. 단계별 실습

2.1 LangChain ChatPromptTemplate 사용법

LangChain의 `ChatPromptTemplate`은 프롬프트를 구조화하고 재사용할 수 있게 해주는 핵심 도구입니다.

Step 1: 기본 ChatPromptTemplate 만들기

111

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# 필요한 패키지 설치
# pip install langchain-core langchain-anthropic

from langchain_core.prompts import ChatPromptTemplate

# from_messages()로 System/Human 메시지를 분리하여 생성
prompt = ChatPromptTemplate.from_messages([
    (
        "system",   # 역할: LLM의 행동 규칙
        "당신은 코드 리뷰 전문가입니다.\n"
        "주어진 코드의 버그와 개선점을 분석하세요.\n"
        "반드시 JSON 형식으로 응답하세요."
    ),
    (
        "human",    # 역할: 사용자 요청 + 데이터
        "## 분석할 코드\n"
        "언어: {language}\n"
        "```\n{code}\n```\n\n"
        "위 코드를 분석해주세요."
    ),
])
```

> **주목**: `{language}` 와 `{code}` 는 **변수 자리표시자(placeholder)** 입니다. 나중에 실제 값으로 치환됩니다.

Step 2: 변수 치환하기

```python
# 변수에 실제 값을 넣어서 프롬프트 완성
formatted = prompt.format_messages(
    language="Python",
    code="def divide(a, b):\n    return a / b"
)

# 결과 확인
for msg in formatted:
    print(f"[{msg.type}] {msg.content[:80]}...")
# [system] 당신은 코드 리뷰 전문가입니다...
# [human] ## 분석할 코드\n언어: Python...
```

Step 3: 체인(Chain) 파이프라인 만들기

LangChain의 핵심 개념인 **파이프라인**을 사용하면, 프롬프트 -> LLM 호출 -> 결과 파싱을 한 줄로 연결할 수 있습니다.

112

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
from langchain_anthropic import ChatAnthropic
from langchain_core.output_parsers import StrOutputParser

# LLM 인스턴스 생성
llm = ChatAnthropic(model="claude-sonnet-4-5-20250514", temperature=0.1)

# 파이프라인: 프롬프트 -> LLM -> 텍스트 파서
# "|" 연산자로 체인을 연결합니다 (UNIX 파이프와 유사)
chain = prompt | llm | StrOutputParser()

# 실행: 변수를 딕셔너리로 넣어서 호출
result = chain.invoke({
    "language": "Python",
    "code": "def divide(a, b):\n    return a / b"
})

print(result)
# {"bugs": [{"description": "ZeroDivisionError when b is 0", ...}], ...}
```

파이프라인 동작 흐름:

```
{language, code}                  <-- 입력 딕셔너리
      |
      v
[ChatPromptTemplate]  -- 변수 치환 --> System/Human 메시지 리스트
      |
      v
[ChatAnthropic]       -- API 호출 --> AIMessage(content="...")
      |
      v
[StrOutputParser]     -- 텍스트 추출 --> "결과 문자열"
```

2.2 Few-shot 학습

Few-shot 학습은 프롬프트에 **입출력 예시를 포함**시켜 LLM이 원하는 형식과 품질을 학습하게 하는 기법입니다.

Zero-shot vs One-shot vs Few-shot 비교

기법	설명	장점	단점
Zero-shot	예시 없이 지시만 제공	간결, 토큰 절약	출력 형식 불안정
One-shot	예시 1개 제공	형식 이해 도움	예시가 편향될 수 있음
Few-shot	예시 2~5개 제공	안정적 출력, 높은 품질	토큰 사용 증가

같은 작업에 다른 기법 적용 비교

작업: 이메일 내용을 분류하기 (카테고리: 문의, 불만, 칭찬, 기타)

113

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

Zero-shot (예시 없음):

```
zero_shot = ChatPromptTemplate.from_messages([
    ("system", "이메일을 분류하세요. 카테고리: 문의, 불만, 칭찬, 기타"),
    ("human", "이메일 내용: {email}\n\nJSON으로 답하세요."),
])
```

Few-shot (예시 3개):

```
few_shot = ChatPromptTemplate.from_messages([
    (
        "system",
        "이메일을 분류하세요. 카테고리: 문의, 불만, 칭찬, 기타\n"
        "반드시 JSON 형식으로만 응답하세요."
    ),
    # --- Few-shot 예시 시작 ---
    # 예시 1: 문의
    ("human", "이메일 내용: 환불 절차가 어떻게 되나요?"),
    ("assistant", '{{"category": "문의"}}'),
    # 예시 2: 불만
    ("human", "이메일 내용: 배송이 2주나 지연되어 정말 화가 납니다."),
    ("assistant", '{{"category": "불만"}}'),
    # 예시 3: 칭찬
    ("human", "이메일 내용: 친절한 상담 감사합니다. 문제 해결했어요!"),
    ("assistant", '{{"category": "칭찬"}}'),
    # --- Few-shot 예시 끝 ---
    # 실제 입력
    ("human", "이메일 내용: {email}"),
])
```

> **실험 결과**: Few-shot을 사용하면 JSON 형식 준수율이 약 70%에서 95% 이상으로 향상됩니다.

좋은 Few-shot 예시 vs 나쁜 Few-shot 예시

나쁜 예시 – 너무 짧고 모호함:

```
- input: "코드 변경"
  output: '{"summary": "변경됨"}'
```

좋은 예시 – 구체적이고 현실적:

114

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
- input: |
    ## 변경 정보
    - 소스 브랜치: feature/user-auth
    - 언어/프레임워크: java / spring-boot

    ## Diff
    + @PostMapping("/api/v1/auth/login")
    + public ResponseEntity<LoginResponse> login(@RequestBody LoginRequest req) {
    +     User user = userService.authenticate(req.getEmail(), req.getPassword());
    +     String token = jwtService.generateToken(user);
    +     return ResponseEntity.ok(new LoginResponse(token));
    + }
  output: |
    {
      "summary": "사용자 로그인 API 엔드포인트 추가",
      "changed_features": [
        {
          "feature": "사용자 인증 - 로그인 API",
          "change_type": "added",
          "description": "POST /api/v1/auth/login. 이메일/비밀번호 인증 후 JWT 토큰 발급"
        }
      ],
      "test_scenarios": [
        {
          "scenario": "올바른 이메일/비밀번호로 로그인 시 JWT 토큰 반환 확인",
          "test_type": "functional",
          "priority": "HIGH"
        },
        {
          "scenario": "잘못된 비밀번호로 로그인 시 401 에러 반환 확인",
          "test_type": "edge_case",
          "priority": "HIGH"
        }
      ]
    }
```

좋은 예시를 만드는 4가지 원칙:

1. **현실적 입력**: 실제 프로젝트에서 나올 법한 데이터 사용
2. **모든 필드 포함**: 출력 JSON의 모든 필드를 빠짐없이 채움
3. **다양한 케이스**: 정상 케이스와 예외 케이스를 모두 포함
4. **구체적 내용**: "변경됨" 대신 "POST /api/v1/auth/login 엔드포인트 추가"

ChatPromptTemplate에 Few-shot 추가하기

115

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
prompt_with_few_shots = ChatPromptTemplate.from_messages([
    # 1) System 메시지: 역할 + 규칙
    (
        "system",
        "당신은 시니어 소프트웨어 엔지니어입니다.\n"
        "코드 변경사항을 분석하고 테스트 시나리오를 식별하세요.\n"
        "반드시 JSON 형식으로 응답하세요."
    ),
    # 2) Few-shot 예시 쌍 (human -> assistant 반복)
    ("human", "## 변경 정보\n- 브랜치: feature/add-login\n- 언어: java\n\n## Diff\n+
        @PostMapping(\"/login\") ..."),
    ("assistant", '{{"summary": "로그인 API 추가", "test_scenarios": [{{"scenario": "정상
        로그인 검증", "priority": "HIGH"}}]}}'),
    ("human", "## 변경 정보\n- 브랜치: fix/null-check\n- 언어: python\n\n## Diff\n+ if user
        is None: return None"),
    ("assistant", '{{"summary": "null 체크 추가", "test_scenarios": [{{"scenario": "None
        입력 시 안전 반환 검증", "priority": "MEDIUM"}}]}}'),
    # 3) 실제 Human 메시지 (변수 사용)
    (
        "human",
        "## 변경 정보\n- 브랜치: {branch}\n- 언어: {language}\n\n## Diff\n{diff}"
    ),
])
```

메시지 순서 규칙:

```
[System]    역할 + 규칙            -- 항상 첫 번째
[Human]     Few-shot 입력 1        -- 예시 쌍 시작
[Assistant] Few-shot 출력 1
[Human]     Few-shot 입력 2        -- 필요한 만큼 반복
[Assistant] Few-shot 출력 2
[Human]     실제 입력              -- 항상 마지막
```

2.3 프롬프트 외부 파일 관리

프롬프트를 Python 코드 안에 하드코딩하면 다음과 같은 문제가 생깁니다: - 프롬프트를 수정할 때마다 코드를 배포해야 함 - 프롬프트 변경 이력 추적이 어려움 - A/B 테스트(두 프롬프트 비교)가 불가능

해결책: 프롬프트를 YAML 파일로 분리하고, 로더 유틸리티로 불러오는 방식.

Step 1: YAML 파일 형식 설계

116

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```yaml
# prompts/code_review/v1.yaml
version: "v1"
description: "코드 변경사항 분석 프롬프트"
created_at: "2026-03-23"

system: |
  당신은 시니어 소프트웨어 엔지니어로, 코드 변경사항을 분석합니다.

  ## 핵심 규칙
  - 구체적이고 테스트 가능한 시나리오를 식별하세요
  - 정상 케이스와 예외 케이스를 모두 포함하세요
  - 반드시 JSON 형식으로 응답하세요
human: |
  ## 변경 정보
  - 소스 브랜치: {source_branch}
  - 언어/프레임워크: {language} / {framework}

  ## 변경된 파일
  {changed_files}

  ## Diff
  {diff_text}

  위 변경사항을 분석해주세요.

few_shots:
  - input: |
      ## 변경 정보
      - 소스 브랜치: feature/user-auth
      - 언어: java
      ## Diff
      + @PostMapping("/api/v1/auth/login") ...
    output: |
      {"summary": "로그인 API 추가", "test_scenarios": [...]}
```

Step 2: 버전 관리 전략

각 프롬프트 디렉토리에 `_meta.yaml` 파일을 두어 활성 버전을 관리합니다:

```yaml
# prompts/code_review/_meta.yaml
active_version: "v1"
history:
  - version: "v1"
    activated_at: "2026-03-23"
    note: "초기 버전"
```

버전 관리 규칙: - 새 프롬프트는 `v2.yaml` 로 만들고, 테스트 후 `_meta.yaml` 의 `active_version` 을 변경 - 기존 버전 파일은 절대 삭제하지 않음 (롤백 가능) - 환경 변수 `PROMPT_VERSION_OVERRIDE` 로 특정 버전 강제 가능

Step 3: 프롬프트 로더 유틸리티 구현

117

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# prompts/loader.py
import os
from pathlib import Path
from functools import lru_cache
import yaml
from langchain_core.prompts import ChatPromptTemplate

TEMPLATES_DIR = Path(__file__).parent / "templates"


@lru_cache(maxsize=32)  # 동일 프롬프트 반복 로드 방지
def load_prompt(
    agent: str,
    node: str,
    version: str | None = None,
) -> ChatPromptTemplate:
    """YAML 파일에서 프롬프트를 로드하여 ChatPromptTemplate을 반환한다.

    Args:
        agent: 에이전트 이름 (예: "code_review", "email_classifier")
        node: 노드/태스크 이름 (예: "analyze", "classify")
        version: 프롬프트 버전. None이면 _meta.yaml의 active_version 사용

    Returns:
        ChatPromptTemplate 인스턴스
    """
    prompt_dir = TEMPLATES_DIR / agent / node

    # 버전 결정 우선순위: 환경변수 > 인자 > _meta.yaml
    env_override = os.getenv("PROMPT_VERSION_OVERRIDE")
    if env_override:
        version = env_override
    elif version is None:
        meta_path = prompt_dir / "_meta.yaml"
        with open(meta_path) as f:
            meta = yaml.safe_load(f)
        version = meta["active_version"]

    # YAML 로드
    yaml_path = prompt_dir / f"{version}.yaml"
    with open(yaml_path) as f:
        data = yaml.safe_load(f)

    # 메시지 구성: system + (few-shot) + human
    messages = [("system", data["system"])]

    # few-shot 예제가 있으면 system과 human 사이에 삽입
    if "few_shots" in data:
        for example in data["few_shots"]:
            messages.append(("human", example["input"]))
            messages.append(("assistant", example["output"]))

    messages.append(("human", data["human"]))
```


118

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
    return ChatPromptTemplate.from_messages(messages)
```

사용법

```
# Before: 코드에 프롬프트 하드코딩
PROMPT = ChatPromptTemplate.from_messages([...])  # 긴 프롬프트 텍스트
chain = PROMPT | llm | parser

# After: YAML에서 로드
from prompts.loader import load_prompt

prompt = load_prompt("code_review", "analyze")
chain = prompt | llm | parser
```

---

3. 실전 예제

3.1 언어/도메인별 프롬프트 특화

같은 "코드 분석" 작업이라도, 프로그래밍 언어에 따라 프롬프트를 다르게 작성해야 최상의 결과를 얻을 수 있습니다.

Java 코드 분석 프롬프트

```
# prompts/code_review/analyze/v1_java.yaml
system: |
  당신은 Java/Spring Boot 전문 시니어 개발자입니다.

  ## 분석 시 주의할 패턴
  - @Transactional 누락 여부
  - NullPointerException 가능성 (Optional 미사용)
  - @Valid/@Validated 어노테이션 누락
  - try-catch에서 Exception을 잡는 안티패턴
  - Spring Security 설정 미비

  ## 응답 규칙
  - 각 이슈에 심각도(HIGH/MEDIUM/LOW)를 부여하세요
  - 수정 코드 예시를 반드시 포함하세요
```

Python 코드 분석 프롬프트

119

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```yaml
# prompts/code_review/analyze/v1_python.yaml
system: |
  당신은 Python 전문 시니어 개발자입니다.

  ## 분석 시 주의할 패턴
  - 타입 힌트 누락 (함수 파라미터, 반환값)
  - mutable default argument (def f(x=[]))
  - bare except (except: 대신 except Exception:)
  - async/await 미사용 (I/O 바운드 작업)
  - f-string 대신 .format() 또는 % 사용

  ## 응답 규칙
  - PEP 8 준수 여부를 확인하세요
  - mypy 호환성을 고려하세요
```

언어에 따라 프롬프트 선택하기

```python
def get_review_prompt(language: str) -> ChatPromptTemplate:
    """언어에 맞는 코드 리뷰 프롬프트를 반환한다."""
    language_map = {
        "java": "v1_java",
        "python": "v1_python",
        "javascript": "v1_javascript",
    }
    version = language_map.get(language.lower(), "v1")  # 기본 범용 프롬프트
    return load_prompt("code_review", "analyze", version=version)
```

3.2 실전: "코드 리뷰 어시스턴트" 프롬프트 설계

전체 코드를 조합하여 완전한 코드 리뷰 어시스턴트를 만들어 봅시다:

120

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_anthropic import ChatAnthropic
from langchain_core.output_parsers import StrOutputParser

# 1) 프롬프트 설계 (Few-shot 포함)
review_prompt = ChatPromptTemplate.from_messages([
    ("system",
     "당신은 시니어 코드 리뷰어입니다.
"
     "코드의 버그, 보안 이슈, 개선점을 분석하세요.

"
     "## 출력 형식
"
     "JSON으로 응답하세요:
"
     '{"bugs": [{"line": N, "issue": "...", "fix": "..."}], '
     '"security": [...], "improvements": [...], "score": 1-10}'),
    # Few-shot 예시
    ("human", "언어: python
```

def login(pw): if pw == 'admin': return True

```
    ("assistant",
     '{"bugs": [], '
     '"security": [{"line": 2, "issue": "하드코딩된 비밀번호", "fix": "환경변수 또는 해시 비교
사용"}], '
     '"improvements": [{"line": 1, "issue": "타입 힌트 없음", "fix": "def login(pw: str) -
> bool:"}], '
     '"score": 3}'),
    # 실제 입력
    ("human", "언어: {language}
```

{code}

```
])

# 2) 체인 구성
llm = ChatAnthropic(model="claude-sonnet-4-5-20250514", temperature=0.1)
chain = review_prompt | llm | StrOutputParser()

# 3) 실행
result = chain.invoke({
    "language": "python",
    "code": "def get_user(id):
    conn = sqlite3.connect('db.sqlite')
"
            "    return conn.execute(f'SELECT * FROM users WHERE id={id}').fetchone()"
})
print(result)
# SQL Injection 취약점, 커넥션 미종료 등을 발견할 것입니다
```


121

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

4. 연습 문제

연습 1: 이메일 자동 응답 프롬프트 개선

과제: 아래의 Zero-shot 프롬프트를 Few-shot으로 개선하세요.

시작 코드 (Zero-shot):

```
email_prompt = ChatPromptTemplate.from_messages([
    ("system", "고객 이메일에 대한 자동 응답을 생성하세요. 정중하고 도움이 되는 톤을 사용하세요."),
    ("human", "고객 이메일: {email}

자동 응답을 작성하세요."),
])
```

요구사항: 1. 3가지 유형의 이메일에 대한 Few-shot 예시 추가 (문의, 불만, 환불 요청) 2. 각 예시의 응답은 150자 이내 3. 응답에 고객 이름을 포함하는 변수 `{customer_name}` 추가

힌트: Few-shot 예시에서는 변수를 사용하지 않고 고정 텍스트를 사용합니다. 변수는 마지막 Human 메시지에서만 사용합니다.

연습 2: YAML 프롬프트 파일 작성

과제: "회의록 요약기" 프롬프트를 YAML 파일로 작성하세요.

요구사항: - System 메시지: 회의록 요약 전문가 역할 - Human 메시지: 회의 제목, 참석자, 회의 내용을 변수로 받기 - Few-shot 예시 1개 포함 - JSON 출력 형식: `{"summary": "...", "action_items": [...], "decisions": [...]}`

5. 핵심 정리

122

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

개념	핵심 내용
프롬프트 구조	역할(Role) + 맥락(Context) + 지시(Instruction) + 형식(Format)
System vs Human	System = 규칙/역할 (불변), Human = 데이터/요청 (가변)
ChatPromptTemplate	`from_messages()` 로 생성, `{variable}` 로 변수 치환
체인 파이프라인	`prompt | llm | parser` – 파이프로 연결
Few-shot	예시 2~5개로 출력 품질/형식 안정화. Human/Assistant 쌍으로 삽입
외부 파일 관리	YAML로 분리 + `_meta.yaml` 로 버전 관리 + 로더 유틸리티
언어별 특화	같은 작업이라도 언어/프레임워크에 맞는 분석 기준 제공

---

6. 참고 자료

- **Anthropic 프롬프트 엔지니어링 가이드**: https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering
  - Claude에 특화된 프롬프트 작성 가이드. 역할 부여, 출력 형식, XML 태그 활용법 등을 다룹니다.
- **LangChain ChatPromptTemplate 문서**: https://python.langchain.com/docs/concepts/prompt_templates/
  - `from_messages()`, 변수 치환, MessagesPlaceholder 등 상세 사용법.
- **OpenAI 프롬프트 베스트 프랙티스**: https://platform.openai.com/docs/guides/prompt-engineering
  - 모델에 무관한 일반적인 프롬프트 작성 원칙. 명확한 지시, 참조 텍스트 활용 등.
- **LangChain Few-shot 예제**: https://python.langchain.com/docs/how_to/few_shot_examples_chat/
  - ChatPromptTemplate에 Few-shot을 추가하는 구체적인 코드 예시.

다음 단계

프롬프트 엔지니어링을 마스터했다면, 다음 모듈에서는 LLM의 **응답을 프로그램이 사용할 수 있는 구조화된 데이터로 변환**하는 방법을 배웁니다.

Module 06: 구조화된 출력 에서 다루는 내용: - LLM의 자유 텍스트 응답을 Pydantic 모델로 검증하기 - `with_structured_output()` 으로 스키마 강제하기 - JSON 출력 파서 패턴과 폴백 전략

123

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

⬆ 목차로 돌아가기

Module 06: 구조화된 출력 - LLM 응답을 데이터로 변환하기

학습 목표

이 모듈을 완료하면 다음을 할 수 있습니다:

1. LLM의 자유 텍스트 응답이 왜 프로그램에서 직접 사용하기 어려운지 이해한다
2. Pydantic 모델을 사용하여 출력 스키마를 정의할 수 있다
3. `with_structured_output()` 으로 LLM 응답을 강제할 수 있다
4. JSON 출력 파서를 구현하고 폴백 전략을 적용할 수 있다
5. 실전에서 사용할 수 있는 출력 스키마를 설계할 수 있다

사전 지식

- **Python 기초**: 클래스, 데코레이터, 타입 힌트
- **Module 05**: 프롬프트 엔지니어링 기본 (ChatPromptTemplate, 체인)
- **JSON 기초**: JSON 형식과 Python dict의 관계

> **용어 정리** - **구조화된 출력(Structured Output)**: LLM이 정해진 스키마에 맞게 응답하도록 강제하는 것 - **Pydantic**: Python의 데이터 검증 라이브러리. 타입 검사와 데이터 변환을 자동 수행 - **스키마(Schema)**: 데이터의 구조를 정의하는 설계도 (어떤 필드가 있고, 각 필드의 타입은 무엇인지) - **파싱(Parsing)**: 텍스트를 프로그램이 사용할 수 있는 데이터 구조로 변환하는 것

---

1. 개념 설명

1.1 문제: LLM은 자유 텍스트를 반환한다

124

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

LLM에게 "버그를 분석해줘"라고 요청하면, 다양한 형태로 응답할 수 있습니다:

응답 예시 A (깔끔한 JSON):

```
{"bugs": [{"line": 5, "issue": "null check missing"}], "severity": "HIGH"}
```

응답 예시 B (마크다운 코드블록에 감싸진 JSON):

```
분석 결과입니다:

\```json
{"bugs": [{"line": 5, "issue": "null check missing"}], "severity": "HIGH"}
\```

추가로 고려해야 할 사항은...
```

응답 예시 C (JSON이 아닌 자유 텍스트):

```
5번 줄에서 null 체크가 누락되어 있습니다.
심각도는 높음입니다.
```

프로그램에서 이 응답을 처리하려면 `result["bugs"][0]["line"]` 처럼 접근해야 하는데, 응답 형태가 매번 다르면 프로그램이 깨집니다.

1.2 접근법 비교

접근법	방식	신뢰도	난이도
텍스트 기반 JSON 스키마	프롬프트에 "이 형식으로 답하세요" 텍스트 추가	낮음 (LLM이 무시 가능)	쉬움
정규표현식 파싱	응답에서 정규식으로 데이터 추출	낮음 (형식 변경에 취약)	중간
JSON 파서 + 폴백	여러 단계로 JSON 추출 시도	중간 (대부분 성공)	중간
Pydantic + with_structured_output()	**LLM이 스키마에 맞게 생성하도록 강제**	**높음 (구조 보장)**	**쉬움**

> **결론**: `with_structured_output()` 이 가장 안정적이고, JSON 파서는 폴백용으로 함께 사용합니다.

125

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

2. 단계별 실습

2.1 Pydantic 모델 기초

Pydantic은 Python의 **데이터 검증 라이브러리**입니다. 클래스를 정의하면, 데이터가 그 구조에 맞는지 자동으로 검증해줍니다.

기본 사용법

```
# pip install pydantic

from pydantic import BaseModel, Field

# BaseModel을 상속하여 데이터 스키마 정의
class BugReport(BaseModel):
    """버그 리포트 스키마."""
    line: int = Field(description="버그가 있는 라인 번호", ge=1)      # ge=1: 1 이상
    issue: str = Field(description="버그 설명", min_length=1)         # min_length: 빈
        문자열 금지
    severity: str = Field(description="심각도", pattern=r"^(HIGH|MEDIUM|LOW)$")  # 정규식
        패턴

# 올바른 데이터 -> 성공
bug = BugReport(line=5, issue="null check missing", severity="HIGH")
print(bug.model_dump())
# {"line": 5, "issue": "null check missing", "severity": "HIGH"}

# 잘못된 데이터 -> 에러 발생
try:
    bad_bug = BugReport(line=-1, issue="", severity="CRITICAL")
except Exception as e:
    print(e)
    # line: Input should be greater than or equal to 1
    # issue: String should have at least 1 character
    # severity: String should match pattern '^(HIGH|MEDIUM|LOW)$'
```

Field 옵션 정리

126

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

옵션	용도	예시
`description`	필드 설명 (LLM에게 힌트 제공)	`Field(description="버그 설명")`
`min_length`	문자열 최소 길이	`Field(min_length=1)` – 빈 문자열 금지
`max_length`	문자열 최대 길이	`Field(max_length=500)`
`ge` / `le`	숫자 범위 (이상/이하)	`Field(ge=0, le=1.0)` – 0~1 범위
`pattern`	정규식 패턴 매칭	`Field(pattern=r"^(HIGH|MEDIUM|LOW)$")`
`default`	기본값	`Field(default="MEDIUM")`
`default_factory`	기본값 팩토리 (리스트 등)	`Field(default_factory=list)`

Enum으로 허용값 제한

```
from enum import Enum

class Severity(str, Enum):
    """심각도를 3가지로 제한."""
    HIGH = "HIGH"
    MEDIUM = "MEDIUM"
    LOW = "LOW"

class TestStatus(str, Enum):
    """테스트 결과를 3가지로 제한."""
    PASS = "PASS"
    FAIL = "FAIL"
    BLOCKED = "BLOCKED"

class TestResult(BaseModel):
    status: TestStatus                              # PASS, FAIL, BLOCKED만 허용
    confidence: float = Field(ge=0.0, le=1.0)       # 0.0 ~ 1.0 범위
    reasoning: str = Field(min_length=1)             # 빈 문자열 금지
    severity: Severity = Field(default=Severity.MEDIUM)  # 기본값: MEDIUM
```

field_validator로 커스텀 검증

127

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
from pydantic import BaseModel, Field, field_validator

class CodeAnalysis(BaseModel):
    affected_files: list[str] = Field(min_length=1)
    confidence: float = Field(ge=0.0, le=1.0)
    root_cause: str = Field(min_length=1)

    @field_validator("confidence")
    @classmethod
    def round_confidence(cls, v: float) -> float:
        """confidence를 소수점 2자리로 반올림."""
        return round(v, 2)

    @field_validator("affected_files")
    @classmethod
    def validate_file_paths(cls, v: list[str]) -> list[str]:
        """파일 경로가 비어있지 않은지 확인."""
        return [f for f in v if f.strip()]  # 빈 문자열 제거

# 사용
analysis = CodeAnalysis(
    affected_files=["src/main.py", "", "src/utils.py"],
    confidence=0.85678,
    root_cause="Missing null check"
)
print(analysis.confidence)       # 0.86 (반올림됨)
print(analysis.affected_files)   # ["src/main.py", "src/utils.py"] (빈 문자열 제거됨)
```

2.2 with_structured_output() 사용법

LangChain의 `with_structured_output()`은 LLM이 **Pydantic 모델에 맞는 출력을 생성하도록 강제**합니다.

작동 원리

```
기존 방식:
  프롬프트("JSON 형식으로 답해줘") -> LLM -> 자유 텍스트 -> 파서로 JSON 추출 (실패 가능)

with_structured_output 방식:
  Pydantic 모델 정의 -> LLM이 스키마에 맞게 생성 -> Pydantic 인스턴스 반환 (구조 보장)
```

내부적으로는 **제약된 디코딩(Constrained Decoding)**이라는 기술을 사용합니다. LLM이 토큰을 생성할 때, 스키마에 맞지 않는 토큰은 선택하지 못하도록 제한하는 방식입니다.

사용법

128

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
from langchain_anthropic import ChatAnthropic
from langchain_core.prompts import ChatPromptTemplate
from pydantic import BaseModel, Field
from enum import Enum

# 1) 출력 스키마 정의
class Severity(str, Enum):
    HIGH = "HIGH"
    MEDIUM = "MEDIUM"
    LOW = "LOW"

class Bug(BaseModel):
    line: int = Field(ge=1, description="버그가 있는 라인 번호")
    issue: str = Field(description="버그 설명")
    severity: Severity = Field(description="심각도")

class CodeReviewResult(BaseModel):
    """코드 리뷰 결과 스키마."""
    bugs: list[Bug] = Field(default_factory=list, description="발견된 버그 목록")
    score: int = Field(ge=1, le=10, description="코드 품질 점수 (1~10)")
    summary: str = Field(description="전체 리뷰 요약")

# 2) LLM에 structured output 적용
llm = ChatAnthropic(model="claude-sonnet-4-5-20250514", temperature=0.1)
structured_llm = llm.with_structured_output(CodeReviewResult)

# 3) 프롬프트 구성 (JSON 스키마 텍스트 불필요!)
prompt = ChatPromptTemplate.from_messages([
    ("system", "당신은 코드 리뷰 전문가입니다.\n주어진 코드의 버그를 분석하세요."),
    ("human", "언어: {language}\n코드:\n{code}"),
])

# 4) 체인 실행 (파서 불필요!)
chain = prompt | structured_llm
result = chain.invoke({
    "language": "python",
    "code": "def divide(a, b):\n    return a / b"
})

# result는 CodeReviewResult 인스턴스 (Pydantic 모델)
print(type(result))  # <class 'CodeReviewResult'>
print(result.bugs)   # [Bug(line=2, issue="ZeroDivisionError...", severity=
        <Severity.HIGH>)]
print(result.score)  # 4
```

> **주목**: `with_structured_output()` 을 사용하면 프롬프트에 JSON 스키마를 텍스트로 적을 필요가 없습니다. LLM이 Pydantic 모델의 구조를 자동으로 이해합니다.

2.3 JSON 출력 파서 패턴

129

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

`with_structured_output()`이 지원되지 않는 모델을 사용하거나, 폴백이 필요한 경우를 위한 JSON 파서를 만들어 봅시다.

기본 JSON 파서 (3단계 폴백)

LLM 응답에서 JSON을 추출하는 3단계 전략:

130

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
import json
import re
import logging

logger = logging.getLogger(__name__)


def parse_json_response(text: str) -> dict:
    """LLM 응답에서 JSON을 추출한다.

    3단계 폴백 전략:
    1) 직접 JSON 파싱 시도
    2) 마크다운 코드블록(```json ... ```) 내부 추출
    3) 첫 번째 { ... } 블록 탐색

    Args:
        text: LLM의 원본 응답 텍스트

    Returns:
        파싱된 딕셔너리

    Raises:
        ValueError: 모든 전략 실패 시
    """
    # 1단계: 직접 파싱 (응답 전체가 유효한 JSON인 경우)
    try:
        return json.loads(text.strip())
    except json.JSONDecodeError:
        pass

    # 2단계: 마크다운 코드블록 추출
    code_block_pattern = r"```(?:json)?\s*\n?(.*?)\n?\s*```"
    matches = re.findall(code_block_pattern, text, re.DOTALL)
    for match in matches:
        try:
            return json.loads(match.strip())
        except json.JSONDecodeError:
            continue

    # 3단계: 중괄호 블록 탐색 (가장 바깥쪽 { ... })
    brace_start = text.find("{")
    brace_end = text.rfind("}")
    if brace_start != -1 and brace_end != -1 and brace_end > brace_start:
        try:
            return json.loads(text[brace_start:brace_end + 1])
        except json.JSONDecodeError:
            pass

    raise ValueError(f"JSON 파싱 실패. 원본 응답: {text[:200]}...")
```

ValidatedJsonOutputParser: 파싱 + Pydantic 검증 결합

JSON 파싱에 성공해도, 구조가 맞는지 추가로 검증해야 합니다:

131

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
from pydantic import BaseModel, ValidationError


class ValidatedJsonOutputParser:
    """JSON 파싱 + Pydantic 스키마 검증을 결합한 파서.

    Usage:
        parser = ValidatedJsonOutputParser(schema=MyModel)
        result = parser.parse(llm_response_text)
    """

    def __init__(self, schema: type[BaseModel] | None = None, strict: bool = False):
        self.schema = schema
        self.strict = strict  # True: 검증 실패 시 예외, False: 원본 dict 반환

    def parse(self, text: str) -> dict:
        """텍스트를 파싱하고 스키마로 검증한다."""
        # 1) JSON 파싱
        raw = parse_json_response(text)

        # 2) 스키마 검증 (스키마가 지정된 경우)
        if self.schema is None:
            return raw

        try:
            validated = self.schema.model_validate(raw)
            logger.info("Schema validation passed: %s", self.schema.__name__)
            return validated.model_dump()
        except ValidationError as exc:
            logger.warning(
                "Schema validation failed: %s, errors: %s",
                self.schema.__name__,
                exc.errors()[:3],  # 처음 3개 오류만 로그
            )
            if self.strict:
                raise

            # 부분 복구: Pydantic의 기본값으로 누락 필드를 채워봄
            try:
                validated = self.schema.model_validate(raw, strict=False)
                return validated.model_dump()
            except ValidationError:
                logger.error("Recovery failed, returning raw dict")
                return raw
```

사용법:

132

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
from langchain_core.output_parsers import StrOutputParser

# 방법 1: with_structured_output (권장)
structured_llm = llm.with_structured_output(CodeReviewResult)
chain = prompt | structured_llm

# 방법 2: JSON 파서 + Pydantic 검증 (폴백용)
parser = ValidatedJsonOutputParser(schema=CodeReviewResult)
chain = prompt | llm | StrOutputParser()
raw_text = chain.invoke({...})
result = parser.parse(raw_text)
```

2.4 스키마 설계 모범 사례

필수 vs 선택 필드

```
class AnalysisResult(BaseModel):
    # 필수 필드: 반드시 있어야 하는 핵심 데이터
    summary: str = Field(min_length=1, description="분석 요약")
    affected_files: list[str] = Field(min_length=1, description="영향받는 파일")

    # 선택 필드: 없을 수도 있는 부가 데이터
    confidence: float = Field(default=0.5, ge=0.0, le=1.0, description="신뢰도")
    tags: list[str] = Field(default_factory=list, description="분류 태그")
    notes: str = Field(default="", description="추가 메모")
```

기본값 전략

```
class TestEvaluation(BaseModel):
    status: TestStatus                                      # 필수 (기본값 없음)
    confidence: float = Field(default=0.5, ge=0.0, le=1.0)  # 기본 0.5 (판단 불가 시)
    reasoning: str = Field(default="분석 정보 부족")          # 기본 메시지
    defects: list[str] = Field(default_factory=list)         # 기본 빈 리스트
```

> **원칙**: 핵심 필드는 필수로, 부가 필드는 합리적인 기본값과 함께 선택 필드로 설정합니다.

중첩 모델

133

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
class DefectInfo(BaseModel):
    """발견된 결함 정보."""
    description: str = Field(min_length=1)
    affected_file: str = Field(default="")
    severity: Severity

class TestEvaluation(BaseModel):
    """테스트 평가 결과."""
    status: TestStatus
    confidence: float = Field(ge=0.0, le=1.0)
    reasoning: str = Field(min_length=1)
    defects_found: list[DefectInfo] = Field(default_factory=list)  # 중첩 모델 리스트
```

---

3. 실전 예제

3.1 “버그 리포트 분석기” 출력 스키마 설계

완전한 버그 리포트 분석기를 만들어 봅시다:

134

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
from pydantic import BaseModel, Field, field_validator
from enum import Enum
from langchain_anthropic import ChatAnthropic
from langchain_core.prompts import ChatPromptTemplate


# --- 스키마 정의 ---
class BugCategory(str, Enum):
    LOGIC = "logic"            # 로직 오류
    NULL_REFERENCE = "null_reference"  # Null 참조
    SECURITY = "security"      # 보안 취약점
    PERFORMANCE = "performance"  # 성능 이슈
    OTHER = "other"

class Priority(str, Enum):
    CRITICAL = "CRITICAL"
    HIGH = "HIGH"
    MEDIUM = "MEDIUM"
    LOW = "LOW"

class SuggestedFix(BaseModel):
    description: str = Field(description="수정 방법 설명")
    code_snippet: str = Field(default="", description="수정 코드 예시")

class BugAnalysis(BaseModel):
    """단일 버그 분석 결과."""
    title: str = Field(min_length=1, description="버그 제목 (한 줄)")
    category: BugCategory = Field(description="버그 카테고리")
    priority: Priority = Field(description="우선순위")
    affected_lines: list[int] = Field(description="영향받는 라인 번호들")
    description: str = Field(min_length=10, description="상세 설명")
    suggested_fix: SuggestedFix = Field(description="수정 제안")

    @field_validator("affected_lines")
    @classmethod
    def validate_lines(cls, v: list[int]) -> list[int]:
        return [line for line in v if line > 0]

class BugReportAnalysis(BaseModel):
    """전체 버그 리포트 분석 결과."""
    total_bugs: int = Field(ge=0, description="발견된 총 버그 수")
    bugs: list[BugAnalysis] = Field(default_factory=list)
    overall_risk: Priority = Field(description="전체 위험도")
    summary: str = Field(min_length=1, description="분석 요약")

    @field_validator("total_bugs")
    @classmethod
    def match_bug_count(cls, v: int, info) -> int:
        bugs = info.data.get("bugs", [])
        if bugs and v != len(bugs):
            return len(bugs)
        return v
```


135

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# --- 프롬프트 + 체인 ---
prompt = ChatPromptTemplate.from_messages([
    ("system",
     "당신은 시니어 버그 분석가입니다.\n"
     "주어진 코드의 버그를 분석하고 수정 방안을 제시하세요."),
    ("human", "파일: {file_path}\n언어: {language}\n\n```\n{code}\n```"),
])

llm = ChatAnthropic(model="claude-sonnet-4-5-20250514", temperature=0.1)
structured_llm = llm.with_structured_output(BugReportAnalysis)
chain = prompt | structured_llm

# --- 실행 ---
result = chain.invoke({
    "file_path": "src/auth/login_service.py",
    "language": "python",
    "code": (
        "def authenticate(email, password):\n"
        "    user = db.query(f'SELECT * FROM users WHERE email={email}')\n"
        "    if user.password == password:\n"
        "        return create_token(user)\n"
        "    return None"
    ),
})

print(f"총 {result.total_bugs}개 버그 발견")
for bug in result.bugs:
    print(f"  [{bug.priority.value}] {bug.title} (카테고리: {bug.category.value})")
    print(f"    설명: {bug.description}")
    print(f"    수정: {bug.suggested_fix.description}")
```

---

4. 연습 문제

연습 1: "회의록 요약기" Pydantic 모델 설계

과제: 회의록을 요약하는 LLM의 출력 스키마를 Pydantic으로 설계하세요.

요구사항: 1. `MeetingSummary` 모델에 다음 필드 포함: - `title`: 회의 제목 (필수, 빈 문자열 금지) - `date`: 회의 날짜 (문자열, 필수) - `attendees`: 참석자 목록 (최소 1명) - `summary`: 요약 (최소 20자) - `action_items`: 액션 아이템 목록 (각 항목은 중첩 모델) - `decisions`: 결정 사항 목록

2. `ActionItem` 중첩 모델:
   - `task`: 할 일 설명 (필수)
   - `assignee`: 담당자 (필수)
   - `deadline`: 마감일 (선택, 기본값 "미정")
   - `priority`: HIGH/MEDIUM/LOW (Enum)

136

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

3. `with_structured_output()` 을 사용하는 체인 구성

힌트: Enum과 Field의 옵션을 적극 활용하세요.

연습 2: JSON 파서 테스트

과제: `parse_json_response()` 함수를 다음 3가지 입력에 대해 테스트하세요.

```
# 입력 1: 깔끔한 JSON
text1 = '{"status": "PASS", "score": 8}'

# 입력 2: 마크다운 코드블록
text2 = "분석 결과입니다:\\n```json\\n{\"status\": \"FAIL\", \"score\": 3}\\n```"

# 입력 3: 텍스트 속에 묻힌 JSON
text3 = '결과를 알려드립니다. {"status": "BLOCKED", "score": 5} 추가 분석이 필요합니다.'
```

각 입력에 대해 파싱 결과를 확인하고, 어떤 단계(1단계/2단계/3단계)에서 성공했는지 설명하세요.

---

5. 핵심 정리

개념	핵심 내용
문제	LLM은 자유 텍스트를 반환하지만, 프로그램은 구조화된 데이터가 필요
Pydantic	`BaseModel` 상속으로 스키마 정의, `Field` 로 제약 조건 설정
Enum	허용 값을 명시적으로 제한 (예: HIGH/MEDIUM/LOW)
field_validator	커스텀 검증 로직 추가 (반올림, 빈 값 필터링 등)
with_structured_output()	LLM이 Pydantic 스키마에 맞게 응답하도록 강제. 가장 안정적
JSON 파서 폴백	3단계: 직접 파싱 -> 코드블록 추출 -> 중괄호 탐색
ValidatedJsonOutputParser	JSON 파싱 + Pydantic 검증 결합. 폴백용
스키마 설계	필수 필드는 기본값 없이, 선택 필드는 합리적 기본값과 함께

---

6. 참고 자료

- **Pydantic 공식 문서**: https://docs.pydantic.dev/latest/
  - BaseModel, Field, Validator의 모든 옵션을 상세히 설명합니다.

137

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

- **LangChain Structured Output 가이드**: https://python.langchain.com/docs/how_to/structured_output/
  - `with_structured_output()` 의 사용법과 다양한 LLM 프로바이더별 차이점.
- **LangChain Output Parsers**: https://python.langchain.com/docs/concepts/output_parsers/
  - StrOutputParser, JsonOutputParser 등 다양한 파서의 사용법.
- **Anthropic Structured Output**: https://docs.anthropic.com/en/docs/build-with-claude/structured-output
  - Claude의 네이티브 JSON Schema 지원에 대한 설명.

다음 단계

구조화된 출력을 마스터했다면, 다음 모듈에서는 LLM 호출의 **비용, 속도, 안정성을 최적화**하는 방법을 배웁니다.

Module 07: LLM 호출 최적화 에서 다루는 내용: - 모델별 가격 비교와 태스크에 맞는 모델 선택 - 토큰 사용량 추적과 비용 모니터링 - 재시도, 타임아웃, 캐싱 전략

138

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

⬆ 목차로 돌아가기

Module 07: LLM 호출 최적화 - 비용, 속도, 안정성

학습 목표

이 모듈을 완료하면 다음을 할 수 있습니다:

1. LLM 비용 구조(입력/출력 토큰, 모델별 가격)를 이해한다
2. 태스크 복잡도에 따라 적절한 모델을 선택하는 ModelRouter를 구현할 수 있다
3. LangChain Callback으로 토큰 사용량을 추적할 수 있다
4. LLM 응답 캐싱으로 중복 호출을 방지할 수 있다
5. 타임아웃과 재시도로 안정적인 LLM 호출을 구현할 수 있다

사전 지식

- **Module 05**: 프롬프트 엔지니어링 (ChatPromptTemplate, 체인)
- **Module 06**: 구조화된 출력 (Pydantic, with_structured_output)
- **Python 기초**: 클래스, 데코레이터, 딕셔너리, 예외 처리

> **용어 정리** - **토큰(Token)**: LLM이 텍스트를 처리하는 최소 단위. "Hello world" = 약 2토큰, "안녕하세요" = 약 3~5토큰. - **입력 토큰**: LLM에게 보내는 텍스트의 토큰 수 (프롬프트 + 데이터) - **출력 토큰**: LLM이 생성하는 응답의 토큰 수 - **Callback**: LLM 호출의 시작/종료/에러 시점에 실행되는 후크(hook) 함수 - **TTL (Time To Live)**: 캐시 데이터의 유효 기간

---

1. 개념 설명

1.1 LLM 비용 구조 이해

LLM API는 **토큰 단위로 과금**됩니다. 중요한 점은 입력 토큰과 출력 토큰의 가격이 다르다는 것입니다.

139

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

토큰이란?

```
영어: "Hello, how are you?" = 5 tokens
한국어: "안녕하세요, 잘 지내시나요?" = 약 12 tokens (한국어는 토큰이 더 많이 필요)

코드: "def calculate(x, y): return x + y" = 약 10 tokens
```

> **경험 법칙**: 영어 1단어 = 약 1.3토큰, 한국어 1글자 = 약 1~2토큰

모델별 가격 비교표 (USD / 1M tokens 기준)

모델	입력 가격	출력 가격	특징	적합한 용도
Claude Haiku 3.5	$1.00	$5.00	빠르고 저렴	분류, 필터링, 간단한 분석
Claude Sonnet 4	$3.00	$15.00	균형잡힌 성능	코드 생성, 상세 분석
Claude Opus 4	$15.00	$75.00	최고 성능	복잡한 추론, 연구
GPT-4o-mini	$0.15	$0.60	매우 저렴	간단한 작업
GPT-4o	$2.50	$10.00	높은 성능	범용

> **핵심 인사이트**: 출력 토큰이 입력 토큰보다 3~5배 비쌉니다. 따라서 출력 길이를 제어하는 것이 비용 절감의 핵심입니다.

비용 계산 예시

140

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
def estimate_cost(model: str, input_tokens: int, output_tokens: int) -> float:
    """LLM 호출 비용을 추정한다 (USD).

    Args:
        model: 모델 이름
        input_tokens: 입력 토큰 수
        output_tokens: 출력 토큰 수

    Returns:
        추정 비용 (USD)
    """
    # 모델별 가격표 (USD per 1M tokens)
    pricing = {
        "claude-haiku":  {"input": 1.00, "output": 5.00},
        "claude-sonnet": {"input": 3.00, "output": 15.00},
        "claude-opus":   {"input": 15.00, "output": 75.00},
        "gpt-4o-mini":   {"input": 0.15, "output": 0.60},
        "gpt-4o":        {"input": 2.50, "output": 10.00},
    }

    price = pricing.get(model, pricing["claude-sonnet"])  # 기본값: Sonnet
    cost = (input_tokens * price["input"] + output_tokens * price["output"]) / 1_000_000
    return round(cost, 6)

# 예시: Sonnet으로 코드 분석 (입력 2000토큰, 출력 500토큰)
cost = estimate_cost("claude-sonnet", input_tokens=2000, output_tokens=500)
print(f"비용: ${cost}")  # $0.0135

# 같은 작업을 Haiku로 하면?
cost_haiku = estimate_cost("claude-haiku", input_tokens=2000, output_tokens=500)
print(f"Haiku 비용: ${cost_haiku}")  # $0.0045 (67% 절감!)
```

1.2 왜 최적화가 필요한가?

모든 LLM 호출에 가장 비싼 모델을 사용하면:

```
에이전트 파이프라인: 분석 -> 생성 -> 검증 (3번 호출)
- 모두 Sonnet: $0.0135 x 3 = $0.0405/건
- 하루 1,000건 처리: $40.50/일 = $1,215/월

최적화 후 (분석은 Haiku, 생성은 Sonnet):
- Haiku + Sonnet + Haiku: $0.0045 + $0.0135 + $0.0045 = $0.0225/건
- 하루 1,000건 처리: $22.50/일 = $675/월 (44% 절감)
```

---

2. 단계별 실습

2.1 모델 라우팅 (Model Routing)

141

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

핵심 아이디어: 간단한 작업에는 저렴한 모델을, 복잡한 작업에는 고성능 모델을 사용합니다.

태스크 복잡도별 모델 선택 기준표

태스크 유형	예시	권장 모델 (Tier)	근거
분류/라우팅	"이 이슈는 버그인가 기능요청인가?"	Fast (Haiku)	간단한 판단, 짧은 출력
요약/추출	"변경사항을 요약해줘"	Fast (Haiku)	구조화된 입력 -> 짧은 출력
코드 분석 (소규모)	"이 함수의 버그를 찾아줘"	Fast (Haiku)	단일 파일, 짧은 코드
코드 분석 (대규모)	"이 PR 전체를 분석해줘"	Smart (Sonnet)	다중 파일, 복잡한 관계
코드 생성	"버그를 수정하는 코드를 작성해줘"	Smart (Sonnet)	정밀한 코드 생성 필요
테스트 케이스 생성	"테스트 시나리오를 만들어줘"	Smart (Sonnet)	창의적 + 구조적 생성

ModelRouter 클래스 구현

142

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
import logging
from dataclasses import dataclass
from enum import Enum
from langchain_anthropic import ChatAnthropic

logger = logging.getLogger(__name__)


class ModelTier(str, Enum):
    """모델 등급."""
    FAST = "fast"      # Haiku: 빠르고 저렴
    SMART = "smart"    # Sonnet: 정밀하고 고성능


@dataclass
class RoutingRule:
    """라우팅 규칙."""
    task_name: str              # 태스크 이름
    default_tier: ModelTier     # 기본 모델 등급
    upgrade_threshold: int | None = None  # 이 바이트 이상이면 Smart로 업그레이드


class ModelRouter:
    """태스크 복잡도 기반 LLM 모델 라우터.

    간단한 태스크에는 Fast(저렴) 모델을, 복잡한 태스크에는 Smart(고성능) 모델을 할당한다.

    Usage:
        router = ModelRouter()
        llm = router.get_model("classify")         # -> Haiku
        llm = router.get_model("generate_code")    # -> Sonnet
        llm = router.get_model("analyze", input_size=10000)  # -> 크기가 크면 Sonnet
    """

    def __init__(
        self,
        fast_model: str = "claude-haiku-4-5-20250514",
        smart_model: str = "claude-sonnet-4-5-20250514",
        temperature: float = 0.1,
        rules: list[RoutingRule] | None = None,
    ):
        self._fast_model = fast_model
        self._smart_model = smart_model
        self._temperature = temperature

        # 기본 라우팅 테이블
        default_rules = [
            RoutingRule("classify", ModelTier.FAST),
            RoutingRule("summarize", ModelTier.FAST),
            RoutingRule("analyze", ModelTier.FAST, upgrade_threshold=5120),
            RoutingRule("generate_code", ModelTier.SMART),
            RoutingRule("generate_test", ModelTier.SMART),
            RoutingRule("evaluate", ModelTier.SMART),
        ]
```


143

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
        self._rules = {r.task_name: r for r in (rules or default_rules)}
        self._cache: dict[str, ChatAnthropic] = {}

    def get_model(self, task_name: str, input_size: int = 0) -> ChatAnthropic:
        """태스크에 적합한 LLM 인스턴스를 반환한다.

        Args:
            task_name: 태스크 이름 (라우팅 테이블에서 조회)
            input_size: 입력 데이터 크기 (바이트). 동적 업그레이드 판단용.

        Returns:
            ChatAnthropic 인스턴스
        """
        rule = self._rules.get(task_name)

        if rule is None:
            logger.warning("No routing rule for '%s', using smart model", task_name)
            tier = ModelTier.SMART
        else:
            tier = rule.default_tier
            # 입력 크기가 임계값 초과 시 Smart로 업그레이드
            if (
                tier == ModelTier.FAST
                and rule.upgrade_threshold
                and input_size > rule.upgrade_threshold
            ):
                logger.info(
                    "Upgrading %s: input_size=%d > threshold=%d",
                    task_name, input_size, rule.upgrade_threshold,
                )
                tier = ModelTier.SMART

        model_name = self._fast_model if tier == ModelTier.FAST else self._smart_model

        # 모델 인스턴스 캐싱 (동일 모델은 재사용)
        if model_name not in self._cache:
            self._cache[model_name] = ChatAnthropic(
                model=model_name,
                temperature=self._temperature,
            )

        logger.info("Routed: %s -> %s (%s)", task_name, model_name, tier.value)
        return self._cache[model_name]
```

사용 예시

144

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
router = ModelRouter()

# 간단한 분류 -> Haiku (저렴)
classify_llm = router.get_model("classify")
# 코드 생성 -> Sonnet (고성능)
generate_llm = router.get_model("generate_code")
# 작은 코드 분석 -> Haiku
analyze_llm = router.get_model("analyze", input_size=1000)
# 큰 코드 분석 -> Sonnet (자동 업그레이드)
analyze_llm_big = router.get_model("analyze", input_size=10000)
```

비용 절감 추정

시나리오	전부 Sonnet	ModelRouter 적용	절감율
분류 (1,000건/일)	$13.50	$4.50 (Haiku)	-67%
코드 분석 (500건, 평균)	$6.75	~$4.00 (혼합)	-41%
코드 생성 (200건/일)	$2.70	$2.70 (Sonnet)	0%
합계	**$22.95/일**	**$11.20/일**	**-51%**

2.2 토큰 사용량 추적

LLM 비용을 관리하려면 먼저 **얼마나 사용하고 있는지** 알아야 합니다.

LangChain Callback 메커니즘

LangChain은 LLM 호출의 생명주기에 **Callback**을 걸 수 있습니다:

```
LLM 호출 시작  --->  on_llm_start()   : 시작 시간 기록
       |
       v
LLM 응답 수신  --->  on_llm_end()     : 토큰 수, 비용 계산
       |
       v
LLM 오류 발생  --->  on_llm_error()   : 실패 카운터 증가
```

토큰 카운터 콜백 구현

145

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
import time
import logging
from langchain_core.callbacks import BaseCallbackHandler
from langchain_core.outputs import LLMResult

logger = logging.getLogger(__name__)


class TokenCounterCallback(BaseCallbackHandler):
    """LLM 호출의 토큰 사용량과 비용을 추적하는 콜백.

    Usage:
        counter = TokenCounterCallback(agent_name="my-agent")
        llm = ChatAnthropic(model="...", callbacks=[counter])
        llm.invoke(...)
        print(counter.total_cost)
    """

    # 모델별 가격표 (USD per 1M tokens)
    PRICING = {
        "claude-haiku":  {"input": 1.00, "output": 5.00},
        "claude-sonnet": {"input": 3.00, "output": 15.00},
        "default":       {"input": 3.00, "output": 15.00},
    }

    def __init__(self, agent_name: str = "unknown"):
        super().__init__()
        self.agent_name = agent_name
        self.total_input_tokens = 0
        self.total_output_tokens = 0
        self.total_cost = 0.0
        self.call_count = 0
        self._start_time: float | None = None

    def on_llm_start(self, serialized, prompts, **kwargs):
        """LLM 호출 시작 시 호출됨."""
        self._start_time = time.monotonic()

    def on_llm_end(self, response: LLMResult, **kwargs):
        """LLM 호출 완료 시 호출됨."""
        elapsed = time.monotonic() - self._start_time if self._start_time else 0

        # 토큰 사용량 추출 (LLM 프로바이더에 따라 위치가 다를 수 있음)
        usage = {}
        if response.llm_output:
            usage = response.llm_output.get("usage", {})

        input_tokens = usage.get("input_tokens", 0)
        output_tokens = usage.get("output_tokens", 0)

        # 비용 계산
        model = response.llm_output.get("model_name", "default") if response.llm_output
         else "default"
        # 모델명에서 가격 키 추출 ("claude-haiku-4-5-..." -> "claude-haiku")
```


146

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
        price_key = "default"
        for key in self.PRICING:
            if key in model:
                price_key = key
                break

        price = self.PRICING[price_key]
        cost = (input_tokens * price["input"] + output_tokens * price["output"]) /
        1_000_000

        # 누적
        self.total_input_tokens += input_tokens
        self.total_output_tokens += output_tokens
        self.total_cost += cost
        self.call_count += 1

        logger.info(
            "[%s] LLM call #%d: input=%d, output=%d, cost=$%.6f, elapsed=%.2fs",
            self.agent_name, self.call_count, input_tokens, output_tokens, cost,
        elapsed,
        )

    def on_llm_error(self, error, **kwargs):
        """LLM 호출 실패 시 호출됨."""
        logger.error("[%s] LLM call failed: %s", self.agent_name, str(error))

    def get_summary(self) -> dict:
        """사용량 요약을 반환한다."""
        return {
            "agent": self.agent_name,
            "calls": self.call_count,
            "input_tokens": self.total_input_tokens,
            "output_tokens": self.total_output_tokens,
            "total_cost_usd": round(self.total_cost, 6),
        }
```

사용법

147

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
# 콜백 생성
counter = TokenCounterCallback(agent_name="code-reviewer")

# LLM 생성 시 콜백 주입
llm = ChatAnthropic(
    model="claude-sonnet-4-5-20250514",
    temperature=0.1,
    callbacks=[counter],  # 콜백 등록
)

# 체인 실행 (자동으로 콜백이 호출됨)
chain = prompt | llm | StrOutputParser()
result = chain.invoke({"language": "python", "code": "..."})

# 사용량 확인
print(counter.get_summary())
# {"agent": "code-reviewer", "calls": 1, "input_tokens": 1523,
#  "output_tokens": 487, "total_cost_usd": 0.011844}
```

Prometheus 메트릭 연동 개념

프로덕션 환경에서는 Prometheus + Grafana로 메트릭을 시각화합니다:

```
# prometheus_client 패키지 사용 (개념 설명)
from prometheus_client import Counter, Histogram

# 메트릭 정의 (애플리케이션 시작 시 1회)
llm_tokens_total = Counter(
    "llm_tokens_total",
    "Total LLM tokens used",
    ["agent", "model", "direction"],  # direction: input/output
)
llm_cost_total = Counter(
    "llm_cost_dollars_total",
    "Total LLM cost in USD",
    ["agent", "model"],
)
llm_duration = Histogram(
    "llm_call_duration_seconds",
    "LLM call duration",
    ["agent", "model"],
)

# TokenCounterCallback의 on_llm_end에서 메트릭 기록
# llm_tokens_total.labels(agent="my-agent", model="sonnet", direction="input").inc(1523)
# llm_cost_total.labels(agent="my-agent", model="sonnet").inc(0.011844)
```

2.3 LLM 응답 캐싱

동일한 입력에 대해 LLM을 반복 호출하는 것은 비용 낭비입니다. 캐싱으로 이를 방지합니다.

148

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

캐싱이 필요한 경우

상황	캐싱 적합도	이유
동일 코드를 여러 번 분석	HIGH	같은 입력 = 같은 결과
재처리/재시도	HIGH	이전 결과 재사용 가능
코드 생성 (비결정적)	LOW	매번 다른 결과가 나올 수 있음
사용자 대화	LOW	맥락이 매번 다름

해시 기반 캐시 키 설계

```
import hashlib
import json


def compute_cache_key(task_name: str, model: str, inputs: dict) -> str:
    """LLM 호출의 캐시 키를 생성한다.

    동일한 태스크/모델/입력이면 같은 키를 반환한다.
    입력 딕셔너리의 키 순서가 달라도 같은 키가 생성된다.
    """
    key_data = {
        "task": task_name,
        "model": model,
        "inputs": inputs,
    }
    serialized = json.dumps(key_data, sort_keys=True, ensure_ascii=False)
    return hashlib.sha256(serialized.encode("utf-8")).hexdigest()

# 같은 입력 -> 같은 키
key1 = compute_cache_key("analyze", "sonnet", {"code": "def f(): pass", "lang":
        "python"})
key2 = compute_cache_key("analyze", "sonnet", {"lang": "python", "code": "def f():
        pass"})
assert key1 == key2  # 키 순서가 달라도 같은 해시
```

간단한 인메모리 캐시 구현

149

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
import time
import logging
from typing import Any

logger = logging.getLogger(__name__)


class LLMCache:
    """LLM 응답 캐시 (메모리 기반).

    TTL(Time To Live) 기반으로 캐시를 관리하며, 히트율을 추적한다.

    Usage:
        cache = LLMCache(default_ttl=3600)  # 1시간 TTL

        # 캐시 조회
        cached = cache.get("analyze", "sonnet", {"code": "..."})
        if cached:
            return cached  # 캐시 히트!

        # LLM 호출
        result = chain.invoke(...)

        # 캐시 저장
        cache.set("analyze", "sonnet", {"code": "..."}, result)
    """

    def __init__(self, default_ttl: int = 3600, max_entries: int = 500):
        self._default_ttl = default_ttl
        self._max_entries = max_entries
        self._store: dict[str, tuple[Any, float]] = {}  # key -> (value, expires_at)
        self._hits = 0
        self._misses = 0

    def get(self, task: str, model: str, inputs: dict) -> Any | None:
        """캐시에서 결과를 조회한다. 없거나 만료되면 None."""
        key = compute_cache_key(task, model, inputs)

        if key in self._store:
            value, expires_at = self._store[key]
            if time.time() < expires_at:
                self._hits += 1
                logger.info("Cache HIT: %s (hit_rate=%.1f%%)", task, self.hit_rate *
        100)
                return value
            else:
                del self._store[key]  # 만료된 항목 삭제

        self._misses += 1
        return None

    def set(self, task: str, model: str, inputs: dict, result: Any, ttl: int | None =
        None):
        """결과를 캐시에 저장한다."""
        key = compute_cache_key(task, model, inputs)
```


150

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
        ttl = ttl or self._default_ttl

        # 용량 초과 시 가장 오래된 항목 제거
        if len(self._store) >= self._max_entries:
            oldest = min(self._store, key=lambda k: self._store[k][1])
            del self._store[oldest]

        self._store[key] = (result, time.time() + ttl)
        logger.info("Cache SET: %s, ttl=%ds, entries=%d", task, ttl, len(self._store))

    @property
    def hit_rate(self) -> float:
        """캐시 히트율 (0.0 ~ 1.0)."""
        total = self._hits + self._misses
        return self._hits / total if total > 0 else 0.0
```

2.4 타임아웃과 재시도

LLM API 호출은 네트워크를 통해 이루어지므로 실패할 수 있습니다.

LLM API 호출 실패 유형

HTTP 코드	의미	재시도 가능?	대응
429	Rate Limit Exceeded	O	대기 후 재시도
500	Internal Server Error	O	즉시 재시도
503	Service Unavailable	O	대기 후 재시도
Timeout	응답 시간 초과	O	타임아웃 늘리거나 재시도
400	Bad Request	X	입력 수정 필요
401	Unauthorized	X	API 키 확인

기본 재시도 래퍼 구현

151

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
import time
import logging
from typing import TypeVar, Callable

logger = logging.getLogger(__name__)
T = TypeVar("T")


class LLMCallError(Exception):
    """LLM 호출 실패 (재시도 소진 후)."""
    def __init__(self, task: str, attempts: int, last_error: Exception):
        super().__init__(f"LLM call failed after {attempts} attempts: [{task}]
        {last_error}")
        self.task = task
        self.attempts = attempts
        self.last_error = last_error


def resilient_llm_call(
    fn: Callable[[], T],
    task_name: str,
    *,
    max_retries: int = 2,
    initial_delay: float = 1.0,
    backoff_factor: float = 2.0,
) -> T:
    """LLM 호출을 재시도 + 지수 백오프로 보호한다.

    Args:
        fn: LLM 호출 함수 (인자 없는 callable)
        task_name: 태스크 이름 (로깅용)
        max_retries: 최대 재시도 횟수
        initial_delay: 첫 재시도 대기 시간 (초)
        backoff_factor: 대기 시간 배율 (지수 백오프)

    Returns:
        fn()의 반환값

    Raises:
        LLMCallError: 모든 재시도 실패 시
    """
    max_attempts = max_retries + 1
    delay = initial_delay
    last_error = None

    for attempt in range(1, max_attempts + 1):
        try:
            start = time.monotonic()
            result = fn()
            elapsed = time.monotonic() - start

            logger.info(
                "[%s] LLM call succeeded: attempt=%d, elapsed=%.2fs",
                task_name, attempt, elapsed,
```


152

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
            )
            return result

        except Exception as exc:
            last_error = exc
            if attempt == max_attempts:
                break

            logger.warning(
                "[%s] LLM call failed (attempt %d/%d), retrying in %.1fs: %s",
                task_name, attempt, max_attempts, delay, str(exc),
            )
            time.sleep(delay)
            delay *= backoff_factor  # 1초 -> 2초 -> 4초 (지수 백오프)

    raise LLMCallError(task_name, max_attempts, last_error)
```

사용법

```
# Before: 실패 시 즉시 에러
result = chain.invoke({"code": "..."})

# After: 2번 재시도, 지수 백오프
result = resilient_llm_call(
    fn=lambda: chain.invoke({"code": "..."}),
    task_name="analyze_code",
    max_retries=2,        # 최대 2번 재시도 (총 3번 시도)
    initial_delay=1.0,    # 첫 재시도: 1초 대기
    backoff_factor=2.0,   # 두 번째 재시도: 2초 대기
)
```

지수 백오프 시각화:

```
시도 1: 즉시 실행 -> 실패
  (1초 대기)
시도 2: 재시도 -> 실패
  (2초 대기)
시도 3: 재시도 -> 성공! (또는 LLMCallError 발생)
```

2.5 동적 max_tokens

입력 길이에 따라 `max_tokens`를 조정하면 불필요한 출력을 방지하고 비용을 절감할 수 있습니다.

153

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
def calculate_max_tokens(input_text: str, task_type: str) -> int:
    """입력 길이와 태스크 유형에 따라 max_tokens를 계산한다.

    Args:
        input_text: 입력 텍스트
        task_type: 태스크 유형 ("classify", "analyze", "generate")

    Returns:
        적절한 max_tokens 값
    """
    input_length = len(input_text)

    # 태스크별 출력 비율
    ratios = {
        "classify": 0.05,     # 분류: 입력의 5% 정도 출력
        "summarize": 0.2,     # 요약: 입력의 20%
        "analyze": 0.3,       # 분석: 입력의 30%
        "generate": 0.5,      # 생성: 입력의 50%
    }

    ratio = ratios.get(task_type, 0.3)
    estimated = int(input_length * ratio)

    # 최소/최대 제한
    min_tokens = 100
    max_tokens = 4096
    return max(min_tokens, min(estimated, max_tokens))

# 사용 예시
short_code = "def f(): pass"
long_code = "..." * 5000  # 매우 긴 코드

print(calculate_max_tokens(short_code, "classify"))   # 100 (최소값)
print(calculate_max_tokens(long_code, "generate"))     # 4096 (최대값)
```

---

3. 실전 예제

3.1 비용 효율적인 "코드 분석 + 수정 생성" 파이프라인

모든 최적화 기법을 조합한 완전한 파이프라인을 만들어 봅시다:

154

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# 1) 라우터, 캐시, 콜백 초기화
router = ModelRouter()
cache = LLMCache(default_ttl=3600)
counter = TokenCounterCallback(agent_name="code-pipeline")


def analyze_and_fix(code: str, language: str) -> dict:
    """코드를 분석하고 수정을 생성하는 파이프라인.

    Step 1: 코드 분석 (Haiku - 저렴) - 캐싱 적용
    Step 2: 수정 생성 (Sonnet - 고성능)
    """
    inputs = {"code": code, "language": language}

    # --- Step 1: 코드 분석 (Fast tier + 캐싱) ---
    cached_analysis = cache.get("analyze", "haiku", inputs)
    if cached_analysis:
        analysis = cached_analysis
    else:
        analyze_llm = router.get_model("analyze", input_size=len(code))
        analyze_prompt = ChatPromptTemplate.from_messages([
            ("system", "코드의 버그를 분석하세요. JSON으로 응답하세요."),
            ("human", "언어: {language}\n코드:\n{code}"),
        ])
        chain = analyze_prompt | analyze_llm | StrOutputParser()
        analysis = resilient_llm_call(
            fn=lambda: chain.invoke(inputs),
            task_name="analyze",
        )
        cache.set("analyze", "haiku", inputs, analysis)

    # --- Step 2: 수정 생성 (Smart tier) ---
    fix_llm = router.get_model("generate_code")
    fix_prompt = ChatPromptTemplate.from_messages([
        ("system", "분석 결과를 바탕으로 코드를 수정하세요."),
        ("human", "분석 결과:\n{analysis}\n\n원본 코드:\n{code}"),
    ])
    fix_chain = fix_prompt | fix_llm | StrOutputParser()
    fix = resilient_llm_call(
        fn=lambda: fix_chain.invoke({"analysis": analysis, "code": code}),
        task_name="generate_code",
    )

    # --- 결과 ---
    print(f"\n=== 비용 요약 ===")
    print(f"총 호출: {counter.call_count}회")
    print(f"총 비용: ${counter.total_cost:.6f}")
    print(f"캐시 히트율: {cache.hit_rate:.1%}")

    return {"analysis": analysis, "fix": fix}
```


155

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

4. 연습 문제

연습 1: ModelRouter를 적용한 2-노드 에이전트 만들기

과제: 다음 2단계 파이프라인을 ModelRouter로 최적화하세요.

```
# 현재: 모든 단계에 동일한 Sonnet 모델 사용
llm = ChatAnthropic(model="claude-sonnet-4-5-20250514")

# Step 1: 이메일 분류 (문의/불만/칭찬/기타)
classify_chain = classify_prompt | llm | StrOutputParser()

# Step 2: 자동 응답 생성
respond_chain = respond_prompt | llm | StrOutputParser()
```

요구사항: 1. ModelRouter를 생성하고 적절한 RoutingRule을 정의하세요 2. Step 1(분류)은 Fast 모델, Step 2(응답 생성)는 Smart 모델 사용 3. TokenCounterCallback을 추가하여 각 단계의 비용을 확인하세요 4. Step 1에 LLMCache를 적용하세요 (같은 이메일은 캐싱)

연습 2: 재시도 래퍼 테스트

과제: `resilient_llm_call` 의 동작을 테스트하세요.

```
# 테스트용: 처음 2번은 실패, 3번째에 성공하는 함수
call_count = 0

def flaky_function():
    global call_count
    call_count += 1
    if call_count < 3:
        raise Exception(f"Simulated failure #{call_count}")
    return "Success!"

# resilient_llm_call로 감싸서 호출하세요
# max_retries=3으로 설정하면 성공해야 합니다
```

5. 핵심 정리

156

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

개념	핵심 내용
토큰 비용	출력 토큰이 입력의 3~5배 비쌈. 출력 길이 제어가 핵심
ModelRouter	간단한 작업 = Fast(Haiku), 복잡한 작업 = Smart(Sonnet). 입력 크기로 동적 업그레이드
Callback	`on_llm_start/end/error` 로 토큰, 비용, 시간 추적
캐싱	동일 입력 반복 호출 방지. 해시 기반 키 + TTL 관리
재시도	지수 백오프로 429/500/timeout 대응. `resilient_llm_call()`
max_tokens	입력 길이와 태스크 유형에 따라 동적 조정
비용 모니터링	일/월 예산 설정 + 초과 시 알림

---

6. 참고 자료

- **Anthropic 모델 가격**: https://docs.anthropic.com/en/docs/about-claude/models
  - Claude 모델별 가격, 컨텍스트 윈도우, 성능 비교.
- **LangChain Callbacks 가이드**: https://python.langchain.com/docs/concepts/callbacks/
  - BaseCallbackHandler, AsyncCallbackHandler 등 콜백 메커니즘 상세 설명.
- **LangChain Caching 가이드**: https://python.langchain.com/docs/how_to/llm_caching/
  - InMemoryCache, SQLiteCache 등 LangChain 내장 캐싱 사용법.
- **OpenAI Tokenizer**: https://platform.openai.com/tokenizer
  - 텍스트를 입력하면 토큰 수를 확인할 수 있는 도구 (Claude도 유사한 토큰화).

다음 단계

LLM 호출 최적화를 마스터했다면, 다음 모듈에서는 여러 LLM 호출을 **그래프 기반 워크플로우로 연결**하는 방법을 배웁니다.

Module 08: LangGraph 기초 에서 다루는 내용: - 노드와 엣지로 에이전트 워크플로우 구성하기 - StateGraph로 상태 관리하기 - 조건부 분기와 루프 구현하기

157

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

⬆ 목차로 돌아가기

Module 08: 에러 처리와 회복 탄력성

> 실패해도 멈추지 않는 에이전트 만들기

학습 목표

이 모듈을 완료하면 다음을 할 수 있습니다:

1. AI 에이전트에서 발생하는 에러 유형을 분류하고, 재시도 가능 여부를 판단할 수 있다
2. Exponential Backoff + Jitter를 이해하고 직접 구현할 수 있다
3. LangGraph의 RetryPolicy를 노드에 적용할 수 있다
4. Circuit Breaker 패턴을 이해하고 에이전트에 적용할 수 있다
5. 우아한 실패 처리(Graceful Degradation)로 DLQ 직행을 방지할 수 있다

사전 지식

주제	수준	설명
Python 기초	필수	try/except, 데코레이터, 클래스
LangGraph 기초	필수	Module 05~07에서 다룬 StateGraph, 노드, 엣지
HTTP 기초	권장	상태 코드(200, 429, 500 등)의 의미
asyncio	선택	비동기 프로그래밍 기초 (없어도 학습 가능)

---

1. 개념 설명

1.1 왜 에러 처리가 중요한가?

158

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

AI 에이전트는 혼자서 동작하지 않습니다. LLM API를 호출하고, 외부 서비스(Jira, Slack, DB 등)와 통신하며, 파일을 읽고 쓰는 등 다양한 외부 시스템에 의존합니다.

```
┌──────────────┐     ┌──────────┐     ┌──────────┐
│  AI 에이전트  │──>│  LLM API │     │  Jira    │
│              │──>│ (Claude) │     │  API     │
│  Node A      │     └──────────┘     └──────────┘
│  Node B      │──>┌──────────┐     ┌──────────┐
│  Node C      │     │  Slack   │     │ Database │
│              │──>│  API     │──>│          │
└──────────────┘     └──────────┘     └──────────┘
```

이 중 **하나라도 장애가 발생하면** 에이전트 전체가 멈출 수 있습니다. 하지만 모든 에러가 치명적인 것은 아닙니다. 일시적인 네트워크 오류는 잠시 기다렸다 재시도하면 해결되고, 서버 과부하는 시간이 지나면 회복됩니다.

> **핵심 질문**: "이 에러는 다시 시도하면 성공할 수 있는가?"

1.2 에이전트 시스템에서 발생하는 에러 유형

AI 에이전트에서 만나는 에러를 4가지 범주로 나눌 수 있습니다:

(1) LLM API 에러

LLM(Claude, GPT 등)을 호출할 때 발생하는 에러입니다.

에러	HTTP 코드	원인	빈도
Rate Limit	429	분당 요청 수 초과	높음
Server Error	500, 503	LLM 서버 일시 장애	보통
Timeout	-	응답 시간 초과 (긴 프롬프트)	보통
Authentication	401	API 키 만료/잘못됨	낮음
Bad Request	400	프롬프트가 너무 길거나 잘못된 형식	낮음

(2) 외부 API 에러

Jira, Slack, GitLab 등 외부 서비스 호출 시 발생합니다.

159

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

에러	HTTP 코드	원인	빈도
Rate Limit	429	API 호출 한도 초과	보통
서버 오류	500, 502, 503	외부 서비스 장애	낮음
권한 없음	403	토큰 권한 부족	낮음
리소스 없음	404	잘못된 ID/경로	보통

(3) 데이터 에러

LLM 응답이나 입력 데이터를 처리할 때 발생합니다.

에러	원인	예시
JSON 파싱 실패	LLM이 잘못된 JSON 반환	`json.JSONDecodeError`
입력 데이터 부재	필수 필드가 비어있음	`affected_files` 가 빈 리스트
스키마 불일치	LLM 응답이 예상 구조와 다름	필수 키 누락

(4) 시스템 에러

실행 환경 자체의 문제입니다.

에러	원인	예시
메모리 부족	대용량 데이터 처리	`MemoryError`
디스크 풀	Git clone 누적	`OSError: No space left`
네트워크 단절	DNS 실패, 연결 불가	`ConnectionError`

1.3 Retriable vs Non-retriable 에러 분류

에러를 만났을 때 가장 먼저 해야 할 질문: **"다시 시도하면 성공할 가능성이 있는가?"**

160

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
에러 발생!
    │
    ▼
┌──────────────────┐
│ 재시도하면 성공할까? │
└──────────────────┘
          │
    ┌─────┴─────┐
    │           │
    ▼           ▼
  [YES]       [NO]
 Retriable   Non-retriable
    │           │
    ▼           ▼
 재시도!     에러 기록 후
 (backoff)   다른 경로로
```

상세 분류표

에러 유형	분류	재시도	이유
LLM Rate Limit (429)	**Retriable**	O (최대 3회, 2/4/8초)	시간이 지나면 한도 초기화
LLM Timeout	**Retriable**	O (최대 2회, 5/10초)	일시적 서버 부하 가능성
LLM Server Error (5xx)	**Retriable**	O (최대 2회, 3/6초)	서버 재시작/복구 가능
HTTP 401 Unauthorized	**Non-retriable**	X	인증 정보가 바뀌어야 해결
HTTP 403 Forbidden	**Non-retriable**	X	권한 설정 변경 필요
HTTP 404 Not Found	**Non-retriable**	X	존재하지 않는 리소스
JSON 파싱 실패	**Non-retriable**	X	같은 입력 = 같은 실패
입력 데이터 부재	**Non-retriable**	X	데이터가 없으면 계속 없음
Jira/GitLab 429, 5xx	**Conditional**	O (해당 코드만)	서버 일시 문제
파일 시스템 오류	**Non-retriable**	X	구조적 문제

> **Conditional(조건부)**: HTTP 상태 코드에 따라 재시도 여부가 달라집니다. 429, 500, 502, 503, 504는

161

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

> 재시도하고, 400, 401, 403, 404는 재시도하지 않습니다.

---

2. 단계별 실습

2.1 Exponential Backoff + Jitter

개념: 도서관 비유

도서관에서 인기 있는 책을 빌리려고 합니다. 사서에게 갔더니 "지금 바빠요"라고 합니다.

- **단순 재시도**: 1초 후 다시 가기 -> 또 바빠요 -> 1초 후 다시 가기 -> …
  - 문제: 다른 사람들도 1초마다 와서 사서가 더 바빠집니다
- **Exponential Backoff**: 1초 후 -> 2초 후 -> 4초 후 -> 8초 후 …
  - 점점 더 오래 기다려서 사서에게 여유를 줍니다
- **Jitter**: 1~2초 후 -> 2~4초 후 -> 4~8초 후 …
  - 무작위 대기 시간으로, 동시에 몰리는 것을 방지합니다

```
단순 재시도 (모두 1초 간격):
사용자A: —X——X——X——X——X——> (모두 같은 시점에 재시도)
사용자B: —X——X——X——X——X——>
사용자C: —X——X——X——X——X——>
                 ↑ 서버 과부하!

Exponential Backoff + Jitter:
사용자A: —X———X————X——————————X——>
사용자B: —X————X——————X————————————X——>
사용자C: —X———X——————X————————X——>
                 ↑ 재시도가 분산되어 서버 부하 감소
```

Thundering Herd (몰려오는 짐승 떼) 문제

Jitter가 없으면, 동시에 실패한 100개의 요청이 **정확히 같은 시점에** 재시도합니다. 이를 "Thundering Herd"라고 합니다. Jitter(랜덤 지연)를 추가하면 재시도 시점이 분산되어 서버 부하를 줄입니다.

구현: retry_with_backoff 함수

162

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
import time
import random
import logging

logger = logging.getLogger(__name__)


def retry_with_backoff(
    fn,
    max_retries: int = 3,
    base_delay: float = 1.0,
    max_delay: float = 30.0,
    backoff_factor: float = 2.0,
    jitter: bool = True,
    retriable_exceptions: tuple = (Exception,),
):
    """Exponential Backoff + Jitter로 함수를 재시도합니다.

    Args:
        fn: 실행할 함수 (인자 없는 callable)
        max_retries: 최대 재시도 횟수
        base_delay: 첫 번째 재시도 전 기본 대기 시간(초)
        max_delay: 최대 대기 시간(초)
        backoff_factor: 대기 시간 증가 배수 (보통 2.0)
        jitter: True면 랜덤 지연 추가
        retriable_exceptions: 재시도할 예외 타입

    Returns:
        fn()의 반환값

    Raises:
        마지막 시도에서 발생한 예외
    """
    last_exception = None

    for attempt in range(max_retries + 1):  # 최초 시도 + 재시도
        try:
            return fn()
        except retriable_exceptions as exc:
            last_exception = exc

            if attempt >= max_retries:
                # 마지막 시도도 실패
                logger.error(
                    "모든 재시도 실패",
                    extra={
                        "attempts": attempt + 1,
                        "error": str(exc)[:200],
                    },
                )
                raise

            # 대기 시간 계산: base_delay * (backoff_factor ^ attempt)
            delay = min(base_delay * (backoff_factor ** attempt), max_delay)
```


163

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
            # Jitter 추가: 0 ~ delay 사이의 랜덤 값
            if jitter:
                delay = random.uniform(0, delay)

            logger.warning(
                "재시도 대기 중",
                extra={
                    "attempt": attempt + 1,
                    "max_retries": max_retries,
                    "delay_seconds": round(delay, 2),
                    "error_type": type(exc).__name__,
                },
            )
            time.sleep(delay)

    raise last_exception
```

사용 예시

```
import httpx

def call_llm_api(prompt: str) -> str:
    """LLM API를 호출하는 함수."""
    response = httpx.post(
        "https://api.anthropic.com/v1/messages",
        json={"prompt": prompt},
        headers={"Authorization": "Bearer YOUR_KEY"},
        timeout=60.0,
    )
    response.raise_for_status()
    return response.json()["content"]


# Exponential Backoff로 LLM API 호출
result = retry_with_backoff(
    fn=lambda: call_llm_api("안녕하세요"),
    max_retries=3,
    base_delay=2.0,
    retriable_exceptions=(httpx.HTTPStatusError, httpx.TimeoutException),
)
```

2.2 LangGraph RetryPolicy

LangGraph는 노드에 **내장 RetryPolicy**를 설정할 수 있습니다. `add_node()` 의 `retry=` 파라미터로 전달합니다.

기본 사용법

164

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
from langgraph.graph import StateGraph
from langgraph.pregel import RetryPolicy
from typing import TypedDict


class AgentState(TypedDict):
    query: str
    result: str
    error: str | None


# RetryPolicy 정의
llm_retry_policy = RetryPolicy(
    max_attempts=3,           # 최대 3번 시도
    initial_interval=2.0,     # 첫 대기: 2초
    backoff_factor=2.0,       # 배수: 2 -> 4 -> 8초
    max_interval=30.0,        # 최대 대기: 30초
    jitter=True,              # 랜덤 지연 추가
)

# 노드 함수 정의
def analyze_node(state: AgentState) -> dict:
    """LLM을 호출하여 분석하는 노드."""
    # 이 함수에서 예외가 발생하면 RetryPolicy에 따라 자동 재시도
    result = call_llm("분석해주세요: " + state["query"])
    return {"result": result}


# 그래프에 노드 추가 (retry 정책 적용)
graph = StateGraph(AgentState)
graph.add_node("analyze", analyze_node, retry=llm_retry_policy)  # retry= 파라미터!
```

RetryPolicy 파라미터 상세

파라미터	타입	기본값	설명
`max_attempts`	int	3	최대 시도 횟수 (최초 포함)
`initial_interval`	float	1.0	첫 재시도 전 대기 시간(초)
`backoff_factor`	float	2.0	대기 시간 증가 배수
`max_interval`	float	128.0	최대 대기 시간(초)
`jitter`	bool	True	랜덤 지연 추가 여부
`retry_on`	callable	None	예외를 받아 재시도 여부를 반환하는 함수

165

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

retry_on으로 특정 예외만 재시도

```
import anthropic
import httpx


def should_retry(exc: Exception) -> bool:
    """재시도 가능한 예외인지 판별합니다."""
    # LLM Rate Limit -> 재시도
    if isinstance(exc, anthropic.RateLimitError):
        return True
    # LLM 서버 오류 -> 재시도
    if isinstance(exc, anthropic.InternalServerError):
        return True
    # 타임아웃 -> 재시도
    if isinstance(exc, (httpx.ReadTimeout, httpx.ConnectTimeout)):
        return True
    # 그 외 -> 재시도하지 않음
    return False


llm_retry_policy = RetryPolicy(
    max_attempts=3,
    initial_interval=2.0,
    backoff_factor=2.0,
    retry_on=should_retry,  # 이 함수로 재시도 여부 결정
)
```

내장 RetryPolicy의 한계

RetryPolicy는 편리하지만, **재시도를 모두 소진하면 예외를 그대로 raise**합니다. 즉, 그래프 전체가 중단됩니다. 에러를 "부드럽게" 처리하려면 (예: state에 에러 기록 후 다른 경로로 이동) 커스텀 데코레이터가 필요합니다.

2.3 커스텀 retry_on_llm_error 데코레이터

실전에서는 다음 두 가지를 모두 처리해야 합니다: - **Retriable 에러** -> backoff 재시도 - **Non-retriable 에러** -> `state["error"]`에 기록하고 failure 경로로 이동

166

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
import time
import logging
from functools import wraps

import anthropic
import httpx

logger = logging.getLogger(__name__)

# 재시도 가능한 예외 목록
RETRIABLE_EXCEPTIONS = (
    anthropic.RateLimitError,       # 429 Rate Limit
    anthropic.InternalServerError,  # 500 서버 오류
    anthropic.APIConnectionError,   # 네트워크 연결 실패
    httpx.ReadTimeout,              # 읽기 타임아웃
    httpx.ConnectTimeout,           # 연결 타임아웃
)


def retry_on_llm_error(
    max_retries: int = 3,
    base_delay: float = 2.0,
    max_delay: float = 30.0,
    backoff_factor: float = 2.0,
):
    """LLM 호출 노드용 재시도 데코레이터.

    - Retriable 에러: exponential backoff로 재시도
    - Non-retriable 에러: state["error"]에 기록하고 반환
    - 모든 재시도 실패: state["error"]에 기록하고 반환 (DLQ 직행 방지)

    사용법:
        @retry_on_llm_error(max_retries=3, base_delay=2.0)
        def my_node(state: MyState) -> dict:
            result = call_llm(...)
            return {"result": result}
    """
    def decorator(node_fn):
        @wraps(node_fn)
        def wrapper(state, **kwargs):
            last_exception = None

            for attempt in range(max_retries + 1):
                try:
                    # 노드 함수 실행
                    return node_fn(state, **kwargs)

                except RETRIABLE_EXCEPTIONS as exc:
                    # --- Retriable: 재시도 ---
                    last_exception = exc

                    if attempt < max_retries:
                        delay = min(
                            base_delay * (backoff_factor ** attempt),
```


167

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
                        max_delay,
                    )
                    logger.warning(
                        "LLM 호출 실패 (재시도 가능), 재시도 중...",
                        extra={
                            "node": node_fn.__name__,
                            "attempt": attempt + 1,
                            "max_retries": max_retries,
                            "delay_seconds": delay,
                            "error_type": type(exc).__name__,
                        },
                    )
                    time.sleep(delay)
                else:
                    logger.error(
                        "LLM 호출 - 모든 재시도 실패",
                        extra={
                            "node": node_fn.__name__,
                            "total_attempts": max_retries + 1,
                            "error_type": type(exc).__name__,
                        },
                    )

            except Exception as exc:
                # --- Non-retriable: 즉시 에러 반환 ---
                logger.error(
                    "LLM 호출 실패 (재시도 불가)",
                    extra={
                        "node": node_fn.__name__,
                        "error_type": type(exc).__name__,
                        "error": str(exc)[:500],
                    },
                )
                return {
                    "error": f"[{node_fn.__name__}] {type(exc).__name__}: {exc}",
                    "current_step": node_fn.__name__,
                }

        # 모든 재시도를 소진한 경우: state["error"]에 기록 (DLQ 직행 방지)
        return {
            "error": (
                f"[{node_fn.__name__}] {type(last_exception).__name__}: "
                f"{last_exception} (after {max_retries + 1} attempts)"
            ),
            "current_step": node_fn.__name__,
        }

    return wrapper
return decorator
```

데코레이터 적용 예시

168

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# 적용 전: 에러 발생 시 raise -> 그래프 전체 중단 -> DLQ
def analyze_node(state: AgentState) -> dict:
    try:
        result = call_llm(state["query"])
        return {"result": result, "current_step": "analyze"}
    except Exception as exc:
        logger.error("분석 실패", error=str(exc))
        raise  # -> 그래프 중단!


# 적용 후: Retriable은 재시도, Non-retriable은 에러 경로로
@retry_on_llm_error(max_retries=3, base_delay=2.0)
def analyze_node(state: AgentState) -> dict:
    result = call_llm(state["query"])
    return {"result": result, "current_step": "analyze"}
    # Retriable 에러 -> 자동 재시도 (최대 3회)
    # Non-retriable 에러 -> state["error"] 설정 -> 조건부 엣지에서 failure로 라우팅
```

2.4 Circuit Breaker 패턴

개념: 가정의 차단기 비유

집에서 전기를 너무 많이 쓰면 차단기(두꺼비집)가 내려갑니다. 이는 과부하로 인한 화재를 방지하기 위한 **안전 장치**입니다.

소프트웨어에서 Circuit Breaker도 같은 역할을 합니다: - 외부 시스템(Jira, LLM 등)이 계속 실패하면 -> 요청을 **차단** - 이유: 실패할 게 뻔한 요청에 시간(타임아웃 30초 x 재시도 3회 = 90초)을 낭비하지 않기 위해 - 일정 시간 후 -> 소량의 요청으로 복구 여부 **확인**

상태 전이 다이어그램

```
                 연속 실패 >= threshold (예: 5회)
   CLOSED ──────────────────────────────────> OPEN
  (정상)                                     (차단)
     ^                                          |
     |                                          | timeout 경과 (예: 60초)
     |                                          v
     |        테스트 성공                    HALF_OPEN
     └──────────────────────────────────── (테스트 중)
                                                |
                                                | 테스트 실패
                                                v
                                              OPEN
                                             (다시 차단)
```

각 상태의 역할:

169

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

상태	동작	비유
CLOSED (정상)	모든 요청을 통과시키고, 실패 횟수를 추적	차단기 올라감 (전기 사용 가능)
OPEN (차단)	모든 요청을 즉시 거부 (타임아웃 대기 없음)	차단기 내려감 (전기 차단)
HALF_OPEN (테스트)	1건만 통과시켜 복구 확인	차단기 살짝 올려서 테스트

구현: CircuitBreaker 클래스

170

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
import time
import threading
import logging
from enum import Enum
from dataclasses import dataclass, field

logger = logging.getLogger(__name__)


class CircuitState(Enum):
    """Circuit Breaker의 3가지 상태."""
    CLOSED = "closed"       # 정상 - 요청 통과
    OPEN = "open"           # 차단 - 요청 거부
    HALF_OPEN = "half_open" # 테스트 - 제한적 통과


class CircuitOpenError(Exception):
    """Circuit Breaker가 OPEN 상태일 때 발생하는 예외."""
    def __init__(self, name: str, remaining_seconds: float):
        self.name = name
        self.remaining_seconds = remaining_seconds
        super().__init__(
            f"Circuit '{name}'이 OPEN 상태입니다. "
            f"{remaining_seconds:.1f}초 후 재시도하세요."
        )


@dataclass
class CircuitBreaker:
    """Circuit Breaker 구현.

    외부 시스템 호출을 감싸서, 연속 실패 시 자동으로 차단합니다.

    Args:
        name: 서킷 식별자 (예: "jira-api", "llm-api")
        failure_threshold: OPEN으로 전환되는 연속 실패 횟수 (기본 5)
        recovery_timeout: OPEN에서 HALF_OPEN으로 전환되는 대기 시간(초) (기본 60)
        expected_exceptions: 실패로 카운트할 예외 타입
    """
    name: str
    failure_threshold: int = 5
    recovery_timeout: float = 60.0
    expected_exceptions: tuple = (Exception,)

    # 내부 상태 (외부에서 초기화하지 않음)
    _state: CircuitState = field(default=CircuitState.CLOSED, init=False)
    _failure_count: int = field(default=0, init=False)
    _last_failure_time: float = field(default=0.0, init=False)
    _lock: threading.Lock = field(default_factory=threading.Lock, init=False)

    @property
    def state(self) -> CircuitState:
        """현재 상태를 반환합니다. OPEN -> HALF_OPEN 자동 전환 포함."""
        with self._lock:
```


171

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
        if self._state == CircuitState.OPEN:
            elapsed = time.monotonic() - self._last_failure_time
            if elapsed >= self.recovery_timeout:
                self._state = CircuitState.HALF_OPEN
                logger.info(
                    f"Circuit '{self.name}': OPEN -> HALF_OPEN "
                    f"({elapsed:.1f}초 경과)"
                )
        return self._state

    def _record_success(self) -> None:
        """성공을 기록합니다."""
        with self._lock:
            self._failure_count = 0
            if self._state == CircuitState.HALF_OPEN:
                self._state = CircuitState.CLOSED
                logger.info(f"Circuit '{self.name}': HALF_OPEN -> CLOSED (복구됨)")

    def _record_failure(self) -> None:
        """실패를 기록합니다."""
        with self._lock:
            self._failure_count += 1
            self._last_failure_time = time.monotonic()
            if self._failure_count >= self.failure_threshold:
                self._state = CircuitState.OPEN
                logger.warning(
                    f"Circuit '{self.name}': -> OPEN "
                    f"(연속 {self._failure_count}회 실패)"
                )

    def call(self, fn, *args, **kwargs):
        """Circuit Breaker를 통해 함수를 호출합니다.

        Args:
            fn: 호출할 함수
            *args, **kwargs: 함수 인자

        Returns:
            fn()의 반환값

        Raises:
            CircuitOpenError: 서킷이 OPEN 상태일 때
        """
        current_state = self.state

        # OPEN 상태: 즉시 거부 (타임아웃 대기 없이 빠르게 실패)
        if current_state == CircuitState.OPEN:
            remaining = self.recovery_timeout - (
                time.monotonic() - self._last_failure_time
            )
            raise CircuitOpenError(self.name, max(0, remaining))

        # CLOSED 또는 HALF_OPEN: 요청 통과
        try:
            result = fn(*args, **kwargs)
```


172

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
            self._record_success()
            return result
        except self.expected_exceptions:
            self._record_failure()
            raise
```

에이전트 노드에 Circuit Breaker 적용

```
import httpx

# Jira API용 Circuit Breaker 생성
jira_circuit = CircuitBreaker(
    name="jira-api",
    failure_threshold=5,          # 5번 연속 실패하면 차단
    recovery_timeout=60.0,        # 60초 후 복구 시도
    expected_exceptions=(httpx.HTTPError, httpx.TimeoutException),
)


def create_jira_ticket_node(state: AgentState) -> dict:
    """Jira 티켓을 생성하는 노드."""
    try:
        # Circuit Breaker를 통해 Jira API 호출
        result = jira_circuit.call(
            _do_jira_request,
            url="https://your-jira.atlassian.net/rest/api/3/issue",
            payload={"fields": {"summary": state["title"]}},
        )
        return {"jira_key": result["key"]}

    except CircuitOpenError as exc:
        # 서킷 열림 -> 즉시 실패 (타임아웃 없이)
        logger.warning(f"Jira 서킷 열림: {exc.remaining_seconds:.0f}초 후 재시도")
        return {"error": str(exc)}

    except Exception as exc:
        # 기타 에러
        return {"error": f"Jira 호출 실패: {exc}"}


def _do_jira_request(url: str, payload: dict) -> dict:
    """실제 Jira API 호출."""
    with httpx.Client(timeout=30) as client:
        response = client.post(url, json=payload)
        response.raise_for_status()
    return response.json()
```

---

3. 실전 예제

173

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

외부 API + LLM 호출이 있는 에이전트에 전체 에러 처리 적용

실전에서는 하나의 에이전트에 LLM 호출, 외부 API 호출, 데이터 처리가 모두 포함됩니다. 이 예제에서는 3-노드 에이전트에 지금까지 배운 모든 패턴을 적용합니다.

```
[fetch_data] ──> [analyze] ──> [create_ticket]
 (외부 API)       (LLM)         (외부 API)
     │               │              │
     ▼               ▼              ▼
  Circuit          Retry         Circuit
  Breaker         Decorator      Breaker
```

전체 코드

174

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
import operator
import httpx
import logging
from typing import Annotated, TypedDict
from langgraph.graph import StateGraph, END

logger = logging.getLogger(__name__)


# ── 1. State 정의 ──

def _replace(existing, new):
    """기존 값을 새 값으로 교체하는 reducer."""
    return new


class TaskState(TypedDict):
    """에이전트의 공유 상태."""
    query: Annotated[str, _replace]
    fetched_data: Annotated[str | None, _replace]
    analysis: Annotated[str | None, _replace]
    ticket_key: Annotated[str | None, _replace]
    error: Annotated[str | None, _replace]
    current_step: Annotated[str, _replace]


# ── 2. Circuit Breaker 설정 ──

api_circuit = CircuitBreaker(
    name="external-api",
    failure_threshold=5,
    recovery_timeout=60.0,
    expected_exceptions=(httpx.HTTPError, httpx.TimeoutException),
)

ticket_circuit = CircuitBreaker(
    name="ticket-api",
    failure_threshold=5,
    recovery_timeout=60.0,
    expected_exceptions=(httpx.HTTPError, httpx.TimeoutException),
)


# ── 3. 노드 정의 ──

def fetch_data_node(state: TaskState) -> dict:
    """외부 API에서 데이터를 가져오는 노드. (Circuit Breaker 적용)"""
    try:
        data = api_circuit.call(
            _call_external_api, state["query"]
        )
        return {"fetched_data": data, "current_step": "fetch_data"}
    except CircuitOpenError as exc:
        return {"error": f"API 서킷 열림: {exc}", "current_step": "fetch_data"}
```


175

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
    except Exception as exc:
        return {"error": f"데이터 가져오기 실패: {exc}", "current_step": "fetch_data"}


@retry_on_llm_error(max_retries=3, base_delay=2.0)
def analyze_node(state: TaskState) -> dict:
    """LLM으로 분석하는 노드. (Retry 데코레이터 적용)"""
    if not state.get("fetched_data"):
        return {"error": "분석할 데이터 없음", "current_step": "analyze"}

    result = call_llm(f"다음을 분석하세요: {state['fetched_data']}")
    return {"analysis": result, "current_step": "analyze"}


def create_ticket_node(state: TaskState) -> dict:
    """외부 시스템에 티켓을 생성하는 노드. (Circuit Breaker 적용)"""
    try:
        key = ticket_circuit.call(
            _create_ticket, state["analysis"]
        )
        return {"ticket_key": key, "current_step": "create_ticket"}
    except CircuitOpenError as exc:
        return {"error": f"티켓 서킷 열림: {exc}", "current_step": "create_ticket"}
    except Exception as exc:
        return {"error": f"티켓 생성 실패: {exc}", "current_step": "create_ticket"}


def handle_error_node(state: TaskState) -> dict:
    """에러를 로깅하고 알림을 보내는 노드."""
    logger.error(f"에러 발생: {state.get('error')}")
    return {"current_step": "handle_error"}


# ── 4. 라우팅 함수 ──

def route_after_fetch(state: TaskState) -> str:
    """fetch 후 라우팅: 에러 -> handle_error, 정상 -> analyze"""
    if state.get("error"):
        return "handle_error"
    return "analyze"


def route_after_analyze(state: TaskState) -> str:
    """analyze 후 라우팅: 에러 -> handle_error, 정상 -> create_ticket"""
    if state.get("error"):
        return "handle_error"
    return "create_ticket"


def route_after_ticket(state: TaskState) -> str:
    """ticket 생성 후 라우팅: 에러 -> handle_error, 정상 -> 종료"""
    if state.get("error"):
        return "handle_error"
    return END
```


176

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# ── 5. 그래프 구성 ──

graph = StateGraph(TaskState)

# 노드 등록
graph.add_node("fetch_data", fetch_data_node)
graph.add_node("analyze", analyze_node)
graph.add_node("create_ticket", create_ticket_node)
graph.add_node("handle_error", handle_error_node)

# 엣지 연결
graph.set_entry_point("fetch_data")
graph.add_conditional_edges("fetch_data", route_after_fetch)
graph.add_conditional_edges("analyze", route_after_analyze)
graph.add_conditional_edges("create_ticket", route_after_ticket)
graph.add_edge("handle_error", END)

# 컴파일
app = graph.compile()

# 실행
result = app.invoke({
    "query": "버그 리포트 분석",
    "current_step": "start",
})
```

에러 처리 흐름 요약

```
정상 흐름:
  fetch_data ──> analyze ──> create_ticket ──> END

fetch 실패 (Circuit Breaker):
  fetch_data ──> handle_error ──> END
       │
       └─ CircuitOpenError 또는 HTTPError
          -> state["error"] 설정
          -> route_after_fetch가 "handle_error"로 라우팅

LLM 실패 (Retry 데코레이터):
  fetch_data ──> analyze ──> handle_error ──> END
                    │
                    ├─ Retriable: 2초 -> 4초 -> 8초 재시도
                    └─ 3회 모두 실패 or Non-retriable
                       -> state["error"] 설정
                       -> route_after_analyze가 "handle_error"로 라우팅

티켓 생성 실패 (Circuit Breaker):
  fetch_data ──> analyze ──> create_ticket ──> handle_error ──> END
```


177

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

4. 연습 문제

연습 1: retry_with_backoff 동작 확인

다음 코드를 실행하고, 재시도 로그를 관찰하세요.

```
import random
import logging

logging.basicConfig(level=logging.WARNING)

call_count = 0

def flaky_api():
    """70% 확률로 실패하는 API (학습용)."""
    global call_count
    call_count += 1
    if random.random() < 0.7:
        raise ConnectionError(f"서버 연결 실패 (시도 #{call_count})")
    return f"성공! (시도 #{call_count})"

# TODO: retry_with_backoff를 사용하여 flaky_api를 호출하세요
# - max_retries: 5
# - base_delay: 1.0
# - retriable_exceptions: (ConnectionError,)
```

연습 2: Circuit Breaker 상태 전이 관찰

178

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# TODO: CircuitBreaker를 생성하고 (failure_threshold=3, recovery_timeout=5),
#       연속 3번 실패시키면 OPEN 상태가 되는지 확인하세요.
#       그 후 5초 대기 후 HALF_OPEN으로 전환되는지 확인하세요.

cb = CircuitBreaker(
    name="test-circuit",
    failure_threshold=3,
    recovery_timeout=5.0,
    expected_exceptions=(ValueError,),
)

def always_fail():
    raise ValueError("항상 실패")

# 3번 실패시키기
for i in range(3):
    try:
        cb.call(always_fail)
    except ValueError:
        print(f"실패 #{i+1}, 상태: {cb.state.value}")

# OPEN 상태 확인
print(f"현재 상태: {cb.state.value}")  # -> "open"

# 5초 대기 후 HALF_OPEN 확인
import time
time.sleep(5)
print(f"5초 후 상태: {cb.state.value}")  # -> "half_open"
```

연습 3: 3-노드 에이전트에 에러 처리 추가

다음 에이전트에 retry 데코레이터와 Circuit Breaker를 추가하세요.

179

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
from typing import TypedDict
from langgraph.graph import StateGraph, END


class QuizState(TypedDict):
    topic: str
    questions: str | None
    score: str | None
    error: str | None


def generate_questions(state: QuizState) -> dict:
    """TODO: @retry_on_llm_error 데코레이터를 추가하세요."""
    questions = call_llm(f"{state['topic']}에 대한 퀴즈 3문제를 만들어주세요")
    return {"questions": questions}


def evaluate_answers(state: QuizState) -> dict:
    """TODO: @retry_on_llm_error 데코레이터를 추가하세요."""
    score = call_llm(f"다음 퀴즈를 채점하세요: {state['questions']}")
    return {"score": score}


def save_result(state: QuizState) -> dict:
    """TODO: Circuit Breaker를 적용하여 DB 저장을 보호하세요."""
    save_to_database(state["score"])
    return {}


# TODO: 라우팅 함수를 만들어서 에러 시 handle_error 노드로 이동하게 하세요
# TODO: handle_error 노드를 추가하세요
# TODO: 그래프를 완성하세요
```

---

5. 핵심 정리

한눈에 보는 에러 처리 전략

180

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
┌──────────────────────────────────────────────────────────┐
│                    에러 발생!                            │
│                        │                                 │
│            ┌───────────┴──────────┐                      │
│            │                      │                      │
│     Retriable?              Non-retriable?               │
│       (429, 5xx,              (401, 404,                 │
│        timeout)               파싱 실패)                 │
│            │                      │                      │
│            ▼                      ▼                      │
│     Retry with              state["error"]               │
│     Backoff + Jitter        설정 후 failure              │
│            │                경로로 이동                  │
│            │                                             │
│     연속 실패 많으면?                                    │
│            │                                             │
│            ▼                                             │
│     Circuit Breaker                                      │
│     OPEN (즉시 실패)                                     │
│            │                                             │
│     시간 경과 후                                         │
│            │                                             │
│            ▼                                             │
│     HALF_OPEN                                            │
│     (테스트 요청)                                        │
└──────────────────────────────────────────────────────────┘
```

패턴별 요약

패턴	사용 시점	핵심 키워드
Exponential Backoff	일시적 에러 재시도	대기 시간 2배씩 증가
Jitter	다수 클라이언트의 동시 재시도 방지	랜덤 지연
RetryPolicy	LangGraph 노드 레벨 자동 재시도	`retry=` 파라미터
retry_on_llm_error	Retriable/Non-retriable 분기	커스텀 데코레이터
Circuit Breaker	외부 시스템 장애 시 빠른 실패	CLOSED/OPEN/HALF_OPEN
Graceful Degradation	DLQ 직행 방지	`state["error"]` + 조건부 엣지

기억할 3가지 원칙

1. **모든 에러를 같은 방식으로 처리하지 마세요**: Retriable과 Non-retriable을 구분하세요
2. **실패를 숨기지 말고, 관리하세요**: `state["error"]`에 기록하고 적절한 경로로 이동

181

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

3. **빠른 실패가 느린 실패보다 낫습니다**: Circuit Breaker로 불필요한 대기를 줄이세요

---

6. 참고 자료

자료	링크	설명
LangGraph RetryPolicy	langgraph.graph.graph.CompiledGraph	LangGraph 내장 재시도 정책 API 레퍼런스
Circuit Breaker 패턴	Azure Architecture Patterns	Microsoft의 Circuit Breaker 패턴 설명
tenacity 라이브러리	tenacity.readthedocs.io	Python 표준 재시도 라이브러리
Exponential Backoff	Google Cloud - Exponential Backoff	Google의 Exponential Backoff 가이드
Resilience Patterns	Microsoft - Resiliency patterns	회복 탄력성 패턴 모음

다음 단계

이번 모듈에서 에러 처리와 회복 탄력성의 기본 패턴을 배웠습니다. 다음 모듈에서는 에이전트가 의존하는 **외부 시스템 (API, 메시지 큐, 데이터베이스)**과의 연동을 깊이 다룹니다.

다음: Module 09: 외부 시스템 연동 - API, 메시지 큐, 데이터베이스

182

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

⬆ 목차로 돌아가기

Module 09: 외부 시스템 연동

> API, 메시지 큐, 데이터베이스와 안정적으로 통신하는 에이전트

학습 목표

이 모듈을 완료하면 다음을 할 수 있습니다:

1. 에이전트가 의존하는 외부 시스템의 유형과 특성을 이해한다
2. 동기 vs 비동기 I/O의 차이를 이해하고, asyncio 기본 패턴을 사용할 수 있다
3. 커넥션 풀링을 적용하여 HTTP 클라이언트를 효율적으로 관리할 수 있다
4. Rate Limit 처리를 구현하여 외부 API 호출을 안전하게 관리할 수 있다
5. Health Check 엔드포인트를 설계하여 시스템 상태를 모니터링할 수 있다
6. tenacity 라이브러리로 표준화된 재시도를 구현할 수 있다

사전 지식

주제	수준	설명
Python 기초	필수	클래스, 데코레이터, 컨텍스트 매니저(`with`)
HTTP 기초	필수	GET/POST, 상태 코드, 헤더
Module 08	권장	에러 처리와 회복 탄력성 기본 개념
asyncio	선택	async/await 기초 (이 모듈에서 설명)

---

1. 개념 설명

1.1 에이전트의 외부 의존성

183

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

AI 에이전트는 단독으로 동작하지 않습니다. 다양한 외부 시스템과 통신하면서 작업을 수행합니다.

```
                    ┌──────────────────┐
                    │   AI 에이전트    │
                    │                  │
                    │  ┌────────────┐  │
                    │  │ LLM 호출   │  │
                    │  │ 노드       │  │
                    │  └────────────┘  │
                    └──────────────────┘
                         │  │  │  │
              ┌──────────┘  │  │  └──────────┐
              │             │  │             │
              ▼             ▼  ▼             ▼
        ┌──────────┐   ┌──────┐ ┌──────┐  ┌──────────┐
        │ 메시지 큐 │   │REST  │ │ DB   │  │ CLI 도구 │
        │(Redis,   │   │ API  │ │      │  │(Git,     │
        │ Kafka)   │   │      │ │      │  │ Docker)  │
        └──────────┘   └──────┘ └──────┘  └──────────┘
```

외부 시스템 유형별 특성

시스템 유형	예시	통신 방식	장애 특성
메시지 큐	Redis Streams, Kafka	비동기 메시지 소비/생산	연결 끊김 시 메시지 유실 가능
REST API	Jira, GitLab, Slack	HTTP 요청/응답	Rate Limit, 타임아웃
데이터베이스	PostgreSQL, SQLite	SQL 쿼리	커넥션 풀 소진, 데드락
CLI 도구	Git, Docker	subprocess 실행	프로세스 행, 디스크 풀

현실적인 문제들

에이전트가 외부 시스템과 통신할 때 흔히 발생하는 문제를 살펴봅시다:

184

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# 문제 1: 매 호출마다 새 연결 생성 (TCP 핸드셰이크 오버헤드)
def call_jira():
    with httpx.Client(timeout=30) as client:  # 매번 새 TCP 연결!
        response = client.post(url, json=payload)
    return response.json()

# 문제 2: 동기 호출로 다른 작업 차단
def process_message():
    data = redis_client.xread(...)  # 여기서 멈춤 (blocking)
    result = call_llm(data)          # 여기서도 멈춤
    save_to_db(result)               # 여기서도 멈춤
    # -> 모든 작업이 순차적으로 기다림

# 문제 3: Rate Limit 무시
def call_api_100_times():
    for i in range(100):
        response = httpx.get(api_url)  # 429 받으면 어떻게?
```

1.2 동기 vs 비동기 I/O

동기 호출의 문제: 줄 서서 기다리기

동기 코드에서는 하나의 I/O 작업이 완료될 때까지 **프로그램 전체가 대기**합니다.

```
동기 실행 (하나씩 순서대로):

시간 ──────────────────────────────────────────────>

Task 1: [API 호출 ═══════════]
                                  [완료]
Task 2:                                   [API 호출 ═══════════]
                                                                  [완료]
Task 3:                                                                   [API 호출
═══]

[완료]

총 소요: ═══════════════════════════════════════════════════════════════════════
         30초 + 30초 + 10초 = 70초
```

비동기 호출: 동시에 기다리기

비동기 코드에서는 하나의 I/O가 대기 중일 때 **다른 작업을 실행**할 수 있습니다.

185

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
비동기 실행 (동시에 기다리기):

시간 ──────────────────────────────>

Task 1: [API 호출 ═══════════]  [완료]
Task 2: [API 호출 ═══════════]  [완료]
Task 3: [API 호출 ═══]          [완료]

총 소요: ═══════════════════════
         max(30초, 30초, 10초) = 30초
```

> **비유**: 식당에서 주문하고 기다리는 것(동기)과, 주문 후 다른 볼일을 보다가 진동벨이 울리면 음식을 가져오는 것(비동기)의 차이입니다.

1.3 asyncio 기초

Python의 `asyncio`는 비동기 프로그래밍을 위한 표준 라이브러리입니다.

핵심 키워드: async와 await

```python
import asyncio

# 동기 함수 (일반 함수)
def sync_hello():
    return "안녕하세요"

# 비동기 함수 (코루틴)
async def async_hello():
    return "안녕하세요"

# async 함수는 await로 호출
async def main():
    result = await async_hello()
    print(result)

# 실행
asyncio.run(main())
```

비동기로 여러 작업 동시 실행

186

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
import asyncio
import time


async def fetch_data(name: str, delay: float) -> str:
    """delay초 동안 대기 후 결과 반환 (API 호출 시뮬레이션)."""
    print(f"[{name}] 요청 시작...")
    await asyncio.sleep(delay)  # 비동기 대기 (다른 작업 실행 가능)
    print(f"[{name}] 응답 수신!")
    return f"{name}: 완료"


async def main():
    start = time.time()

    # 3개의 비동기 작업을 동시에 실행
    results = await asyncio.gather(
        fetch_data("Jira", 2.0),
        fetch_data("GitLab", 1.5),
        fetch_data("Slack", 1.0),
    )

    elapsed = time.time() - start
    print(f"\n결과: {results}")
    print(f"소요 시간: {elapsed:.1f}초")  # 약 2.0초 (가장 긴 작업 기준)


asyncio.run(main())
```

실행 결과:

```
[Jira] 요청 시작...
[GitLab] 요청 시작...
[Slack] 요청 시작...
[Slack] 응답 수신!
[GitLab] 응답 수신!
[Jira] 응답 수신!

결과: ['Jira: 완료', 'GitLab: 완료', 'Slack: 완료']
소요 시간: 2.0초
```

> 3개 작업을 동기로 실행하면 2.0 + 1.5 + 1.0 = 4.5초이지만, 비동기로는 2.0초만 소요됩니다.

---

2. 단계별 실습

187

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

2.1 redis.asyncio로 비동기 메시지 소비

메시지 큐(Redis Streams 등)에서 메시지를 소비할 때, 비동기 방식을 사용하면 메시지 처리 중에도 다음 메시지를 미리 가져올 수 있습니다.

동기 vs 비동기 비교

188

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# === 동기 방식 (문제) ===
from redis import Redis

def sync_consume():
    client = Redis(host="localhost", port=6379)
    while True:
        # 블로킹: 메시지가 올 때까지 이 줄에서 멈춤
        response = client.xreadgroup(
            groupname="my-group",
            consumername="consumer-1",
            streams={"my-stream": ">"},
            count=1,
            block=1000,  # 최대 1초 대기
        )
        if response:
            process(response)  # 처리 중에는 다른 메시지 소비 불가


# === 비동기 방식 (개선) ===
from redis.asyncio import Redis as AsyncRedis, ConnectionPool


async def async_consume():
    # ConnectionPool: 커넥션 재사용 + 자동 재연결
    pool = ConnectionPool(
        host="localhost",
        port=6379,
        decode_responses=True,
        max_connections=10,        # 최대 커넥션 수
        retry_on_timeout=True,     # 타임아웃 시 자동 재연결
        health_check_interval=30,  # 30초마다 PING으로 상태 확인
    )
    client = AsyncRedis(connection_pool=pool)

    while True:
        try:
            # 비동기 대기: 대기 중 다른 작업 실행 가능
            response = await client.xreadgroup(
                groupname="my-group",
                consumername="consumer-1",
                streams={"my-stream": ">"},
                count=1,
                block=1000,
            )
            if response:
                await process(response)
        except Exception as err:
            logger.error(f"메시지 소비 오류: {err}")
            await asyncio.sleep(3)  # 3초 대기 후 재시도
```

2.2 커넥션 풀링

189

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

매번 새 연결을 만드는 문제

HTTP 요청을 할 때마다 새 연결을 만들면: 1. TCP 3-way handshake (약 10~50ms) 2. TLS handshake (약 50~100ms, HTTPS의 경우) 3. 요청 전송 & 응답 수신 4. 연결 종료

```
매 호출마다 새 연결:

호출 1: [TCP 연결]──[TLS]──[요청/응답]──[연결 종료]
호출 2: [TCP 연결]──[TLS]──[요청/응답]──[연결 종료]  ← 같은 서버인데 또!
호출 3: [TCP 연결]──[TLS]──[요청/응답]──[연결 종료]  ← 또!
         ~~~~~~~~~~  ~~~~~
         매번 50ms+  매번 100ms+ = 오버헤드
```

```
커넥션 풀링 (연결 재사용):

초기:   [TCP 연결]──[TLS]──[연결 풀에 저장]
호출 1: [풀에서 가져오기]──[요청/응답]──[풀에 반환]
호출 2: [풀에서 가져오기]──[요청/응답]──[풀에 반환]  ← 기존 연결 재사용!
호출 3: [풀에서 가져오기]──[요청/응답]──[풀에 반환]  ← 재사용!
         ~~~~~~~~~~~~~~~~
         거의 0ms
```

httpx.AsyncClient 재사용 패턴

190

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
import httpx
import logging

logger = logging.getLogger(__name__)


class ManagedHttpClient:
    """라이프사이클이 관리되는 HTTP 클라이언트.

    - 커넥션 풀링으로 TCP/TLS 핸드셰이크 재사용
    - startup/shutdown으로 명시적 생명주기 관리
    """

    def __init__(
        self,
        timeout: float = 30.0,
        max_connections: int = 20,
        max_keepalive: int = 10,
    ):
        self._timeout = timeout
        self._max_connections = max_connections
        self._max_keepalive = max_keepalive
        self._client: httpx.AsyncClient | None = None

    async def startup(self) -> None:
        """클라이언트를 초기화합니다. 에이전트 시작 시 1번 호출."""
        limits = httpx.Limits(
            max_connections=self._max_connections,
            max_keepalive_connections=self._max_keepalive,
        )
        self._client = httpx.AsyncClient(
            limits=limits,
            timeout=httpx.Timeout(self._timeout),
        )
        logger.info(f"HTTP 클라이언트 초기화 (최대 {self._max_connections} 커넥션)")

    async def shutdown(self) -> None:
        """클라이언트를 종료합니다. 에이전트 종료 시 1번 호출."""
        if self._client and not self._client.is_closed:
            await self._client.aclose()
            logger.info("HTTP 클라이언트 종료")

    @property
    def client(self) -> httpx.AsyncClient:
        """HTTP 클라이언트를 반환합니다."""
        if self._client is None or self._client.is_closed:
            raise RuntimeError("HTTP 클라이언트가 초기화되지 않았습니다. startup()을 먼저
        호출하세요.")
        return self._client


# 사용 예시
http = ManagedHttpClient(timeout=30.0, max_connections=20)
```


191

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
async def main():
    await http.startup()

    try:
        # 같은 클라이언트를 재사용 (커넥션 풀링)
        resp1 = await http.client.get("https://api.example.com/data/1")
        resp2 = await http.client.get("https://api.example.com/data/2")
        resp3 = await http.client.post("https://api.example.com/data", json={...})
    finally:
        await http.shutdown()
```

컨텍스트 매니저 패턴

`async with`를 사용하면 자동으로 리소스를 정리할 수 있습니다.

```
from contextlib import asynccontextmanager


@asynccontextmanager
async def managed_client(timeout: float = 30.0, max_connections: int = 20):
    """HTTP 클라이언트를 컨텍스트 매니저로 관리합니다."""
    limits = httpx.Limits(max_connections=max_connections)
    client = httpx.AsyncClient(limits=limits, timeout=httpx.Timeout(timeout))
    try:
        yield client
    finally:
        await client.aclose()


# 사용
async def fetch_data():
    async with managed_client() as client:
        response = await client.get("https://api.example.com/data")
        return response.json()
    # async with 블록을 벗어나면 자동으로 client.aclose() 호출
```

2.3 Rate Limit 핸들링

HTTP 429 상태코드의 의미

외부 API는 과도한 요청을 방지하기 위해 **Rate Limit**을 설정합니다. 한도를 초과하면 HTTP 429 (Too Many Requests) 응답을 반환합니다.

192

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
클라이언트                    API 서버
    |                            |
    |── 요청 1 ──────────────>   |  200 OK
    |── 요청 2 ──────────────>   |  200 OK
    |── 요청 3 ──────────────>   |  200 OK
    |── 요청 4 ──────────────>   |  429 Too Many Requests
    |                            |  Retry-After: 30
    |                            |
    |   (30초 대기)              |
    |                            |
    |── 요청 5 ──────────────>   |  200 OK
```

Retry-After 헤더 존중

429 응답에는 보통 `Retry-After` 헤더가 포함됩니다. 이 헤더는 "이 시간 후에 다시 시도하세요"라는 의미입니다.

193

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
import asyncio
import httpx
import logging

logger = logging.getLogger(__name__)


async def call_api_with_rate_limit(
    client: httpx.AsyncClient,
    method: str,
    url: str,
    max_retries: int = 3,
    **kwargs,
) -> httpx.Response:
    """Rate Limit을 처리하며 API를 호출합니다.

    429 응답 시 Retry-After 헤더를 존중하여 대기 후 재시도합니다.

    Args:
        client: httpx.AsyncClient 인스턴스
        method: HTTP 메서드 ("GET", "POST" 등)
        url: 요청 URL
        max_retries: 최대 재시도 횟수
        **kwargs: httpx 요청 추가 인자 (json, headers 등)

    Returns:
        httpx.Response

    Raises:
        httpx.HTTPStatusError: 재시도 실패 시
    """
    for attempt in range(max_retries + 1):
        response = await client.request(method, url, **kwargs)

        if response.status_code == 429:
            # Retry-After 헤더에서 대기 시간 추출
            retry_after = response.headers.get("Retry-After")

            if retry_after:
                wait_seconds = float(retry_after)
            else:
                # 헤더가 없으면 Exponential Backoff
                wait_seconds = 2 ** attempt

            logger.warning(
                f"Rate Limit 초과 (429). {wait_seconds}초 후 재시도 "
                f"(시도 {attempt + 1}/{max_retries + 1})"
            )

            if attempt < max_retries:
                await asyncio.sleep(wait_seconds)
                continue

        # 429가 아니면 일반 응답 처리
```


194

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
        response.raise_for_status()
        return response

    # 모든 재시도 실패
    response.raise_for_status()
    return response  # 여기에 도달하지 않음
```

토큰 버킷 알고리즘 개념

API 호출 **전에** Rate Limit을 예방하는 방법입니다.

```
토큰 버킷 비유:
- 양동이(버킷)에 토큰이 있음
- API 호출 시 토큰 1개를 소비
- 일정 속도로 토큰이 보충됨
- 토큰이 없으면 대기

┌─────────┐
│ ● ● ● ● │ ← 버킷 (최대 10개)
│ ● ● ● ● │
│ ● ●     │ ← 현재 6개
└─────────┘
     │
     │ 1초마다 2개씩 보충
     ▼
  API 호출 → 토큰 1개 소비
```


195

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
import asyncio
import time


class TokenBucket:
    """토큰 버킷 기반 Rate Limiter.

    초당 허용 요청 수를 제한합니다.

    Args:
        rate: 초당 토큰 보충 속도 (예: 10 = 초당 10개)
        capacity: 버킷 최대 용량
    """

    def __init__(self, rate: float, capacity: int):
        self.rate = rate
        self.capacity = capacity
        self.tokens = capacity
        self.last_refill = time.monotonic()
        self._lock = asyncio.Lock()

    async def acquire(self) -> None:
        """토큰을 1개 획득합니다. 없으면 대기합니다."""
        async with self._lock:
            await self._refill()

            while self.tokens < 1:
                # 토큰이 보충될 때까지 대기
                wait_time = (1 - self.tokens) / self.rate
                await asyncio.sleep(wait_time)
                await self._refill()

            self.tokens -= 1

    async def _refill(self) -> None:
        """경과 시간에 따라 토큰을 보충합니다."""
        now = time.monotonic()
        elapsed = now - self.last_refill
        self.tokens = min(
            self.capacity,
            self.tokens + elapsed * self.rate,
        )
        self.last_refill = now


# 사용 예시: 초당 5개 요청으로 제한
rate_limiter = TokenBucket(rate=5, capacity=10)


async def safe_api_call(client: httpx.AsyncClient, url: str):
    """Rate Limit을 지키며 API를 호출합니다."""
    await rate_limiter.acquire()  # 토큰 대기
    return await client.get(url)  # 토큰 획득 후 호출
```


196

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

2.4 Health Check 엔드포인트

에이전트가 시작할 때, 의존하는 외부 시스템이 정상인지 확인해야 합니다. 또한 운영 중에도 `/health` 엔드포인트로 시스템 상태를 모니터링할 수 있어야 합니다.

197

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
import asyncio
import httpx
import logging
from dataclasses import dataclass
from enum import Enum

logger = logging.getLogger(__name__)


class HealthStatus(Enum):
    HEALTHY = "healthy"
    DEGRADED = "degraded"  # 일부 시스템만 장애
    UNHEALTHY = "unhealthy"


@dataclass
class SystemHealth:
    """개별 시스템의 상태."""
    name: str
    status: HealthStatus
    latency_ms: float | None = None
    error: str | None = None


async def check_redis_health(host: str, port: int) -> SystemHealth:
    """Redis/Valkey 연결 상태를 확인합니다."""
    try:
        from redis.asyncio import Redis
        client = Redis(host=host, port=port)
        start = asyncio.get_event_loop().time()
        await client.ping()
        latency = (asyncio.get_event_loop().time() - start) * 1000
        await client.aclose()
        return SystemHealth("redis", HealthStatus.HEALTHY, latency_ms=latency)
    except Exception as exc:
        return SystemHealth("redis", HealthStatus.UNHEALTHY, error=str(exc))


async def check_api_health(url: str, name: str) -> SystemHealth:
    """REST API의 상태를 확인합니다."""
    try:
        async with httpx.AsyncClient(timeout=httpx.Timeout(5.0)) as client:
            start = asyncio.get_event_loop().time()
            response = await client.get(url)
            latency = (asyncio.get_event_loop().time() - start) * 1000
            if response.status_code < 500:
                return SystemHealth(name, HealthStatus.HEALTHY, latency_ms=latency)
            return SystemHealth(name, HealthStatus.UNHEALTHY, error=f"HTTP
        {response.status_code}")
    except Exception as exc:
        return SystemHealth(name, HealthStatus.UNHEALTHY, error=str(exc))


async def check_db_health(db_url: str) -> SystemHealth:
```


198

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
    """데이터베이스 연결 상태를 확인합니다."""
    try:
        # SQLAlchemy 비동기 엔진 사용 시
        from sqlalchemy.ext.asyncio import create_async_engine
        engine = create_async_engine(db_url)
        async with engine.connect() as conn:
            start = asyncio.get_event_loop().time()
            await conn.execute("SELECT 1")
            latency = (asyncio.get_event_loop().time() - start) * 1000
        await engine.dispose()
        return SystemHealth("database", HealthStatus.HEALTHY, latency_ms=latency)
    except Exception as exc:
        return SystemHealth("database", HealthStatus.UNHEALTHY, error=str(exc))


async def run_health_check() -> dict:
    """모든 시스템의 상태를 확인합니다."""
    checks = await asyncio.gather(
        check_redis_health("localhost", 6379),
        check_api_health("https://your-jira.atlassian.net/status", "jira"),
        check_api_health("https://gitlab.example.com/api/v4/version", "gitlab"),
        return_exceptions=True,
    )

    results = []
    for check in checks:
        if isinstance(check, Exception):
            results.append(SystemHealth("unknown", HealthStatus.UNHEALTHY,
        error=str(check)))
        else:
            results.append(check)

    # 전체 상태 판단
    unhealthy = [r for r in results if r.status == HealthStatus.UNHEALTHY]

    if len(unhealthy) == 0:
        overall = HealthStatus.HEALTHY
    elif len(unhealthy) < len(results):
        overall = HealthStatus.DEGRADED
    else:
        overall = HealthStatus.UNHEALTHY

    return {
        "status": overall.value,
        "systems": [
            {
                "name": r.name,
                "status": r.status.value,
                "latency_ms": r.latency_ms,
                "error": r.error,
            }
            for r in results
        ],
    }
```


199

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

FastAPI Health Check 엔드포인트

```
from fastapi import FastAPI, Response

app = FastAPI()


@app.get("/health")
async def health_check():
    """시스템 상태를 확인하는 엔드포인트."""
    result = await run_health_check()

    # 상태에 따라 HTTP 코드 결정
    if result["status"] == "unhealthy":
        return Response(
            content=json.dumps(result),
            status_code=503,  # Service Unavailable
            media_type="application/json",
        )

    return result
```

응답 예시:

```
{
  "status": "degraded",
  "systems": [
    {"name": "redis", "status": "healthy", "latency_ms": 1.2, "error": null},
    {"name": "jira", "status": "unhealthy", "latency_ms": null, "error": "Connection
        timeout"},
    {"name": "gitlab", "status": "healthy", "latency_ms": 45.3, "error": null}
  ]
}
```

2.5 tenacity 라이브러리로 표준화된 재시도

`tenacity` 는 Python 재시도 로직의 표준 라이브러리입니다. Module 08에서 직접 구현한 재시도를 더 간결하게 작성할 수 있습니다.

설치

```
pip install tenacity
```

기본 사용법

200

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
from tenacity import (
    retry,
    stop_after_attempt,
    wait_exponential,
    retry_if_exception_type,
    before_sleep_log,
)
import logging
import httpx

logger = logging.getLogger(__name__)


# === 기본: 3번 재시도, exponential backoff ===
@retry(
    stop=stop_after_attempt(3),
    wait=wait_exponential(multiplier=2, min=1, max=30),
)
def simple_api_call():
    response = httpx.get("https://api.example.com/data")
    response.raise_for_status()
    return response.json()


# === 특정 예외만 재시도 ===
@retry(
    retry=retry_if_exception_type((httpx.TimeoutException, httpx.ConnectError)),
    stop=stop_after_attempt(3),
    wait=wait_exponential(multiplier=2, min=1, max=30),
)
def network_call():
    response = httpx.get("https://api.example.com/data", timeout=10)
    response.raise_for_status()
    return response.json()


# === 재시도 전 로깅 ===
@retry(
    stop=stop_after_attempt(3),
    wait=wait_exponential(multiplier=2, min=1, max=30),
    before_sleep=before_sleep_log(logger, logging.WARNING),
    reraise=True,  # 최종 실패 시 원래 예외를 다시 발생
)
def logged_api_call():
    response = httpx.get("https://api.example.com/data")
    response.raise_for_status()
    return response.json()
```

HTTP 상태 코드별 재시도 필터링

201

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
from tenacity import retry_if_exception_type

# 재시도 가능한 HTTP 상태 코드
RETRIABLE_STATUS_CODES = {408, 429, 500, 502, 503, 504}
# 재시도 불가한 상태 코드
NON_RETRIABLE_STATUS_CODES = {400, 401, 403, 404, 405, 422}


def is_retriable_http_error(exc: BaseException) -> bool:
    """HTTP 에러가 재시도 가능한지 판별합니다."""
    if isinstance(exc, httpx.TimeoutException):
        return True  # 타임아웃은 항상 재시도
    if isinstance(exc, httpx.ConnectError):
        return True  # 연결 오류는 항상 재시도
    if isinstance(exc, httpx.HTTPStatusError):
        return exc.response.status_code in RETRIABLE_STATUS_CODES
    return False


class RetriableHTTPError(retry_if_exception_type):
    """재시도 가능한 HTTP 에러만 필터링하는 tenacity 조건."""

    def __init__(self):
        super().__init__((httpx.HTTPError, httpx.TimeoutException))

    def __call__(self, retry_state) -> bool:
        if not super().__call__(retry_state):
            return False
        exc = retry_state.outcome.exception()
        return is_retriable_http_error(exc)


# 사용: 429, 5xx만 재시도하고, 401/404는 즉시 실패
@retry(
    retry=RetriableHTTPError(),
    stop=stop_after_attempt(3),
    wait=wait_exponential(multiplier=2, min=1, max=30),
    reraise=True,
)
def smart_api_call(url: str, payload: dict) -> dict:
    """상태 코드에 따라 선택적으로 재시도하는 API 호출."""
    response = httpx.post(url, json=payload, timeout=30)
    response.raise_for_status()
    return response.json()
```

tenacity 주요 옵션 요약

202

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

옵션	설명	예시
`stop_after_attempt(n)`	n번 시도 후 중단	`stop=stop_after_attempt(3)`
`stop_after_delay(n)`	n초 후 중단	`stop=stop_after_delay(60)`
`wait_exponential(...)`	Exponential Backoff	`wait=wait_exponential(min=1, max=30)`
`wait_fixed(n)`	고정 대기 시간	`wait=wait_fixed(2)`
`retry_if_exception_type(...)`	특정 예외만 재시도	`retry=retry_if_exception_type(TimeoutError)`
`before_sleep_log(...)`	재시도 전 로깅	`before_sleep=before_sleep_log(logger, WARNING)`
`reraise=True`	최종 실패 시 원래 예외 발생	기본값은 `RetryError` 발생

---

3. 실전 예제

httpx.AsyncClient + Rate Limit + Health Check 통합 에이전트

203

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
import asyncio
import httpx
import logging
from typing import TypedDict, Annotated
from langgraph.graph import StateGraph, END

logger = logging.getLogger(__name__)


# ── State 정의 ──

def _replace(existing, new):
    return new


class AgentState(TypedDict):
    task_id: Annotated[str, _replace]
    data: Annotated[dict | None, _replace]
    analysis: Annotated[str | None, _replace]
    error: Annotated[str | None, _replace]
    current_step: Annotated[str, _replace]


# ── 공유 리소스 ──

rate_limiter = TokenBucket(rate=5, capacity=10)  # 초당 5요청
http_client: httpx.AsyncClient | None = None


async def init_resources():
    """에이전트 시작 시 공유 리소스를 초기화합니다."""
    global http_client
    http_client = httpx.AsyncClient(
        limits=httpx.Limits(max_connections=20),
        timeout=httpx.Timeout(30.0),
    )
    # Health Check
    health = await run_health_check()
    if health["status"] == "unhealthy":
        raise RuntimeError(f"외부 시스템 장애: {health}")
    logger.info(f"시스템 상태: {health['status']}")


async def cleanup_resources():
    """에이전트 종료 시 리소스를 정리합니다."""
    global http_client
    if http_client:
        await http_client.aclose()


# ── 노드 정의 ──

async def fetch_task_data(state: AgentState) -> dict:
    """외부 API에서 작업 데이터를 가져옵니다."""
```


204

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
    try:
        await rate_limiter.acquire()
        response = await call_api_with_rate_limit(
            http_client, "GET",
            f"https://api.example.com/tasks/{state['task_id']}",
        )
        return {"data": response.json(), "current_step": "fetch"}
    except Exception as exc:
        return {"error": f"데이터 조회 실패: {exc}", "current_step": "fetch"}


async def analyze_data(state: AgentState) -> dict:
    """LLM으로 데이터를 분석합니다. (재시도 포함)"""
    if not state.get("data"):
        return {"error": "분석할 데이터 없음", "current_step": "analyze"}

    from tenacity import retry, stop_after_attempt, wait_exponential

    @retry(stop=stop_after_attempt(3), wait=wait_exponential(min=2, max=30))
    async def _call_llm(data):
        # LLM 호출 (예시)
        return f"분석 결과: {data.get('title', '제목 없음')}"

    try:
        result = await _call_llm(state["data"])
        return {"analysis": result, "current_step": "analyze"}
    except Exception as exc:
        return {"error": f"분석 실패: {exc}", "current_step": "analyze"}


async def save_result(state: AgentState) -> dict:
    """결과를 외부 시스템에 저장합니다."""
    try:
        await rate_limiter.acquire()
        response = await call_api_with_rate_limit(
            http_client, "POST",
            "https://api.example.com/results",
            json={"task_id": state["task_id"], "analysis": state["analysis"]},
        )
        return {"current_step": "save"}
    except Exception as exc:
        return {"error": f"저장 실패: {exc}", "current_step": "save"}


# ── 라우팅 ──

def route_on_error(state: AgentState) -> str:
    if state.get("error"):
        return "handle_error"
    return "next"


# ── 그래프 구성 ──

graph = StateGraph(AgentState)
```


205

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
graph.add_node("fetch", fetch_task_data)
graph.add_node("analyze", analyze_data)
graph.add_node("save", save_result)
graph.add_node("handle_error", lambda state: {"current_step": "error"})

graph.set_entry_point("fetch")
graph.add_conditional_edges("fetch", route_on_error, {"handle_error": "handle_error",
         "next": "analyze"})
graph.add_conditional_edges("analyze", route_on_error, {"handle_error": "handle_error",
         "next": "save"})
graph.add_edge("save", END)
graph.add_edge("handle_error", END)

app = graph.compile()
```

---

4. 연습 문제

연습 1: 비동기 API 호출

다음 동기 코드를 비동기로 변환하세요.

```
# 현재: 동기 (3개 호출이 순차 실행 = 약 9초)
import httpx

def get_all_data():
    with httpx.Client() as client:
        users = client.get("https://api.example.com/users").json()      # 3초
        projects = client.get("https://api.example.com/projects").json() # 3초
        tickets = client.get("https://api.example.com/tickets").json()   # 3초
    return users, projects, tickets

# TODO: asyncio.gather를 사용하여 3개 호출을 동시에 실행하세요 (약 3초)
# async def get_all_data_async():
#     ...
```

연습 2: Rate Limiter 적용

206

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# TODO: TokenBucket을 사용하여 초당 3개 요청으로 제한하면서
#       10개의 API 호출을 하세요.

rate_limiter = TokenBucket(rate=3, capacity=5)

async def batch_api_calls():
    results = []
    for i in range(10):
        # TODO: rate_limiter.acquire()를 호출한 후 API 호출
        pass
    return results
```

연습 3: Health Check가 있는 에이전트

```python
# TODO: 에이전트 시작 시 Health Check를 수행하고,
#       모든 시스템이 healthy일 때만 작업을 시작하세요.
#       하나라도 unhealthy이면 30초 후 재확인하세요.

async def start_agent_with_health_check():
    for attempt in range(5):
        health = await run_health_check()
        if health["status"] == "healthy":
            print("모든 시스템 정상! 에이전트를 시작합니다.")
            # TODO: 에이전트 시작
            break
        else:
            print(f"시스템 장애 감지. {30}초 후 재확인... ({attempt + 1}/5)")
            # TODO: 대기 후 재확인
    else:
        print("시스템 복구 실패. 에이전트를 시작할 수 없습니다.")
```

---

5. 핵심 정리

한눈에 보는 외부 시스템 연동 패턴

207

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

패턴	문제	해결	도구
비동기 I/O	동기 호출로 전체 프로그램 차단	async/await로 동시 실행	`asyncio`, `redis.asyncio`
커넥션 풀링	매번 새 TCP/TLS 연결 생성	연결을 재사용	`httpx.AsyncClient`, `Limits`
Rate Limit	API 호출 한도 초과 (429)	Retry-After 존중, TokenBucket	`httpx`, 직접 구현
Health Check	장애 시스템에 요청 시도	시작 시 및 주기적 상태 확인	FastAPI `/health` 엔드포인트
tenacity	재시도 로직 중복 구현	표준화된 데코레이터	`@retry` 데코레이터

기억할 3가지 원칙

1. **연결은 재사용하세요**: 매번 새로 만들지 말고, 커넥션 풀을 사용하세요
2. **외부 시스템의 한도를 존중하세요**: Rate Limit을 초과하면 모두에게 피해가 갑니다
3. **시작 전에 확인하세요**: Health Check로 의존 시스템의 상태를 먼저 점검하세요

---

6. 참고 자료

자료	링크	설명
httpx 공식 문서	python-httpx.org	Python 비동기 HTTP 클라이언트
redis-py asyncio	redis.readthedocs.io/asyncio	Redis 비동기 사용 가이드
tenacity	tenacity.readthedocs.io	Python 재시도 라이브러리
asyncio 공식 문서	docs.python.org/asyncio	Python 비동기 프로그래밍 표준 라이브러리
httpx Limits 설정	python-httpx.org/advanced/pool	커넥션 풀 설정 가이드
Token Bucket Algorithm	Wikipedia	토큰 버킷 알고리즘 설명

다음 단계

208

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

이번 모듈에서 외부 시스템과 안정적으로 통신하는 방법을 배웠습니다. 다음 모듈에서는 에이전트가 사용하는 **리소스 (메모리, 디스크, LLM 토큰)**를 효율적으로 관리하는 방법을 다룹니다.

다음: Module 10: 리소스 최적화 - 메모리, 디스크, 토큰 관리

209

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

⬆ 목차로 돌아가기

Module 10: 리소스 최적화

> 메모리, 디스크, 토큰을 효율적으로 관리하는 에이전트

학습 목표

이 모듈을 완료하면 다음을 할 수 있습니다:

1. 에이전트의 3대 리소스(디스크, 메모리, LLM 토큰) 문제를 식별할 수 있다
2. TTL 기반 디스크 관리와 Shallow Clone으로 디스크 사용량을 최적화할 수 있다
3. LangGraph Annotated Reducer 패턴으로 State 복사를 최소화할 수 있다
4. tiktoken으로 정확한 토큰 수를 계산하고, TokenBudget으로 토큰 예산을 관리할 수 있다
5. AST 기반 코드 압축으로 LLM 컨텍스트 윈도우를 효율적으로 활용할 수 있다
6. 노드 간 데이터 프루닝과 청크 분할로 대용량 데이터를 처리할 수 있다

사전 지식

주제	수준	설명
Python 기초	필수	dict, list, 클래스, 타입 힌트
LangGraph 기초	필수	StateGraph, TypedDict, 노드/엣지
Module 08~09	권장	에러 처리, 외부 시스템 연동
Git 기초	선택	clone, fetch 개념 (디스크 관리 부분)

---

1. 개념 설명

1.1 에이전트의 리소스 사용 문제

210

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

AI 에이전트는 실행할 때마다 상당한 리소스를 소비합니다. 한두 번 실행할 때는 문제가 없지만, **수십~수백 건을 연속 처리하면** 리소스 문제가 발생합니다.

3대 리소스 문제

```
┌──────────────────────────────────────────────────────────┐
│              에이전트 리소스 사용                          │
│                                                            │
│  1. 디스크 문제                                            │
│     ┌──────┐ ┌──────┐ ┌──────┐ ┌──────┐                  │
│     │clone1│ │clone2│ │clone3│ │clone4│ ...               │
│     │500MB │ │1.2GB │ │800MB │ │1.5GB │                   │
│     └──────┘ └──────┘ └──────┘ └──────┘                  │
│     → 정리하지 않으면 디스크가 가득 참                      │
│                                                            │
│  2. 메모리 문제                                            │
│     Node1: {**state, "result": ...}  ← 전체 복사          │
│     Node2: {**state, "data": ...}    ← 또 전체 복사        │
│     Node3: {**state, "fix": ...}     ← 또 전체 복사        │
│     → 노드가 많을수록 메모리 사용 증가                      │
│                                                            │
│  3. LLM 토큰 문제                                          │
│     프롬프트: "다음 코드를 분석하세요:                      │
│     [30KB의 코드 전체]"                                    │
│     → 불필요한 코드까지 포함해서 토큰 낭비                  │
│       (비용 증가 + 컨텍스트 윈도우 낭비)                    │
└──────────────────────────────────────────────────────────┘
```

리소스	현실적인 문제	결과
디스크	Git clone이 계속 쌓임, 임시 파일 미삭제	디스크 풀 -> 에이전트 정지
메모리	매 노드마다 State를 통째로 복사	메모리 2~3배 사용
LLM 토큰	30KB 코드를 그대로 프롬프트에 넣기	비용 증가 + 핵심 정보 희석

1.2 문자 수와 토큰 수는 다릅니다

LLM의 과금과 컨텍스트 윈도우는 **토큰** 단위입니다. 하지만 많은 개발자가 **문자 수**로 계산합니다. 이 둘은 같지 않습니다.

```
영어:  "Hello world"  = 11 문자 = 2 토큰
한글:  "안녕하세요"    =  5 문자 = 5~7 토큰 (한글 1자 ≈ 1~2 토큰)
코드:  "def hello():" = 13 문자 = 4 토큰

→ 문자 수 기반 절삭은 실제 토큰 예산을 정확히 제어할 수 없음!
```


211

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

절삭 방식	방법	문제점
문자 기반	`text[:3000]`	토큰 수를 예측할 수 없음
토큰 기반	tiktoken으로 계산 후 절삭	정확하지만 의미를 고려하지 않음
시맨틱 기반	AST로 중요한 부분만 추출	가장 효율적, 구현 복잡

---

2. 단계별 실습

2.1 디스크 관리: TTL 기반 자동 정리

Git clone은 프로젝트에 따라 수백 MB에서 수 GB까지 차지합니다. 에이전트가 작업을 끝낸 후에도 clone이 남아 있으면 디스크가 점점 차게 됩니다.

개념: TTL(Time To Live) 기반 정리

```
TTL = 7일 (기본값)

Day 1: project-A clone (500MB) → 사용 시간 기록
Day 3: project-B clone (1GB)   → 사용 시간 기록
Day 5: project-A 다시 사용     → 사용 시간 갱신
Day 8: project-A → 아직 OK (Day 5에 사용했으므로 TTL 미초과)
        project-B → TTL 초과! (Day 3 이후 미사용) → 삭제
```

구현: CloneManager

212

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
import json
import shutil
import time
import logging
from pathlib import Path
from dataclasses import dataclass, field

logger = logging.getLogger(__name__)


@dataclass
class CloneMetadata:
    """clone의 메타데이터. 각 clone 디렉토리에 저장됩니다."""
    project_key: str
    last_used: float = field(default_factory=time.time)
    clone_size_bytes: int = 0
    clone_type: str = "full"  # "full" 또는 "shallow"

    def to_dict(self) -> dict:
        return {
            "project_key": self.project_key,
            "last_used": self.last_used,
            "clone_size_bytes": self.clone_size_bytes,
            "clone_type": self.clone_type,
        }

    @classmethod
    def from_dict(cls, data: dict) -> "CloneMetadata":
        return cls(**data)


class CloneManager:
    """Git clone의 생명주기를 관리합니다.

    기능:
      - TTL 기반 자동 정리 (기본 7일 미사용 시 삭제)
      - 최대 디스크 사용량 제한 (기본 10GB)
      - 사용 시간 추적

    사용법:
        manager = CloneManager("/data/repos", ttl_seconds=7*24*3600)
        manager.touch("my-project")       # 사용 시간 갱신
        manager.cleanup_expired()         # TTL 지난 clone 삭제
        manager.enforce_disk_limit()      # 디스크 한도 초과 시 정리
    """

    DEFAULT_TTL_SECONDS = 7 * 24 * 3600       # 7일
    DEFAULT_MAX_DISK_BYTES = 10 * 1024**3      # 10GB
    META_FILENAME = ".clone_meta.json"

    def __init__(
        self,
        base_dir: str,
        ttl_seconds: int = DEFAULT_TTL_SECONDS,
```


213

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
        max_disk_bytes: int = DEFAULT_MAX_DISK_BYTES,
    ):
        self.base_dir = Path(base_dir)
        self.ttl_seconds = ttl_seconds
        self.max_disk_bytes = max_disk_bytes

    def touch(self, project_key: str) -> None:
        """clone 사용 시간을 갱신합니다. 작업 시작 시 호출하세요."""
        meta_path = self.base_dir / project_key / self.META_FILENAME
        meta = self._load_meta(meta_path, project_key)
        meta.last_used = time.time()
        meta.clone_size_bytes = self._get_dir_size(self.base_dir / project_key)
        self._save_meta(meta_path, meta)

    def cleanup_expired(self) -> list[str]:
        """TTL이 지난 clone을 삭제합니다.

        Returns:
            삭제된 project_key 목록
        """
        removed = []
        now = time.time()

        if not self.base_dir.exists():
            return removed

        for repo_dir in self.base_dir.iterdir():
            if not repo_dir.is_dir():
                continue

            meta_path = repo_dir / self.META_FILENAME
            meta = self._load_meta(meta_path, repo_dir.name)

            age_hours = (now - meta.last_used) / 3600
            if now - meta.last_used > self.ttl_seconds:
                logger.info(
                    f"TTL 초과 clone 삭제: {repo_dir.name} "
                    f"({age_hours:.0f}시간 미사용)"
                )
                shutil.rmtree(repo_dir, ignore_errors=True)
                removed.append(repo_dir.name)

        return removed

    def enforce_disk_limit(self) -> list[str]:
        """디스크 제한 초과 시 가장 오래된 clone부터 삭제합니다.

        Returns:
            삭제된 project_key 목록
        """
        removed = []

        if not self.base_dir.exists():
            return removed
```


214

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
        # 모든 clone의 메타데이터 수집
        clones: list[tuple[CloneMetadata, Path]] = []
        for repo_dir in self.base_dir.iterdir():
            if not repo_dir.is_dir():
                continue
            meta_path = repo_dir / self.META_FILENAME
            meta = self._load_meta(meta_path, repo_dir.name)
            meta.clone_size_bytes = self._get_dir_size(repo_dir)
            clones.append((meta, repo_dir))

        total_bytes = sum(m.clone_size_bytes for m, _ in clones)

        if total_bytes <= self.max_disk_bytes:
            return removed

        # 가장 오래된 것부터 삭제 (LRU: Least Recently Used)
        clones.sort(key=lambda x: x[0].last_used)

        for meta, repo_dir in clones:
            if total_bytes <= self.max_disk_bytes:
                break
            size_mb = meta.clone_size_bytes / (1024 * 1024)
            logger.info(f"디스크 한도 초과로 삭제: {meta.project_key} ({size_mb:.0f}MB)")
            total_bytes -= meta.clone_size_bytes
            shutil.rmtree(repo_dir, ignore_errors=True)
            removed.append(meta.project_key)

        return removed

    # ── 내부 메서드 ──

    def _load_meta(self, meta_path: Path, project_key: str) -> CloneMetadata:
        if meta_path.exists():
            try:
                data = json.loads(meta_path.read_text())
                return CloneMetadata.from_dict(data)
            except (json.JSONDecodeError, TypeError):
                pass
        return CloneMetadata(project_key=project_key)

    def _save_meta(self, meta_path: Path, meta: CloneMetadata) -> None:
        meta_path.parent.mkdir(parents=True, exist_ok=True)
        meta_path.write_text(json.dumps(meta.to_dict()))

    @staticmethod
    def _get_dir_size(path: Path) -> int:
        """디렉토리 전체 크기를 바이트 단위로 계산합니다."""
        total = 0
        try:
            for f in path.rglob("*"):
                if f.is_file():
                    total += f.stat().st_size
        except OSError:
```


215

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
            pass
        return total
```

사용 예시

```
# 에이전트 초기화 시
manager = CloneManager(
    base_dir="/data/repos",
    ttl_seconds=7 * 24 * 3600,     # 7일
    max_disk_bytes=10 * 1024**3,    # 10GB
)

# 작업 시작 시
manager.touch("my-project")

# 주기적으로 정리 (매 시간 또는 에이전트 시작 시)
expired = manager.cleanup_expired()
if expired:
    print(f"TTL 만료로 삭제된 프로젝트: {expired}")

over_limit = manager.enforce_disk_limit()
if over_limit:
    print(f"디스크 한도 초과로 삭제된 프로젝트: {over_limit}")
```

2.2 Shallow Clone으로 디스크/시간 절약

대부분의 에이전트는 **최신 코드만** 필요합니다. Git의 전체 히스토리(모든 커밋, 모든 브랜치)를 가져올 필요가 없습니다.

```
Full Clone (기본):
  모든 커밋 히스토리 + 모든 브랜치
  → 500MB ~ 2GB (프로젝트에 따라)
  → 30초 ~ 120초 소요

Shallow Clone (--depth 1):
  최신 커밋 1개만
  → 50MB ~ 200MB (80~90% 절감)
  → 5초 ~ 15초 소요
```

Shallow Clone 명령어

```
# Full Clone (기본)
git clone https://github.com/user/repo.git

# Shallow Clone (최신 커밋 1개만)
git clone --depth 1 --single-branch https://github.com/user/repo.git
```

코드에서 활용

216

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
import subprocess
from pathlib import Path


def ensure_repo(
    project_key: str,
    git_url: str,
    branch: str = "main",
    base_dir: str = "/data/repos",
    shallow: bool = True,  # 기본값을 shallow로!
) -> Path:
    """리포지토리를 clone 또는 update합니다.

    Args:
        project_key: 프로젝트 식별자
        git_url: Git URL
        branch: 대상 브랜치
        base_dir: 기본 디렉토리
        shallow: True면 --depth 1로 shallow clone

    Returns:
        리포지토리 경로
    """
    repo_path = Path(base_dir) / project_key

    if (repo_path / ".git").exists():
        # 이미 있으면 fetch + reset
        fetch_args = ["git", "fetch", "origin", "--prune"]
        if shallow:
            fetch_args.extend(["--depth", "1"])
        subprocess.run(fetch_args, cwd=repo_path, check=True)
        subprocess.run(
            ["git", "reset", "--hard", f"origin/{branch}"],
            cwd=repo_path, check=True,
        )
    else:
        # 새로 clone
        repo_path.mkdir(parents=True, exist_ok=True)
        clone_args = ["git", "clone", "-b", branch]
        if shallow:
            clone_args.extend(["--depth", "1", "--single-branch"])
        clone_args.extend([git_url, str(repo_path)])
        subprocess.run(clone_args, check=True)

    return repo_path
```

Shallow Clone 효과 비교

217

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

항목	Full Clone	Shallow Clone	절감률
일반적인 프로젝트	500MB ~ 2GB	50MB ~ 200MB	80~90%
Clone 소요 시간	30 ~ 120초	5 ~ 15초	75~85%
Fetch 시간	10 ~ 30초	3 ~ 8초	60~70%

2.3 State 복사 최소화: LangGraph Annotated Reducer

문제: `{**state, "field": value}` 패턴

LangGraph 에이전트에서 가장 흔한 패턴이지만, **비효율적**입니다.

```
# 현재 패턴: 매 노드마다 전체 state를 복사
def my_node(state: MyState) -> MyState:
    result = do_something()
    return {
        **state,                    # ← 전체 state를 복사! (모든 필드)
        "result": result,           # ← 실제로 변경한 건 이 필드뿐
        "current_step": "my_node",
    }
```

왜 문제인가?

State에 대용량 필드(코드 30KB, diff 30KB 등)가 있으면, 노드를 거칠 때마다 이 데이터가 복사됩니다.

```
State 구조:
  - query: "분석해주세요" (50 bytes)
  - code_context: [30KB의 코드]      ← 대용량!
  - diff_text: [30KB의 diff]          ← 대용량!
  - result: "분석 결과" (500 bytes)

8개 노드를 거치면:
  30KB x 2필드 x 8노드 = 약 480KB의 불필요한 복사
```

해결: Annotated Reducer 패턴

LangGraph는 `Annotated` 타입과 reducer 함수를 사용하면, **노드가 반환한 필드만 State에 병합**합니다.

218

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
from typing import Annotated, TypedDict


def _replace(existing, new):
    """기존 값을 새 값으로 교체하는 reducer.

    LangGraph가 이 함수를 사용하여 state를 업데이트합니다.
    노드가 이 필드를 반환하면 -> 기존 값을 새 값으로 교체
    노드가 이 필드를 반환하지 않으면 -> 기존 값 유지 (복사 없음!)
    """
    return new


# ── Before: 일반 TypedDict ──
class OldState(TypedDict):
    query: str
    code_context: str | None     # 30KB
    diff_text: str | None        # 30KB
    result: str | None
    current_step: str
    error: str | None


# ── After: Annotated Reducer 적용 ──
class NewState(TypedDict):
    query: Annotated[str, _replace]
    code_context: Annotated[str | None, _replace]      # 30KB, 복사 안 됨!
    diff_text: Annotated[str | None, _replace]          # 30KB, 복사 안 됨!
    result: Annotated[str | None, _replace]
    current_step: Annotated[str, _replace]
    error: Annotated[str | None, _replace]
```

노드 반환값 변경

```python
# ── Before: 전체 state 복사 ──
def analyze_node(state: OldState) -> OldState:
    result = do_analysis(state["code_context"])
    return {
        **state,                          # ← 전체 복사 (code_context 30KB 포함!)
        "result": result,
        "current_step": "analyze",
    }


# ── After: 변경 필드만 반환 ──
def analyze_node(state: NewState) -> dict:
    result = do_analysis(state["code_context"])
    return {
        "result": result,                 # ← 변경한 필드만!
        "current_step": "analyze",
    }
    # code_context(30KB)와 diff_text(30KB)는 복사되지 않음!
```


219

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

리스트 필드에 항목 추가하기

`operator.add`를 reducer로 사용하면, 리스트에 항목을 **누적 추가**할 수 있습니다.

```
import operator
from typing import Annotated, TypedDict


class ChatState(TypedDict):
    messages: Annotated[list[str], operator.add]  # 리스트 누적
    current_step: Annotated[str, _replace]


# Node 1
def greet_node(state: ChatState) -> dict:
    return {"messages": ["안녕하세요!"]}  # ["안녕하세요!"]

# Node 2
def ask_node(state: ChatState) -> dict:
    return {"messages": ["무엇을 도와드릴까요?"]}
    # -> 기존 messages + 새 messages = ["안녕하세요!", "무엇을 도와드릴까요?"]

# Node 3
def respond_node(state: ChatState) -> dict:
    return {"messages": ["네, 알겠습니다."]}
    # -> ["안녕하세요!", "무엇을 도와드릴까요?", "네, 알겠습니다."]
```

기대 효과

에이전트 예시	노드 수	전체 복사	Annotated Reducer	절감
분석 에이전트 (8 노드)	8	~240KB	~30KB	87%
테스트 에이전트 (13 노드)	13	~390KB	~45KB	88%
코드 수정 에이전트 (11 노드)	11	~450KB	~50KB	89%

2.4 토큰 최적화: tiktoken

설치

```
pip install tiktoken
```

기본 사용법

220

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
import tiktoken

# 인코더 생성 (cl100k_base는 GPT-4, Claude와 호환되는 근사 인코딩)
enc = tiktoken.get_encoding("cl100k_base")

# 토큰 수 계산
text = "Hello, world!"
tokens = enc.encode(text)
print(f"텍스트: {text}")
print(f"토큰 수: {len(tokens)}")
print(f"토큰 목록: {tokens}")

# 한글 토큰 수 확인
korean_text = "안녕하세요, 세상!"
korean_tokens = enc.encode(korean_text)
print(f"\n텍스트: {korean_text}")
print(f"문자 수: {len(korean_text)}")
print(f"토큰 수: {len(korean_tokens)}")  # 문자 수보다 많음!

# 코드 토큰 수 확인
code = """def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)"""
code_tokens = enc.encode(code)
print(f"\n코드 문자 수: {len(code)}")
print(f"코드 토큰 수: {len(code_tokens)}")
```

TokenBudget 클래스 구현

토큰 예산을 관리하는 유틸리티입니다. "남은 토큰 예산 내에서 가능한 많은 데이터를 포함"하는 전략입니다.

221

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
import tiktoken


class TokenBudget:
    """토큰 예산 관리.

    LLM에 보낼 프롬프트의 토큰 수를 정확하게 관리합니다.

    참고: Claude 모델은 tiktoken의 cl100k_base와 정확히 일치하지 않지만,
    실용적인 근사치로 충분합니다 (오차 약 10%).

    사용법:
        budget = TokenBudget(max_tokens=8000)
        budget.consume(system_prompt)           # 시스템 프롬프트 소비
        budget.consume(user_query)              # 사용자 질문 소비
        code = budget.truncate_to_budget(code)  # 남은 예산에 맞춰 코드 절삭
    """

    def __init__(self, model: str = "cl100k_base", max_tokens: int = 8000):
        self._enc = tiktoken.get_encoding(model)
        self.max_tokens = max_tokens
        self._used = 0

    def count(self, text: str) -> int:
        """텍스트의 토큰 수를 반환합니다."""
        return len(self._enc.encode(text))

    @property
    def remaining(self) -> int:
        """남은 토큰 예산."""
        return max(0, self.max_tokens - self._used)

    @property
    def used(self) -> int:
        """사용한 토큰 수."""
        return self._used

    def consume(self, text: str) -> str | None:
        """예산 내에서 텍스트를 소비합니다.

        Args:
            text: 소비할 텍스트

        Returns:
            예산 내이면 원본 텍스트, 초과 시 None
        """
        tokens = self.count(text)
        if tokens <= self.remaining:
            self._used += tokens
            return text
        return None

    def truncate_to_budget(self, text: str, reserve: int = 100) -> str:
        """남은 예산에 맞게 텍스트를 토큰 단위로 절삭합니다.
```


222

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
        Args:
            text: 절삭할 텍스트
            reserve: 여유분으로 남길 토큰 수 (응답 공간 등)

        Returns:
            예산에 맞게 절삭된 텍스트
        """
        available = self.remaining - reserve
        if available <= 0:
            return ""

        tokens = self._enc.encode(text)
        if len(tokens) <= available:
            self._used += len(tokens)
            return text

        # 토큰 단위로 절삭
        truncated = self._enc.decode(tokens[:available])
        self._used += available
        return truncated + "\n... [토큰 한도로 절삭됨]"
```

TokenBudget 사용 예시

```
# 총 8000 토큰 예산으로 프롬프트 구성
budget = TokenBudget(max_tokens=8000)

# 1. 시스템 프롬프트 (약 200 토큰)
system_prompt = "당신은 코드 분석 전문가입니다. 다음 코드를 분석하세요."
budget.consume(system_prompt)
print(f"시스템 프롬프트 후 남은 토큰: {budget.remaining}")

# 2. 사용자 질문 (약 50 토큰)
user_query = "이 코드에서 버그를 찾아주세요."
budget.consume(user_query)
print(f"사용자 질문 후 남은 토큰: {budget.remaining}")

# 3. 코드 컨텍스트 (대용량 - 예산에 맞게 절삭)
large_code = open("big_file.py").read()  # 10000 토큰짜리 코드
truncated_code = budget.truncate_to_budget(large_code, reserve=500)
print(f"코드 추가 후 남은 토큰: {budget.remaining}")
print(f"코드가 절삭되었는가: {'절삭됨' if '절삭' in truncated_code else '전체 포함'}")
```

2.5 시맨틱 코드 압축: AST 기반 추출

코드를 LLM에 보낼 때, 전체 코드 대신 **구조적으로 중요한 부분만** 추출하면 토큰을 크게 절약할 수 있습니다.

압축 레벨

223

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
Level 0: 전체 코드 (토큰 예산 충분할 때)
Level 1: import 제거 + 주석 제거
Level 2: 함수/메서드 시그니처 + 어노테이션만 (본문 생략)
Level 3: 클래스/함수 선언 목록만
```

Python AST를 활용한 코드 요약

224

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
import ast
from dataclasses import dataclass


@dataclass
class CodeSummary:
    """코드 파일의 구조적 요약."""
    file_path: str
    classes: list[str]
    functions: list[str]
    full_token_count: int
    summary_token_count: int


class PythonCodeCompressor:
    """Python 코드의 구조적 압축.

    AST(Abstract Syntax Tree)를 사용하여 코드의 구조를 추출합니다.
    전체 코드 대신 클래스/함수 시그니처만 추출하면 토큰을 대폭 절약할 수 있습니다.
    """

    def extract_signatures(self, source_code: str) -> str:
        """Python 코드에서 클래스와 함수 시그니처를 추출합니다.

        Args:
            source_code: Python 소스 코드 문자열

        Returns:
            시그니처만 포함된 요약 문자열

        Example:
            입력:
                class Calculator:
                    def add(self, a: int, b: int) -> int:
                        '''두 수를 더합니다.'''
                        result = a + b
                        logger.info(f"Adding {a} + {b}")
                        return result

            출력:
                class Calculator:
                    def add(self, a: int, b: int) -> int:
                        '''두 수를 더합니다.'''
                        ...
        """
        try:
            tree = ast.parse(source_code)
        except SyntaxError:
            return source_code  # 파싱 실패 시 원본 반환

        lines = []
        for node in ast.walk(tree):
            if isinstance(node, ast.ClassDef):
                # 클래스 선언
```


225

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
                decorators = self._get_decorators(node)
                lines.extend(decorators)
                lines.append(f"class {node.name}({self._get_bases(node)}):")

            elif isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef)):
                # 함수/메서드 시그니처
                decorators = self._get_decorators(node)
                lines.extend(decorators)
                prefix = "async def" if isinstance(node, ast.AsyncFunctionDef) else
        "def"
                args = self._get_args(node)
                returns = self._get_return_annotation(node)
                sig = f"    {prefix} {node.name}({args}){returns}:"

                # docstring 포함
                docstring = ast.get_docstring(node)
                if docstring:
                    lines.append(sig)
                    lines.append(f"        '''{docstring}''')
                    lines.append("        ...")
                else:
                    lines.append(sig)
                    lines.append("        ...")

        return "\n".join(lines)

    def _get_decorators(self, node) -> list[str]:
        """데코레이터 문자열 목록을 반환합니다."""
        decorators = []
        for dec in node.decorator_list:
            if isinstance(dec, ast.Name):
                decorators.append(f"    @{dec.id}")
            elif isinstance(dec, ast.Attribute):
                decorators.append(f"    @{ast.dump(dec)}")
        return decorators

    def _get_bases(self, node: ast.ClassDef) -> str:
        """클래스의 부모 클래스 목록을 반환합니다."""
        bases = []
        for base in node.bases:
            if isinstance(base, ast.Name):
                bases.append(base.id)
            elif isinstance(base, ast.Attribute):
                bases.append(f"{base.value.id}.{base.attr}" if isinstance(base.value,
        ast.Name) else base.attr)
        return ", ".join(bases)

    def _get_args(self, node) -> str:
        """함수 인자를 문자열로 반환합니다."""
        args = []
        for arg in node.args.args:
            annotation = ""
            if arg.annotation:
                annotation = f": {ast.unparse(arg.annotation)}"
            args.append(f"{arg.arg}{annotation}")
```


226

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
        return ", ".join(args)

    def _get_return_annotation(self, node) -> str:
        """반환 타입 어노테이션을 반환합니다."""
        if node.returns:
            return f" -> {ast.unparse(node.returns)}"
        return ""
```

압축 효과 시연

227

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
original_code = '''
class DataProcessor:
    """데이터를 처리하는 클래스."""

    def __init__(self, config: dict):
        self.config = config
        self.logger = logging.getLogger(__name__)
        self.cache = {}
        self._initialized = False

    def process(self, data: list[dict]) -> list[dict]:
        """데이터를 처리합니다."""
        results = []
        for item in data:
            validated = self._validate(item)
            if validated:
                transformed = self._transform(validated)
                results.append(transformed)
        self.logger.info(f"Processed {len(results)} items")
        return results

    def _validate(self, item: dict) -> dict | None:
        """항목을 검증합니다."""
        if "id" not in item:
            self.logger.warning("Missing id field")
            return None
        if "value" not in item:
            self.logger.warning("Missing value field")
            return None
        return item

    def _transform(self, item: dict) -> dict:
        """항목을 변환합니다."""
        return {
            "id": item["id"],
            "processed_value": item["value"] * 2,
            "timestamp": time.time(),
        }
'''

compressor = PythonCodeCompressor()
summary = compressor.extract_signatures(original_code)
print(summary)

# 출력:
# class DataProcessor():
#     def __init__(self, config: dict):
#         ...
#     def process(self, data: list[dict]) -> list[dict]:
#         '''데이터를 처리합니다.'''
#         ...
#     def _validate(self, item: dict) -> dict | None:
#         '''항목을 검증합니다.'''
#         ...
```


228

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
#     def _transform(self, item: dict) -> dict:
#         '''항목을 변환합니다.'''
#         ...

budget = TokenBudget(max_tokens=8000)
print(f"원본 토큰 수: {budget.count(original_code)}")  # ~200
print(f"요약 토큰 수: {budget.count(summary)}")         # ~60 (70% 절감)
```

2.6 노드 간 데이터 프루닝

후속 노드에서 더 이상 필요 없는 대용량 필드를 명시적으로 제거합니다.

```
def cleanup_after_analysis(state: AnalysisState) -> dict:
    """분석 완료 후 대용량 원본 데이터를 정리합니다.

    analyze 노드가 끝나면 원본 diff_text(30KB)와
    raw_code(30KB)는 더 이상 필요 없습니다.
    결과(analysis_result)만 남기고 원본을 제거합니다.
    """
    return {
        "diff_text": None,      # 30KB 해제
        "raw_code": None,       # 30KB 해제
        "current_step": "cleanup",
    }

# 그래프에서 분석 노드 후에 정리 노드 추가
# analyze -> cleanup -> generate -> ...
graph.add_edge("analyze", "cleanup")
graph.add_edge("cleanup", "generate")
```

2.7 대용량 데이터 청크 분할

30KB짜리 diff를 한 번에 LLM에 보내면 토큰 한도를 초과할 수 있습니다. 파일 단위로 분할하여 처리합니다.

229

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
def split_diff_by_file(diff_text: str) -> list[dict]:
    """Git diff를 파일 단위로 분할합니다.

    Args:
        diff_text: 전체 diff 문자열

    Returns:
        [{"file_path": "path/to/file.py", "diff": "--- a/...\n+++ b/...\n@@..."}]
    """
    chunks = []
    current_file = None
    current_lines = []

    for line in diff_text.split("\n"):
        if line.startswith("diff --git"):
            # 이전 파일 저장
            if current_file:
                chunks.append({
                    "file_path": current_file,
                    "diff": "\n".join(current_lines),
                })
            # 새 파일 시작
            parts = line.split(" b/")
            current_file = parts[-1] if len(parts) > 1 else "unknown"
            current_lines = [line]
        else:
            current_lines.append(line)

    # 마지막 파일
    if current_file:
        chunks.append({
            "file_path": current_file,
            "diff": "\n".join(current_lines),
        })

    return chunks


def analyze_diff_in_chunks(
    diff_text: str,
    budget: TokenBudget,
    call_llm,
) -> list[dict]:
    """diff를 파일 단위로 분할하여 LLM으로 분석합니다.

    Args:
        diff_text: 전체 diff 문자열
        budget: TokenBudget 인스턴스
        call_llm: LLM 호출 함수

    Returns:
        파일별 분석 결과 리스트
    """
    chunks = split_diff_by_file(diff_text)
```


230

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
    results = []

    for chunk in chunks:
        # 각 파일의 diff가 예산 내인지 확인
        chunk_text = chunk["diff"]

        if budget.remaining < 500:
            # 예산 부족 - 더 이상 분석 불가
            results.append({
                "file_path": chunk["file_path"],
                "analysis": "[토큰 예산 부족으로 건너뜀]",
            })
            continue

        # 예산에 맞게 절삭
        truncated = budget.truncate_to_budget(chunk_text, reserve=200)

        # LLM으로 분석
        analysis = call_llm(
            f"다음 파일의 변경사항을 분석하세요:\n\n"
            f"파일: {chunk['file_path']}\n"
            f"변경사항:\n{truncated}"
        )

        results.append({
            "file_path": chunk["file_path"],
            "analysis": analysis,
        })

    return results
```

---

3. 실전 예제

TokenBudget + Annotated Reducer 적용한 에이전트

231

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
import operator
from typing import Annotated, TypedDict
from langgraph.graph import StateGraph, END


def _replace(existing, new):
    return new


class OptimizedState(TypedDict):
    """최적화된 에이전트 State (Annotated Reducer 적용)."""
    query: Annotated[str, _replace]
    code_files: Annotated[list[dict] | None, _replace]  # 대용량 가능
    analysis: Annotated[str | None, _replace]
    suggestions: Annotated[list[str], operator.add]     # 누적 추가
    current_step: Annotated[str, _replace]
    error: Annotated[str | None, _replace]


# ── TokenBudget 활용 노드 ──

def prepare_context_node(state: OptimizedState) -> dict:
    """코드 파일들을 토큰 예산에 맞게 준비합니다."""
    budget = TokenBudget(max_tokens=6000)

    # 시스템 프롬프트 공간 확보
    budget.consume("시스템 프롬프트와 질문을 위한 공간 확보")

    code_files = state.get("code_files") or []
    compressor = PythonCodeCompressor()

    optimized_files = []
    for file_info in code_files:
        content = file_info["content"]
        path = file_info["path"]

        # 1. 원본이 예산 내이면 그대로 사용
        if budget.consume(content) is not None:
            optimized_files.append({"path": path, "content": content})
            continue

        # 2. 시그니처만 추출
        summary = compressor.extract_signatures(content)
        if budget.consume(summary) is not None:
            optimized_files.append({
                "path": path,
                "content": summary,
                "compressed": True,
            })
            continue

        # 3. 예산 부족 - 건너뜀
        optimized_files.append({
            "path": path,
```


232

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
            "content": f"# 토큰 예산 부족으로 생략 (원본 {budget.count(content)} 토큰)",
            "skipped": True,
        })

    return {
        "code_files": optimized_files,  # 최적화된 파일 목록으로 교체
        "current_step": "prepare_context",
    }


def analyze_node(state: OptimizedState) -> dict:
    """LLM으로 코드를 분석합니다."""
    code_context = "\n\n".join(
        f"# {f['path']}\n{f['content']}"
        for f in (state.get("code_files") or [])
    )

    result = call_llm(
        f"다음 코드를 분석하세요:\n\n"
        f"질문: {state['query']}\n\n"
        f"코드:\n{code_context}"
    )

    return {
        "analysis": result,
        "suggestions": ["코드 분석 완료"],  # 누적됨 (operator.add)
        "current_step": "analyze",
    }


def cleanup_node(state: OptimizedState) -> dict:
    """분석 완료 후 대용량 데이터를 정리합니다."""
    return {
        "code_files": None,     # 대용량 데이터 해제
        "suggestions": ["정리 완료"],
        "current_step": "cleanup",
    }


# ── 그래프 구성 ──

graph = StateGraph(OptimizedState)
graph.add_node("prepare_context", prepare_context_node)
graph.add_node("analyze", analyze_node)
graph.add_node("cleanup", cleanup_node)

graph.set_entry_point("prepare_context")
graph.add_edge("prepare_context", "analyze")
graph.add_edge("analyze", "cleanup")
graph.add_edge("cleanup", END)

app = graph.compile()

# 실행
result = app.invoke({
```


233

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
    "query": "이 코드에서 성능 문제를 찾아주세요",
    "code_files": [
        {"path": "main.py", "content": "...대용량 코드..."},
        {"path": "utils.py", "content": "...대용량 코드..."},
    ],
    "suggestions": [],
    "current_step": "start",
})

print(f"분석 결과: {result['analysis']}")
print(f"제안 사항: {result['suggestions']}")
# -> ["코드 분석 완료", "정리 완료"]  (operator.add로 누적됨)
```

---

4. 연습 문제

연습 1: TokenBudget 사용하기

```
# TODO: TokenBudget을 사용하여 다음 작업을 수행하세요
# 1. 총 4000 토큰 예산으로 TokenBudget 생성
# 2. 시스템 프롬프트 소비: "당신은 코드 리뷰 전문가입니다."
# 3. 사용자 질문 소비: "다음 코드를 리뷰해주세요."
# 4. 남은 예산 확인 후, 대용량 코드를 예산에 맞게 절삭

budget = TokenBudget(max_tokens=4000)

# TODO: 코드를 작성하세요
# system_prompt = ...
# user_query = ...
# code = "..." * 1000  # 대용량 코드
# truncated_code = budget.truncate_to_budget(code, reserve=500)
# print(f"남은 예산: {budget.remaining}")
```

연습 2: Annotated Reducer로 리팩터링

다음 코드를 Annotated Reducer 패턴으로 리팩터링하세요.

234

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# ── Before: 전체 복사 패턴 ──

class TaskState(TypedDict):
    task_id: str
    data: dict | None
    result: str | None
    logs: list[str]
    error: str | None

def node_a(state: TaskState) -> TaskState:
    return {
        **state,  # ← 전체 복사!
        "data": {"key": "value"},
        "logs": state.get("logs", []) + ["Node A 완료"],
    }

def node_b(state: TaskState) -> TaskState:
    return {
        **state,  # ← 또 전체 복사!
        "result": "분석 완료",
        "logs": state.get("logs", []) + ["Node B 완료"],
    }

# TODO: Annotated Reducer 패턴으로 변환하세요
# 힌트:
# - 각 필드에 Annotated[Type, _replace] 적용
# - logs 필드는 Annotated[list[str], operator.add] 사용
# - 노드는 변경 필드만 반환
```

연습 3: 코드 압축 비교

235

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# TODO: PythonCodeCompressor를 사용하여 다음 코드를 압축하고,
#       원본과 압축본의 토큰 수를 비교하세요.

sample_code = '''
import logging
from typing import Optional

logger = logging.getLogger(__name__)

class UserService:
    """사용자 관련 서비스."""

    def __init__(self, db_connection):
        self.db = db_connection
        self.cache = {}
        logger.info("UserService initialized")

    def get_user(self, user_id: int) -> Optional[dict]:
        """사용자 정보를 조회합니다."""
        if user_id in self.cache:
            return self.cache[user_id]
        user = self.db.query(f"SELECT * FROM users WHERE id = {user_id}")
        if user:
            self.cache[user_id] = user
        return user

    def create_user(self, name: str, email: str) -> dict:
        """새 사용자를 생성합니다."""
        user = {"name": name, "email": email}
        result = self.db.insert("users", user)
        logger.info(f"User created: {result}")
        return result

    def delete_user(self, user_id: int) -> bool:
        """사용자를 삭제합니다."""
        self.cache.pop(user_id, None)
        return self.db.delete("users", user_id)
'''

# compressor = PythonCodeCompressor()
# summary = compressor.extract_signatures(sample_code)
# budget = TokenBudget()
# print(f"원본 토큰: {budget.count(sample_code)}")
# print(f"압축 토큰: {budget.count(summary)}")
# print(f"절감률: {(1 - budget.count(summary) / budget.count(sample_code)) * 100:.0f}%")
```

---

5. 핵심 정리

최적화 전략 요약표

236

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

리소스	문제	해결 전략	핵심 도구
디스크	Git clone 누적	TTL 기반 자동 정리	`CloneManager`
디스크	전체 히스토리 clone	Shallow clone (`--depth 1`)	Git CLI 옵션
메모리	매 노드마다 State 전체 복사	Annotated Reducer 패턴	`Annotated[T, reducer]`
메모리	불필요한 대용량 필드 잔존	노드 간 데이터 프루닝	cleanup 노드
토큰	문자 수 기반 부정확한 절삭	tiktoken으로 정확한 토큰 계산	`TokenBudget`
토큰	코드 전체를 프롬프트에 포함	AST 기반 시그니처 추출	`PythonCodeCompressor`
토큰	대용량 diff 한 번에 처리	파일 단위 청크 분할	`split_diff_by_file`

Annotated Reducer 변환 Before/After

```python
# — Before —
class MyState(TypedDict):
    data: str | None
    result: str | None
    logs: list[str]

def my_node(state):
    return {
        **state,            # 전체 복사!
        "result": "done",
        "logs": state["logs"] + ["완료"],  }
    }
```

```python
# — After —
class MyState(TypedDict):
    data: Annotated[str | None, _replace]
    result: Annotated[str | None, _replace]
    logs: Annotated[list[str], operator.add]

def my_node(state):
    return {
     "result": "done",  # 변경분만!
       "logs": ["완료"],   # 누적 추가!
```

기억할 3가지 원칙

1. **측정하고 최적화하세요**: `TokenBudget.count()` 로 실제 토큰 수를 확인한 후 최적화하세요
2. **필요한 것만 보내세요**: LLM에 전체 코드 대신 시그니처, 전체 diff 대신 관련 파일만 보내세요
3. **쓰고 나면 정리하세요**: clone은 TTL로, State는 프루닝으로, 토큰은 예산으로 관리하세요

---

6. 참고 자료

237

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

자료	링크	설명
LangGraph State Reducers	langgraph - Reducers	Annotated Reducer 공식 문서
tiktoken	github.com/openai/tiktoken	OpenAI의 토큰 카운팅 라이브러리
Python operator 모듈	docs.python.org/operator	`operator.add` 등 내장 함수
Python ast 모듈	docs.python.org/ast	AST(추상 구문 트리) 파싱
Git shallow clone	git-scm.com/docs/git-clone	`--depth` 옵션 공식 문서
LangGraph Annotated 예제	langgraph - How to define state	State 정의 가이드

다음 단계

이번 모듈에서 에이전트의 3대 리소스(디스크, 메모리, LLM 토큰)를 효율적으로 관리하는 방법을 배웠습니다. Part 4의 핵심 주제인 에러 처리(Module 08), 외부 시스템 연동(Module 09), 리소스 최적화(Module 10)를 모두 다루었습니다.

이제 여러분은 프로덕션 수준의 안정적이고 효율적인 AI 에이전트를 만들 수 있는 기반을 갖추었습니다.

추천 다음 학습: - LangGraph 체크포인트와 영속성 (장애 복구) - LangGraph 서브그래프 (대규모 에이전트 모듈화) - 모니터링과 옵저버빌리티 (LangSmith, 커스텀 메트릭)

238

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

⬆ 목차로 돌아가기

Module 11: LLM 출력 품질 보장 - 신뢰할 수 있는 에이전트 만들기

학습 목표

이 모듈을 완료하면 다음을 할 수 있습니다:

1. LLM 출력 검증이 왜 필수인지 이해한다
2. 6단계 품질 보장 레이어 구조를 설명할 수 있다
3. Confidence 게이팅으로 낮은 신뢰도 결과를 필터링할 수 있다
4. Self-Reflection 루프를 LangGraph로 구현할 수 있다
5. Golden Set(평가 데이터셋)을 구축하고 자동 평가를 실행할 수 있다
6. Human-in-the-Loop 패턴을 LangGraph에서 구현할 수 있다

사전 지식

항목	필요 수준	참고 모듈
Python 기초	클래스, 함수, 딕셔너리	-
Pydantic	BaseModel, Field, validator	Module 06
LangGraph 기초	StateGraph, 노드, 엣지	Module 09
LLM API 호출	ChatAnthropic 또는 ChatOpenAI	Module 03

---

1. 개념 설명

1.1 LLM 출력을 왜 검증해야 하는가?

LLM은 강력하지만, 출력을 **항상 신뢰할 수는 없습니다**. 다음과 같은 문제가 발생할 수 있습니다:

239

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
[사용자 요청]                    [LLM 응답]
"JSON으로 분석 결과를      →    { "분석": "문제없음"  ← 필수 필드 누락!
 반환해주세요.                    }
 summary, confidence,
 affected_files 포함"
```

주요 문제 유형:

문제	설명	예시
환각 (Hallucination)	존재하지 않는 정보를 만들어냄	없는 함수명을 참조한 코드 생성
JSON 구조 오류	파싱 불가능한 형식 반환	닫는 괄호 누락, 잘못된 콤마
필수 필드 누락	요청한 필드를 빠뜨림	confidence 필드 없이 결과 반환
타입 불일치	기대와 다른 데이터 타입	숫자 대신 문자열 “0.85” 반환
낮은 신뢰도	LLM 스스로 확신이 없는 결과	confidence: 0.2인 분석 결과를 그대로 사용

검증 없이 LLM 출력을 사용하면?

```
LLM 출력 (검증 없음) → 잘못된 데이터 → 후속 처리 실패 → 시스템 전체 오류
                              ↓
                        사용자에게 잘못된 정보 전달
```

1.2 품질 보장 레이어 6단계 (피라미드 구조)

품질 보장은 한 번에 하나씩, **레이어를 쌓아올리는 방식**으로 구현합니다:

240

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
                ┌──────────────────────┐
                │  Layer 6: 정량 평가   │  ← Golden Set + 메트릭
                │  (Quantitative)      │
              ┌─┴──────────────────────┴─┐
              │  Layer 5: 인간 검증       │  ← Human-in-the-Loop
              │  (Human Review)          │
            ┌─┴──────────────────────────┴─┐
            │  Layer 4: 외부 검증           │  ← AST / 빌드 / 린트
            │  (External Validation)       │
          ┌─┴──────────────────────────────┴─┐
          │  Layer 3: 자기 검증                │  ← Self-Reflection
          │  (Self-Reflection)                │
        ┌─┴──────────────────────────────────┴─┐
        │  Layer 2: 의미적 게이팅                  │  ← Confidence 임계값
        │  (Semantic Gating)                     │
      ┌─┴────────────────────────────────────────┴─┐
      │  Layer 1: 스키마 유효성                       │  ← Pydantic 검증
      │  (Schema Validation)                        │
    ┌─┴─────────────────────────────────────────────┴─┐
    │  Layer 0: JSON 파싱                               │  ← 기본 파싱
    │  (JSON Parsing)                                  │
    └──────────────────────────────────────────────────┘
```

각 레이어의 역할:

레이어	검증 대상	질문	도구
Layer 0	구조	"유효한 JSON인가?"	json.loads()
Layer 1	스키마	"필수 필드와 타입이 맞는가?"	Pydantic
Layer 2	신뢰도	"LLM이 자신 있는 결과인가?"	Confidence 임계값
Layer 3	논리	"논리적으로 올바른가?"	LLM Self-Reflection
Layer 4	실행 가능성	"실제로 동작하는가?"	AST, 빌드, 린트
Layer 5	최종 승인	"사람이 봐도 괜찮은가?"	Human-in-the-Loop
Layer 6	정량 품질	"기대 수준을 충족하는가?"	Golden Set 평가

> **팁**: 모든 레이어를 한 번에 구현할 필요는 없습니다. Layer 0 → 1 → 2 순서로 점진적으로 추가하세요.

---

2. 단계별 실습

2.1 Layer 0 + Layer 1: JSON 파싱 + 스키마 검증

241

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

목표: LLM 출력을 JSON으로 파싱하고, Pydantic으로 구조를 검증합니다.

Step 1: 출력 스키마 정의

```
# schemas.py
"""LLM 출력 검증용 Pydantic 스키마 정의."""
from pydantic import BaseModel, Field, field_validator


class AnalysisResult(BaseModel):
    """LLM 분석 결과 스키마.

    LLM이 반환해야 하는 필수 필드와 타입을 정의합니다.
    """
    summary: str = Field(
        min_length=1,
        description="분석 요약"
    )
    confidence: float = Field(
        ge=0.0, le=1.0,
        description="분석 신뢰도 (0.0~1.0)"
    )
    issues: list[str] = Field(
        default_factory=list,
        description="발견된 문제 목록"
    )
    recommendation: str = Field(
        min_length=1,
        description="권장 조치사항"
    )

    @field_validator("confidence")
    @classmethod
    def round_confidence(cls, v: float) -> float:
        """confidence를 소수점 2자리로 반올림합니다."""
        return round(v, 2)
```

Step 2: JSON 파서 + 스키마 검증 결합

242

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# validated_parser.py
"""JSON 파싱 + Pydantic 스키마 검증을 결합한 파서."""
import json
import re
import logging
from typing import Type

from pydantic import BaseModel, ValidationError

logger = logging.getLogger(__name__)


def parse_json_from_llm(text: str) -> dict:
    """LLM 출력에서 JSON을 추출합니다.

    3단계 폴백 전략:
    1. 전체 텍스트를 JSON으로 파싱
    2. ```json ... ``` 마크다운 블록에서 추출
    3. 첫 번째 { ~ 마지막 } 사이를 추출
    """
    # 1단계: 직접 파싱 시도
    try:
        return json.loads(text)
    except json.JSONDecodeError:
        pass

    # 2단계: 마크다운 JSON 블록 추출
    pattern = r"```(?:json)?\s*\n?(.*?)\n?\s*```"
    match = re.search(pattern, text, re.DOTALL)
    if match:
        try:
            return json.loads(match.group(1))
        except json.JSONDecodeError:
            pass

    # 3단계: 중괄호 범위 추출
    first_brace = text.find("{")
    last_brace = text.rfind("}")
    if first_brace != -1 and last_brace > first_brace:
        try:
            return json.loads(text[first_brace:last_brace + 1])
        except json.JSONDecodeError:
            pass

    raise ValueError(f"JSON 파싱 실패: {text[:200]}...")


def validate_with_schema(
    raw_dict: dict,
    schema_class: Type[BaseModel],
    strict: bool = False,
) -> dict:
    """파싱된 dict를 Pydantic 스키마로 검증합니다.
```


243

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
    Args:
        raw_dict: JSON 파싱 결과 딕셔너리.
        schema_class: 검증에 사용할 Pydantic 모델 클래스.
        strict: True이면 검증 실패 시 예외 발생,
                False이면 복구 시도 후 실패하면 원본 반환.

    Returns:
        검증된 딕셔너리.
    """
    try:
        validated = schema_class.model_validate(raw_dict)
        logger.info(
            "스키마 검증 통과: %s",
            schema_class.__name__,
        )
        return validated.model_dump()

    except ValidationError as exc:
        logger.warning(
            "스키마 검증 실패: %s, 오류: %s",
            schema_class.__name__,
            exc.errors()[:3],  # 처음 3개 오류만 로깅
        )

        if strict:
            raise

        # 느슨한 모드: 기본값으로 복구 시도
        try:
            validated = schema_class.model_validate(raw_dict, strict=False)
            logger.info("기본값으로 복구 성공")
            return validated.model_dump()
        except ValidationError:
            logger.error("복구 실패 — 원본 dict 반환")
            return raw_dict


def parse_and_validate(
    text: str,
    schema_class: Type[BaseModel],
    strict: bool = False,
) -> dict:
    """LLM 출력 텍스트를 파싱하고 스키마로 검증합니다.

    JSON 파싱(Layer 0)과 스키마 검증(Layer 1)을 한 번에 수행합니다.
    """
    raw = parse_json_from_llm(text)
    return validate_with_schema(raw, schema_class, strict=strict)
```

Step 3: 테스트

244

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# test_validated_parser.py
"""파서 + 스키마 검증 테스트."""
from schemas import AnalysisResult
from validated_parser import parse_and_validate


def test_valid_json():
    """정상적인 JSON이 올바르게 파싱되고 검증되는지 테스트."""
    llm_output = '''```json
    {
        "summary": "로그인 API에 인증 누락 취약점이 발견되었습니다.",
        "confidence": 0.85,
        "issues": ["토큰 검증 미수행", "rate limit 미적용"],
        "recommendation": "JWT 검증 미들웨어를 추가하세요."
    }
    ```'''

    result = parse_and_validate(llm_output, AnalysisResult)

    assert result["summary"] == "로그인 API에 인증 누락 취약점이 발견되었습니다."
    assert result["confidence"] == 0.85
    assert len(result["issues"]) == 2
    print("테스트 통과: 정상 JSON 파싱 + 스키마 검증 성공")

def test_missing_field_recovery():
    """필수 필드가 누락된 경우 기본값으로 복구되는지 테스트."""
    llm_output = '{"summary": "분석 완료", "confidence": 0.9, "recommendation": "조치
        필요"}'

    # issues 필드 누락 → default_factory=list로 복구
    result = parse_and_validate(llm_output, AnalysisResult, strict=False)

    assert result["issues"] == []
    print("테스트 통과: 누락 필드 기본값 복구 성공")

def test_invalid_confidence_range():
    """confidence가 범위를 벗어나면 검증이 실패하는지 테스트."""
    llm_output = '{"summary": "분석", "confidence": 1.5, "recommendation": "조치"}'

    try:
        parse_and_validate(llm_output, AnalysisResult, strict=True)
        assert False, "ValidationError가 발생해야 합니다"
    except Exception:
        print("테스트 통과: 범위 초과 confidence 거부 성공")

if __name__ == "__main__":
    test_valid_json()
    test_missing_field_recovery()
    test_invalid_confidence_range()
    print("\n모든 테스트 통과!")
```


245
```

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

2.2 Layer 2: Confidence 게이팅

목표: LLM이 반환하는 신뢰도(confidence) 값으로 결과를 필터링합니다.

핵심 개념: 임계값별 동작 분기

```
confidence 값에 따른 동작 분기:

  0.0          0.4          0.7          1.0
   |──REJECT──|──RETRY───|──ACCEPT──|
   |            |            |            |
   |  "결과를    |  "다시     |  "결과를    |
   |   거부"     |   시도"    |   수락"     |
```

범위	동작	설명
0.7 이상	ACCEPT	결과를 그대로 사용
0.4 ~ 0.7 미만	RETRY	LLM에게 다시 요청 (최대 2회)
0.4 미만	REJECT	결과를 거부하고 에러 처리

구현:

246

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# confidence_gate.py
"""Confidence 기반 의사결정 게이팅."""
import logging
from dataclasses import dataclass
from enum import Enum

logger = logging.getLogger(__name__)


class GateAction(str, Enum):
    """게이팅 결과 동작."""
    ACCEPT = "accept"   # 결과 수락
    RETRY = "retry"     # 재시도 권장
    REJECT = "reject"   # 결과 거부


@dataclass
class GateResult:
    """게이팅 판단 결과."""
    action: GateAction
    confidence: float
    threshold_accept: float
    threshold_reject: float
    message: str


def confidence_gate(
    confidence: float,
    gate_name: str = "default",
    threshold_accept: float = 0.7,
    threshold_reject: float = 0.4,
) -> GateResult:
    """Confidence 값에 따라 ACCEPT / RETRY / REJECT를 결정합니다.

    Args:
        confidence: LLM이 반환한 신뢰도 (0.0 ~ 1.0).
        gate_name: 게이트 식별자 (로깅용).
        threshold_accept: 이 값 이상이면 ACCEPT.
        threshold_reject: 이 값 미만이면 REJECT. 그 사이는 RETRY.

    Returns:
        GateResult: 동작, 신뢰도, 임계값, 설명 메시지.

    Examples:
        >>> result = confidence_gate(0.85, "analysis")
        >>> result.action
        GateAction.ACCEPT

        >>> result = confidence_gate(0.5, "analysis")
        >>> result.action
        GateAction.RETRY
    """
    if confidence >= threshold_accept:
        action = GateAction.ACCEPT
```


247

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
        message = f"Confidence {confidence:.2f} >= {threshold_accept} → 수락"
    elif confidence >= threshold_reject:
        action = GateAction.RETRY
        message = f"Confidence {confidence:.2f} — 재시도 권장"
    else:
        action = GateAction.REJECT
        message = f"Confidence {confidence:.2f} < {threshold_reject} → 거부"

    logger.info("[%s] %s", gate_name, message)

    return GateResult(
        action=action,
        confidence=confidence,
        threshold_accept=threshold_accept,
        threshold_reject=threshold_reject,
        message=message,
    )
```

LangGraph 조건부 엣지에서 사용:

248

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# graph_with_confidence.py
"""Confidence 게이팅이 적용된 LangGraph 에이전트."""
from typing import TypedDict
from langgraph.graph import StateGraph, END
from langchain_anthropic import ChatAnthropic

from confidence_gate import confidence_gate, GateAction
from schemas import AnalysisResult
from validated_parser import parse_and_validate


class AgentState(TypedDict):
    """에이전트 상태."""
    query: str                  # 사용자 입력
    analysis: dict | None       # LLM 분석 결과
    retry_count: int            # 재시도 횟수
    max_retries: int            # 최대 재시도 횟수
    error: str | None           # 에러 메시지
    final_result: str | None    # 최종 결과


# --- 노드 정의 ---

def analyze_node(state: AgentState) -> dict:
    """LLM으로 분석을 수행하는 노드."""
    llm = ChatAnthropic(model="claude-sonnet-4-20250514", temperature=0)

    prompt = f"""다음 질문을 분석하고 JSON으로 응답하세요.

질문: {state["query"]}

반드시 아래 형식으로 응답하세요:
{{
    "summary": "분석 요약",
    "confidence": 0.0~1.0 사이 값,
    "issues": ["발견된 문제 목록"],
    "recommendation": "권장 조치사항"
}}"""

    response = llm.invoke(prompt)
    result = parse_and_validate(response.content, AnalysisResult)

    return {
        "analysis": result,
        "retry_count": state.get("retry_count", 0),
    }


def decide_after_analysis(state: AgentState) -> str:
    """Confidence 값에 따라 다음 경로를 결정합니다."""
    analysis = state.get("analysis")
    if analysis is None:
        return "handle_error"
```


249

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
    confidence = analysis.get("confidence", 0.0)
    gate = confidence_gate(confidence, "analysis")

    if gate.action == GateAction.ACCEPT:
        return "format_result"
    elif gate.action == GateAction.RETRY:
        retry_count = state.get("retry_count", 0)
        max_retries = state.get("max_retries", 2)
        if retry_count < max_retries:
            return "analyze"  # 재시도
        return "format_result"  # 재시도 소진 → 그대로 사용
    else:  # REJECT
        return "handle_error"


def format_result_node(state: AgentState) -> dict:
    """분석 결과를 최종 형태로 포맷합니다."""
    analysis = state["analysis"]
    return {
        "final_result": (
            f"분석 요약: {analysis['summary']}\n"
            f"신뢰도: {analysis['confidence']}\n"
            f"권장사항: {analysis['recommendation']}"
        ),
    }


def handle_error_node(state: AgentState) -> dict:
    """에러를 처리합니다."""
    return {
        "error": "분석 결과의 신뢰도가 너무 낮아 사용할 수 없습니다.",
        "final_result": None,
    }


# --- 그래프 구성 ---

def build_graph():
    """Confidence 게이팅이 적용된 분석 그래프를 구성합니다."""
    graph = StateGraph(AgentState)

    graph.add_node("analyze", analyze_node)
    graph.add_node("format_result", format_result_node)
    graph.add_node("handle_error", handle_error_node)

    graph.set_entry_point("analyze")

    graph.add_conditional_edges(
        "analyze",
        decide_after_analysis,
        {
            "analyze": "analyze",           # RETRY → 재시도
            "format_result": "format_result", # ACCEPT → 결과 포맷
            "handle_error": "handle_error",   # REJECT → 에러 처리
        },
```


250

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
    )
    graph.add_edge("format_result", END)
    graph.add_edge("handle_error", END)

    return graph.compile()


# 실행
if __name__ == "__main__":
    app = build_graph()
    result = app.invoke({
        "query": "Python 로그인 API의 보안 취약점을 분석해주세요.",
        "analysis": None,
        "retry_count": 0,
        "max_retries": 2,
        "error": None,
        "final_result": None,
    })
    print(result["final_result"] or result["error"])
```

2.3 Layer 3: Self-Reflection 루프

목표: LLM이 자신의 출력을 스스로 검토하여 품질을 높입니다.

핵심 개념:

```
┌──────────────────────────────────────────┐
│              Self-Reflection 루프          │
│                                          │
│   [generate] → [reflect] → [decision]    │
│       ↑            |           |         │
│       |        미승인 &        |         |
│       |      count < max       승인       |
│       └──────────┘             |         │
│                                ↓         │
│                            [다음 단계]     |
└──────────────────────────────────────────┘

  * 최대 반복 횟수를 반드시 제한! (무한 루프 방지)
```

전체 구현:

251

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# self_reflection_agent.py
"""Self-Reflection 루프가 적용된 에이전트."""
from typing import TypedDict
from langgraph.graph import StateGraph, END
from langchain_anthropic import ChatAnthropic

from validated_parser import parse_json_from_llm

# 최대 반복 횟수 (무한 루프 방지)
MAX_REFLECTION_ROUNDS = 2


class ReflectionState(TypedDict):
    """Self-Reflection 에이전트 상태."""
    topic: str                      # 작성 주제
    draft: str                      # 현재 초안
    reflection: dict | None         # 검토 결과
    reflection_count: int           # 검토 횟수
    final_output: str | None        # 최종 결과


# --- 노드 정의 ---

def generate_node(state: ReflectionState) -> dict:
    """초안을 생성하거나, 피드백을 반영하여 수정합니다."""
    llm = ChatAnthropic(model="claude-sonnet-4-20250514", temperature=0)

    reflection = state.get("reflection")

    if reflection and not reflection.get("approved", True):
        # 이전 검토에서 수정 요청이 있는 경우
        feedback = reflection.get("feedback", "")
        prompt = f"""이전 초안을 아래 피드백을 반영하여 수정해주세요.

이전 초안:
{state["draft"]}

피드백:
{feedback}

수정된 내용만 반환하세요."""
    else:
        # 최초 생성
        prompt = f"""다음 주제에 대해 간결하고 정확한 기술 문서를 작성해주세요.

주제: {state["topic"]}

핵심 내용을 포함하여 작성하세요."""

    response = llm.invoke(prompt)
    return {"draft": response.content}


def reflect_node(state: ReflectionState) -> dict:
```


252

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
    """생성된 초안을 비판적으로 검토합니다."""
    llm = ChatAnthropic(model="claude-sonnet-4-20250514", temperature=0)

    prompt = f"""당신은 엄격한 기술 리뷰어입니다. 아래 문서를 검토하세요.

문서:
{state["draft"]}

검토 관점:
1. 기술적으로 정확한가?
2. 중요한 내용이 누락되지 않았는가?
3. 설명이 명확한가?

반드시 JSON으로 응답하세요:
{{
    "approved": true 또는 false,
    "score": 1~10 점수,
    "feedback": "구체적인 피드백 (승인 시 빈 문자열)"
}}"""

    response = llm.invoke(prompt)
    result = parse_json_from_llm(response.content)

    return {
        "reflection": result,
        "reflection_count": state.get("reflection_count", 0) + 1,
    }


def decide_after_reflection(state: ReflectionState) -> str:
    """검토 결과에 따라 다음 경로를 결정합니다."""
    reflection = state.get("reflection", {})
    count = state.get("reflection_count", 0)

    # 승인되었거나 최대 반복 횟수에 도달하면 완료
    if reflection.get("approved", True) or count >= MAX_REFLECTION_ROUNDS:
        return "finalize"

    # 미승인이면 다시 생성
    return "generate"


def finalize_node(state: ReflectionState) -> dict:
    """최종 결과를 확정합니다."""
    count = state.get("reflection_count", 0)
    score = state.get("reflection", {}).get("score", "N/A")

    return {
        "final_output": (
            f"[검토 {count}회 완료, 점수: {score}]\n\n"
            f"{state['draft']}"
        ),
    }
```


253

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# --- 그래프 구성 ---

def build_reflection_graph():
    """Self-Reflection 루프가 포함된 그래프를 구성합니다.

    그래프 구조:
        generate → reflect → (approved?) → finalize
                     ↑           ↓ (not approved & count < max)
                     └──────────┘
    """
    graph = StateGraph(ReflectionState)

    graph.add_node("generate", generate_node)
    graph.add_node("reflect", reflect_node)
    graph.add_node("finalize", finalize_node)

    graph.set_entry_point("generate")
    graph.add_edge("generate", "reflect")

    # 순환 엣지: reflect → generate (미승인 시) 또는 finalize (승인 시)
    graph.add_conditional_edges(
        "reflect",
        decide_after_reflection,
        {
            "generate": "generate",   # 수정 후 재검토
            "finalize": "finalize",   # 승인 → 완료
        },
    )
    graph.add_edge("finalize", END)

    return graph.compile()


# 실행
if __name__ == "__main__":
    app = build_reflection_graph()
    result = app.invoke({
        "topic": "Python에서 async/await의 동작 원리",
        "draft": "",
        "reflection": None,
        "reflection_count": 0,
        "final_output": None,
    })
    print(result["final_output"])
```

2.4 Layer 4: 코드 수정 Pre-Apply 검증 (AST / 빌드 / 린트)

목표: LLM이 생성한 코드를 실제 적용하기 전에 구문 오류를 검출합니다.

254

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# code_validator.py
"""LLM 생성 코드의 Pre-Apply 검증."""
import ast
import subprocess
import tempfile
import os
import logging

logger = logging.getLogger(__name__)


def validate_python_syntax(code: str) -> dict:
    """Python 코드의 AST 구문 검증을 수행합니다.

    Args:
        code: 검증할 Python 코드 문자열.

    Returns:
        {"valid": bool, "error": str | None}
    """
    try:
        ast.parse(code)
        return {"valid": True, "error": None}
    except SyntaxError as e:
        return {
            "valid": False,
            "error": f"구문 오류 (Line {e.lineno}): {e.msg}",
        }


def validate_with_lint(code: str, filename: str = "temp.py") -> dict:
    """Ruff 린터로 코드 품질을 검사합니다.

    Args:
        code: 검사할 Python 코드.
        filename: 임시 파일 이름.

    Returns:
        {"valid": bool, "warnings": list[str]}
    """
    with tempfile.NamedTemporaryFile(
        mode="w", suffix=".py", delete=False
    ) as f:
        f.write(code)
        temp_path = f.name

    try:
        result = subprocess.run(
            ["ruff", "check", temp_path],
            capture_output=True,
            text=True,
            timeout=30,
        )
        warnings = [
```


255

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
            line for line in result.stdout.strip().split("\n") if line
        ]
        return {
            "valid": result.returncode == 0,
            "warnings": warnings,
        }
    except FileNotFoundError:
        logger.warning("ruff가 설치되어 있지 않습니다. 린트 검사를 건너뜁니다.")
        return {"valid": True, "warnings": ["ruff 미설치 — 검사 생략"]}
    finally:
        os.unlink(temp_path)


def pre_apply_validation(code: str) -> dict:
    """코드를 적용하기 전에 전체 검증을 수행합니다.

    검증 순서:
    1. AST 구문 검증 (필수)
    2. 린트 검사 (권장)

    Returns:
        {
            "can_apply": bool,
            "syntax": {"valid": bool, "error": str | None},
            "lint": {"valid": bool, "warnings": list[str]},
        }
    """
    syntax_result = validate_python_syntax(code)

    if not syntax_result["valid"]:
        # 구문 오류가 있으면 린트 검사도 건너뜀
        return {
            "can_apply": False,
            "syntax": syntax_result,
            "lint": {"valid": False, "warnings": ["구문 오류로 인해 생략"]},
        }

    lint_result = validate_with_lint(code)

    return {
        "can_apply": syntax_result["valid"],
        "syntax": syntax_result,
        "lint": lint_result,
    }


# 사용 예시
if __name__ == "__main__":
    # 정상 코드
    good_code = '''
def hello(name: str) -> str:
    return f"Hello, {name}!"
'''
    print("정상 코드 검증:", pre_apply_validation(good_code))
```


256

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
    # 구문 오류가 있는 코드
    bad_code = '''
def hello(name: str) -> str:
    return f"Hello, {name}!"
    if True  # SyntaxError: 콜론 누락
'''
    print("오류 코드 검증:", pre_apply_validation(bad_code))
```

2.5 Layer 5: Human-in-the-Loop

목표: 중요한 결정 전에 사람의 승인을 받습니다.

257

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# human_in_the_loop.py
"""LangGraph Human-in-the-Loop 패턴."""
from typing import TypedDict
from langgraph.graph import StateGraph, END
from langgraph.checkpoint.memory import MemorySaver


class ApprovalState(TypedDict):
    """승인 워크플로우 상태."""
    task_description: str
    generated_plan: str | None
    human_approved: bool | None
    final_action: str | None


def generate_plan_node(state: ApprovalState) -> dict:
    """실행 계획을 생성합니다."""
    # 실제로는 LLM을 호출하여 계획을 생성
    plan = f"계획: '{state['task_description']}'에 대해 다음 작업을 수행합니다..."
    return {"generated_plan": plan}


def execute_plan_node(state: ApprovalState) -> dict:
    """승인된 계획을 실행합니다."""
    return {"final_action": f"실행 완료: {state['generated_plan']}"}


def reject_plan_node(state: ApprovalState) -> dict:
    """거부된 계획을 처리합니다."""
    return {"final_action": "계획이 거부되어 실행하지 않았습니다."}


def check_approval(state: ApprovalState) -> str:
    """승인 여부에 따라 분기합니다."""
    if state.get("human_approved"):
        return "execute"
    return "reject"


def build_approval_graph():
    """Human-in-the-Loop 그래프를 구성합니다.

    핵심: interrupt_before를 사용하여 'execute' 노드 실행 전에
    그래프를 일시 중단하고, 사람의 승인을 기다립니다.

    그래프 흐름:
        generate_plan → [일시 중단 / 사람 승인 대기]
                            → check_approval
                                → execute (승인) → END
                                → reject (거부)  → END
    """
    graph = StateGraph(ApprovalState)

    graph.add_node("generate_plan", generate_plan_node)
```


258

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
    graph.add_node("check_approval_gate", lambda state: state)  # 패스스루 노드
    graph.add_node("execute", execute_plan_node)
    graph.add_node("reject", reject_plan_node)

    graph.set_entry_point("generate_plan")
    graph.add_edge("generate_plan", "check_approval_gate")

    graph.add_conditional_edges(
        "check_approval_gate",
        check_approval,
        {"execute": "execute", "reject": "reject"},
    )
    graph.add_edge("execute", END)
    graph.add_edge("reject", END)

    # interrupt_before: check_approval_gate 노드 실행 전에 중단
    checkpointer = MemorySaver()
    return graph.compile(
        checkpointer=checkpointer,
        interrupt_before=["check_approval_gate"],
    )


# 사용 예시
if __name__ == "__main__":
    app = build_approval_graph()
    config = {"configurable": {"thread_id": "approval-001"}}

    # Step 1: 계획 생성까지 실행 (interrupt_before로 중단됨)
    result = app.invoke(
        {
            "task_description": "프로덕션 DB 스키마 변경",
            "generated_plan": None,
            "human_approved": None,
            "final_action": None,
        },
        config=config,
    )
    print("생성된 계획:", result["generated_plan"])

    # Step 2: 사람이 검토 후 승인/거부 결정
    # (실제 시스템에서는 UI나 API로 입력을 받음)
    human_decision = True  # True: 승인, False: 거부

    # Step 3: 상태를 업데이트하고 재개
    app.update_state(
        config,
        {"human_approved": human_decision},
    )

    # Step 4: 중단된 지점부터 계속 실행
    result = app.invoke(None, config=config)
    print("최종 결과:", result["final_action"])
```


259

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

2.6 Layer 6: Golden Set (평가 데이터셋)

목표: LLM 출력 품질을 정량적으로 측정하는 데이터셋을 구축합니다.

Golden Set이란?

```
Golden Set = 입력 + 기대 출력 + 평가 기준의 모음

┌──────────────────────────────────────────────────────────┐
│  Golden Case #001                                         │
│                                                           │
│  입력: "로그인 API에서 NullPointerException 발생"     │
│                                                           │
│  기대 출력:                                              │
│    - summary: "인증 토큰 null 체크 누락"              │
│    - confidence: 0.7 이상                                 │
│    - issues: 1개 이상                                    │
│                                                           │
│  평가 기준:                                              │
│    - summary가 비어있지 않은가? (weight: 1.0)         │
│    - confidence가 0.7 이상인가? (weight: 0.8)        │
│    - issues에 항목이 있는가? (weight: 0.5)           │
└──────────────────────────────────────────────────────────┘
```

Step 1: Golden Set JSON 파일 작성

260

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
{
    "case_id": "analysis_001",
    "description": "NullPointerException 버그 분석",
    "input": {
        "query": "UserService.java의 getUser() 메서드에서 NullPointerException이 발생합니다.
        user 객체가 null일 때 getName()을 호출합니다."
    },
    "expected_output": {
        "summary": "null 체크 누락으로 인한 NullPointerException",
        "confidence": 0.85,
        "issues": ["user 객체 null 체크 누락"],
        "recommendation": "user != null 조건 검사 추가"
    },
    "evaluation_criteria": [
        {
            "name": "summary_not_empty",
            "check_type": "field_exists",
            "field": "summary",
            "weight": 1.0
        },
        {
            "name": "confidence_above_threshold",
            "check_type": "value_range",
            "field": "confidence",
            "min_value": 0.5,
            "max_value": 1.0,
            "weight": 0.8
        },
        {
            "name": "has_issues",
            "check_type": "list_min_length",
            "field": "issues",
            "min_length": 1,
            "weight": 0.5
        },
        {
            "name": "recommendation_not_empty",
            "check_type": "field_exists",
            "field": "recommendation",
            "weight": 0.7
        }
    ]
}
```

Step 2: 평가 러너 구현

261

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# golden_set_runner.py
"""Golden Set 기반 LLM 출력 품질 자동 평가."""
import json
import logging
from pathlib import Path

logger = logging.getLogger(__name__)


def evaluate_single_case(case: dict, actual_output: dict) -> dict:
    """단일 Golden Case를 평가합니다.

    Args:
        case: Golden Set 케이스 (기대 출력 + 평가 기준 포함).
        actual_output: LLM이 실제로 반환한 출력.

    Returns:
        평가 결과 딕셔너리 (점수, 개별 체크 결과 포함).
    """
    criteria = case.get("evaluation_criteria", [])
    checks = []
    total_weight = 0.0
    weighted_score = 0.0

    for criterion in criteria:
        weight = criterion.get("weight", 1.0)
        total_weight += weight

        passed = _run_check(criterion, actual_output)
        checks.append({
            "name": criterion["name"],
            "passed": passed,
            "weight": weight,
        })

        if passed:
            weighted_score += weight

    score = weighted_score / total_weight if total_weight > 0 else 0.0

    return {
        "case_id": case["case_id"],
        "score": round(score, 3),
        "passed": score >= 0.8,  # 80% 이상이면 통과
        "checks": checks,
    }


def _run_check(criterion: dict, actual_output: dict) -> bool:
    """개별 평가 기준을 실행합니다."""
    check_type = criterion["check_type"]
    field = criterion["field"]
    value = actual_output.get(field)
```


262

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
    if check_type == "field_exists":
        return value is not None and value != ""

    elif check_type == "value_range":
        if value is None:
            return False
        min_val = criterion.get("min_value", float("-inf"))
        max_val = criterion.get("max_value", float("inf"))
        return min_val <= float(value) <= max_val

    elif check_type == "list_min_length":
        if not isinstance(value, list):
            return False
        return len(value) >= criterion.get("min_length", 1)

    elif check_type == "exact_match":
        return value == criterion.get("expected_value")

    elif check_type == "contains":
        return criterion.get("substring", "") in str(value)

    return False


def run_golden_set(
    golden_set_dir: str,
    llm_function,
) -> dict:
    """Golden Set 디렉토리의 모든 케이스를 평가합니다.

    Args:
        golden_set_dir: Golden Set JSON 파일들이 있는 디렉토리 경로.
        llm_function: 입력을 받아 dict를 반환하는 LLM 호출 함수.

    Returns:
        전체 평가 요약 (평균 점수, 통과/실패 수, 상세 결과).
    """
    cases_dir = Path(golden_set_dir)
    results = []

    for case_file in sorted(cases_dir.glob("*.json")):
        case = json.loads(case_file.read_text(encoding="utf-8"))

        # LLM 호출
        actual_output = llm_function(case["input"])

        # 평가
        result = evaluate_single_case(case, actual_output)
        results.append(result)

        status = "PASS" if result["passed"] else "FAIL"
        logger.info(
            "[%s] %s — score=%.3f",
            status, case["case_id"], result["score"],
        )
```


263

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
    # 전체 요약
    total = len(results)
    passed = sum(1 for r in results if r["passed"])
    avg_score = sum(r["score"] for r in results) / total if total > 0 else 0.0

    summary = {
        "total_cases": total,
        "passed": passed,
        "failed": total - passed,
        "pass_rate": round(passed / total, 3) if total > 0 else 0.0,
        "average_score": round(avg_score, 3),
        "details": results,
    }

    print(f"\n{'='*50}")
    print(f"Golden Set 평가 결과")
    print(f"{'='*50}")
    print(f"총 케이스: {total}")
    print(f"통과: {passed} / 실패: {total - passed}")
    print(f"통과율: {summary['pass_rate']*100:.1f}%")
    print(f"평균 점수: {summary['average_score']:.3f}")
    print(f"{'='*50}")

    return summary
```

---

3. 실전 예제: 6-레이어 품질 보장이 적용된 “코드 리뷰 에이전트”

6개 레이어를 모두 결합한 실전 에이전트를 구현합니다.

264

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# code_review_agent.py
"""6-레이어 품질 보장이 적용된 코드 리뷰 에이전트.

Layer 0: JSON 파싱 (parse_json_from_llm)
Layer 1: 스키마 검증 (Pydantic CodeReviewResult)
Layer 2: Confidence 게이팅 (confidence_gate)
Layer 3: Self-Reflection (reflect_review 노드)
Layer 4: 외부 검증 (코드 구문 검사)
Layer 5: Human-in-the-Loop (interrupt_before)
"""
from typing import TypedDict
from pydantic import BaseModel, Field, field_validator

from langgraph.graph import StateGraph, END
from langgraph.checkpoint.memory import MemorySaver
from langchain_anthropic import ChatAnthropic

from validated_parser import parse_and_validate
from confidence_gate import confidence_gate, GateAction

MAX_REFLECTION_ROUNDS = 2


# --- Layer 1: 스키마 정의 ---

class ReviewIssue(BaseModel):
    """코드 리뷰에서 발견된 문제."""
    severity: str = Field(pattern=r"^(HIGH|MEDIUM|LOW)$")
    description: str = Field(min_length=1)
    line_number: int | None = None
    suggestion: str = ""


class CodeReviewResult(BaseModel):
    """코드 리뷰 결과 스키마."""
    summary: str = Field(min_length=1)
    confidence: float = Field(ge=0.0, le=1.0)
    issues: list[ReviewIssue] = Field(default_factory=list)
    approved: bool
    overall_quality: str = Field(pattern=r"^(GOOD|ACCEPTABLE|NEEDS_WORK|POOR)$")

    @field_validator("confidence")
    @classmethod
    def round_confidence(cls, v: float) -> float:
        return round(v, 2)


# --- 상태 정의 ---

class CodeReviewState(TypedDict):
    code: str                           # 리뷰할 코드
    review: dict | None                 # 리뷰 결과
    reflection: dict | None             # Self-Reflection 결과
    reflection_count: int               # Reflection 횟수
```


265

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
    human_approved: bool | None         # 인간 승인 여부
    final_result: str | None            # 최종 결과
    error: str | None


# --- 노드 정의 ---

def review_code_node(state: CodeReviewState) -> dict:
    """코드를 리뷰합니다. (Layer 0 + 1 적용)"""
    llm = ChatAnthropic(model="claude-sonnet-4-20250514", temperature=0)

    prompt = f"""다음 코드를 리뷰하고 JSON으로 응답하세요.

코드:
```

{state[“code”]}

266

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
JSON 응답 형식:
{{
    "summary": "리뷰 요약",
    "confidence": 0.0~1.0,
    "issues": [
        {{"severity": "HIGH|MEDIUM|LOW", "description": "문제 설명", "line_number": null,
"suggestion": "수정 제안"}}
    ],
    "approved": true/false,
    "overall_quality": "GOOD|ACCEPTABLE|NEEDS_WORK|POOR"
}}"""

    response = llm.invoke(prompt)

    # Layer 0 (JSON 파싱) + Layer 1 (스키마 검증)
    review = parse_and_validate(response.content, CodeReviewResult)
    return {"review": review}


def confidence_check_node(state: CodeReviewState) -> str:
    """Confidence 게이팅을 적용합니다. (Layer 2)"""
    review = state.get("review", {})
    gate = confidence_gate(
        review.get("confidence", 0.0),
        "code_review",
    )

    if gate.action == GateAction.ACCEPT:
        return "reflect"
    elif gate.action == GateAction.RETRY:
        return "review_code"  # 재시도
    else:
        return "handle_error"


def reflect_review_node(state: CodeReviewState) -> dict:
    """리뷰 결과를 자기 검토합니다. (Layer 3)"""
    count = state.get("reflection_count", 0)

    if count >= MAX_REFLECTION_ROUNDS:
        return {"reflection": {"approved": True}, "reflection_count": count}

    llm = ChatAnthropic(model="claude-sonnet-4-20250514", temperature=0)

    prompt = f"""당신은 시니어 코드 리뷰어입니다. 아래 리뷰 결과를 검토하세요.

리뷰 결과:
{state["review"]}

검토 관점:
1. 리뷰가 공정하고 정확한가?
2. 놓친 중요한 이슈가 있는가?
3. 제안이 실용적인가?
```


267

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
JSON으로 응답:
{{"approved": true/false, "feedback": "피드백"}}"""

    response = llm.invoke(prompt)
    from validated_parser import parse_json_from_llm
    reflection = parse_json_from_llm(response.content)

    return {
        "reflection": reflection,
        "reflection_count": count + 1,
    }


def decide_after_reflection(state: CodeReviewState) -> str:
    """Reflection 결과에 따라 분기합니다."""
    reflection = state.get("reflection", {})
    count = state.get("reflection_count", 0)

    if reflection.get("approved", True) or count >= MAX_REFLECTION_ROUNDS:
        return "human_gate"
    return "review_code"  # 미승인 → 재생성


def human_gate_node(state: CodeReviewState) -> dict:
    """Human-in-the-Loop 게이트. (Layer 5)"""
    return state  # interrupt_before에 의해 여기서 중단됨


def check_human_decision(state: CodeReviewState) -> str:
    """인간의 최종 결정에 따라 분기합니다."""
    if state.get("human_approved", True):
        return "finalize"
    return "review_code"  # 인간이 거부하면 재시도


def finalize_node(state: CodeReviewState) -> dict:
    """최종 결과를 확정합니다."""
    review = state.get("review", {})
    return {
        "final_result": (
            f"코드 리뷰 완료\n"
            f"요약: {review.get('summary', 'N/A')}\n"
            f"품질: {review.get('overall_quality', 'N/A')}\n"
            f"신뢰도: {review.get('confidence', 'N/A')}\n"
            f"발견된 이슈: {len(review.get('issues', []))}건"
        ),
    }


def handle_error_node(state: CodeReviewState) -> dict:
    """에러를 처리합니다."""
    return {"error": "코드 리뷰 실패: 신뢰도가 너무 낮습니다."}
```


268

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
# --- 그래프 구성 ---

def build_code_review_graph():
    """6-레이어 품질 보장이 적용된 코드 리뷰 그래프.

    흐름:
        review_code → confidence_check
            → (ACCEPT) → reflect → (approved?) → human_gate
            →                          ↑    ↓(미승인)       → finalize
            →                          └────┘                → END
            → (RETRY) → review_code (재시도)
            → (REJECT) → handle_error → END
    """
    graph = StateGraph(CodeReviewState)

    graph.add_node("review_code", review_code_node)
    graph.add_node("reflect", reflect_review_node)
    graph.add_node("human_gate", human_gate_node)
    graph.add_node("finalize", finalize_node)
    graph.add_node("handle_error", handle_error_node)

    graph.set_entry_point("review_code")

    graph.add_conditional_edges(
        "review_code",
        confidence_check_node,
        {
            "reflect": "reflect",
            "review_code": "review_code",
            "handle_error": "handle_error",
        },
    )

    graph.add_conditional_edges(
        "reflect",
        decide_after_reflection,
        {
            "human_gate": "human_gate",
            "review_code": "review_code",
        },
    )

    graph.add_conditional_edges(
        "human_gate",
        check_human_decision,
        {
            "finalize": "finalize",
            "review_code": "review_code",
        },
    )

    graph.add_edge("finalize", END)
    graph.add_edge("handle_error", END)

    checkpointer = MemorySaver()
```


269

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
    return graph.compile(
        checkpointer=checkpointer,
        interrupt_before=["human_gate"],
    )
```

---

4. 연습 문제

연습 1: Self-Reflection 루프 추가하기 (난이도: 중)

과제: Module 09에서 만든 에이전트 (또는 아래 간단한 에이전트)에 Self-Reflection 루프를 추가하세요.

시작 코드 (Self-Reflection 없음):

```python
# exercise_start.py
from typing import TypedDict
from langgraph.graph import StateGraph, END
from langchain_anthropic import ChatAnthropic


class TranslationState(TypedDict):
    source_text: str          # 원문
    translated: str | None    # 번역 결과
    final_output: str | None


def translate_node(state: TranslationState) -> dict:
    llm = ChatAnthropic(model="claude-sonnet-4-20250514")
    prompt = f"다음 한국어를 영어로 번역하세요:\n\n{state['source_text']}"
    response = llm.invoke(prompt)
    return {"translated": response.content}


def finalize_node(state: TranslationState) -> dict:
    return {"final_output": state["translated"]}


graph = StateGraph(TranslationState)
graph.add_node("translate", translate_node)
graph.add_node("finalize", finalize_node)
graph.set_entry_point("translate")
graph.add_edge("translate", "finalize")
graph.add_edge("finalize", END)
app = graph.compile()
```

요구사항: 1. `reflect_translation` 노드를 추가하세요 (번역 품질 검토) 2. 미승인 시 `translate` 노드로 돌아가는 순환 구조를 구현하세요 3. 최대 반복 횟수를 2회로 제한하세요 4. `reflection_count` 필드를 상태에 추가하세요

270

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

힌트: Section 2.3의 Self-Reflection 패턴을 참고하세요.

연습 2: Confidence 게이팅 + Golden Set 평가 (난이도: 상)

과제: 1. 아래 에이전트에 Confidence 게이팅을 추가하세요 (임계값: 0.6) 2. Golden Set JSON 파일 3개를 작성하세요 3. `run_golden_set()` 함수로 평가를 실행하세요

```
# 힌트: 다음 구조를 참고하세요
#
# golden_sets/
# ├── case_001.json
# ├── case_002.json
# └── case_003.json
#
# 각 JSON 파일에는 input, expected_output, evaluation_criteria를 포함하세요.
```

---

5. 핵심 정리

6단계 품질 보장 레이어 요약

```
┌──────────────────────────────────────────────────────────┐
│                                                          │
│  Layer 0  JSON 파싱         → json.loads() + 폴백 전략    │
│  Layer 1  스키마 검증       → Pydantic BaseModel          │
│  Layer 2  Confidence 게이팅 → 임계값 기반 ACCEPT/RETRY/REJECT │
│  Layer 3  Self-Reflection  → LLM이 자기 출력을 검토       │
│  Layer 4  외부 검증         → AST / 빌드 / 린트           │
│  Layer 5  Human-in-the-Loop → interrupt_before + 상태 업데이트 │
│  Layer 6  정량 평가         → Golden Set + 자동 메트릭    │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

기억해야 할 핵심 원칙

271

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

원칙	설명
점진적 적용	Layer 0 → 1 → 2 순서로 쌓아 올리기
무한 루프 방지	Self-Reflection에 반드시 max_rounds 설정
임계값 조정	초기엔 낮게 설정 → 데이터 수집 후 최적화
정량적 측정	Golden Set으로 "좋아졌는지/나빠졌는지" 객관적 판단
인간 개입 지점	위험도 높은 작업(DB 변경, 프로덕션 배포) 전에 배치

---

6. 참고 자료

주제	링크
LangGraph Human-in-the-Loop	https://langchain-ai.github.io/langgraph/concepts/human_in_the_loop/
LangGraph 순환 그래프 (Cycles)	https://langchain-ai.github.io/langgraph/how-tos/create-react-agent/
Pydantic Validation	https://docs.pydantic.dev/latest/concepts/validators/
LLM Evaluation (Anthropic)	https://docs.anthropic.com/en/docs/build-with-claude/develop-tests
LangGraph Conditional Edges	https://langchain-ai.github.io/langgraph/how-tos/branching/
JSON 파싱 전략	https://python.langchain.com/docs/how_to/output_parser_json/

다음 단계

축하합니다! LLM 출력 품질 보장의 6단계 레이어를 모두 학습했습니다.

다음 모듈에서 배울 내용: - **Module 12: LangGraph 고급 패턴** — 체크포인팅으로 장애 복구, 서브그래프로 모듈 분리, 병렬 실행으로 성능 향상 - 지금까지 만든 에이전트에 프로덕션 수준의 안정성을 추가합니다

복습 체크리스트: - [ ] Pydantic으로 LLM 출력 스키마를 정의할 수 있다 - [ ] Confidence 게이팅으로 3단계 분기를 구현할 수 있다 - [ ] Self-Reflection 루프를 LangGraph 순환 그래프로 구현할 수 있다 - [ ] Golden Set을 작성하고 자동 평가를 실행할 수 있다 - [ ] Human-in-the-Loop 패턴을 interrupt_before로 구현할 수 있다

272

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

⬆ 목차로 돌아가기

Module 12: LangGraph 고급 패턴 - 프로덕션 수준의 에이전트 아키텍처

학습 목표

이 모듈을 완료하면 다음을 할 수 있습니다:

1. 체크포인팅으로 장애 발생 시 마지막 성공 지점부터 재개할 수 있다
2. 서브그래프로 복잡한 워크플로우를 모듈 단위로 분리할 수 있다
3. Send API로 병렬 실행(Fan-out/Fan-in)을 구현할 수 있다
4. 스트리밍으로 실시간 진행 상태를 전달할 수 있다
5. 동적 노드 구성과 커스텀 Reducer를 활용할 수 있다
6. 그래프를 시각화하고 디버깅할 수 있다

사전 지식

항목	필요 수준	참고 모듈
LangGraph 기초	StateGraph, 노드, 엣지, 조건부 분기	Module 09
Python async/await	기본적인 비동기 문법	-
LLM API 호출	ChatAnthropic 또는 ChatOpenAI	Module 03
TypedDict	Python 타입 힌트	Module 09

---

1. 개념 설명

1.1 왜 고급 패턴이 필요한가?

기본 LangGraph 그래프로도 에이전트를 만들 수 있지만, **프로덕션 환경**에서는 추가적인 요구사항이 생깁니다:

273

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
[개발 환경]                          [프로덕션 환경]
  에이전트 실행 실패          →      처음부터 다시? (비용 + 시간 낭비)
  모든 노드가 한 그래프에      →      재사용 불가, 유지보수 어려움
  하나씩 순차 처리            →      느림 (병렬 처리 필요)
  실행 완료까지 상태 모름      →      사용자는 진행 상황을 알고 싶음
```

고급 패턴이 해결하는 문제:

문제	패턴	효과
장애 시 처음부터 재시작	체크포인팅	마지막 성공 지점에서 재개
그래프가 너무 복잡	서브그래프	모듈로 분리하여 재사용
직렬 처리로 느림	병렬 실행 (Send API)	여러 작업 동시 처리
진행 상황 알 수 없음	스트리밍	실시간 상태 전달
설정에 따라 다른 동작	동적 노드 구성	유연한 그래프 빌드

1.2 이 모듈에서 다루는 패턴 전체 그림

```
┌──────────────────────────────────────────────────────────────┐
│                    LangGraph 고급 패턴                        │
│                                                               │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────────┐   │
│  │ 1. Checkpoint │  │ 2. Subgraph  │  │ 3. Parallel      │   │
│  │  (영속성)      │  │  (모듈화)     │  │  (병렬 실행)      │   │
│  │              │  │              │  │                  │   │
│  │  MemorySaver │  │  부모 ↔ 자식  │  │  Send API        │   │
│  │  SqliteSaver │  │  상태 매핑    │  │  Fan-out/Fan-in  │   │
│  │  PostgresSaver│  │  재사용 가능  │  │  operator.add    │   │
│  └──────────────┘  └──────────────┘  └──────────────────┘   │
│                                                               │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────────┐   │
│  │ 4. Streaming  │  │ 5. Dynamic   │  │ 6. State/Reducer │   │
│  │  (실시간)      │  │  (동적 구성)  │  │  (상태 관리)      │   │
│  │              │  │              │  │                  │   │
│  │  astream     │  │  설정 기반    │  │  Annotated       │   │
│  │  토큰 스트림  │  │  Feature flag │  │  Custom Reducer  │   │
│  │  이벤트 감지  │  │  조건부 노드  │  │  add_messages    │   │
│  └──────────────┘  └──────────────┘  └──────────────────┘   │
│                                                               │
│  ┌──────────────────────────────────────────────────────┐   │
│  │ 7. 시각화 & 디버깅 — draw_mermaid, LangSmith          │   │
│  └──────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────┘
```

274

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

2. 단계별 실습

2.1 체크포인팅 (Persistence)

왜 필요한가?

에이전트가 5개 노드를 실행하는 중에 4번째 노드에서 장애가 발생하면:

```
체크포인팅 없이:
  [Node A] → [Node B] → [Node C] → [Node D] ← 장애 발생!
                                          ↓
  전체 재시작 → [Node A] → [Node B] → [Node C] → [Node D] → ...
  (Node A~C를 다시 실행 = 시간 + 비용 낭비)

체크포인팅 있으면:
  [Node A] → ✓저장 → [Node B] → ✓저장 → [Node C] → ✓저장 → [Node D] ← 장애!
                                                          ↓
  마지막 체크포인트에서 재개 → [Node D] → [Node E] → 완료!
  (Node A~C는 건너뜀 = 시간 + 비용 절약)
```

체크포인터 종류 비교

체크포인터	저장 위치	재시작 시 복구	용도
MemorySaver	메모리 (RAM)	프로세스 재시작 시 유실	개발/테스트
SqliteSaver	SQLite 파일	파일이 있으면 복구 가능	로컬 개발
PostgresSaver	PostgreSQL	영속적 복구 가능	프로덕션

Step 1: MemorySaver (테스트용)

275

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# checkpoint_memory.py
"""MemorySaver를 사용한 체크포인팅 기본 예제."""
from typing import TypedDict
from langgraph.graph import StateGraph, END
from langgraph.checkpoint.memory import MemorySaver


class ProcessState(TypedDict):
    """처리 상태."""
    input_data: str
    step1_result: str | None
    step2_result: str | None
    step3_result: str | None
    current_step: str


def step1_node(state: ProcessState) -> dict:
    """1단계: 데이터 수집."""
    print("  [Step 1] 데이터 수집 중...")
    result = f"수집 완료: {state['input_data']}"
    return {"step1_result": result, "current_step": "step1"}


def step2_node(state: ProcessState) -> dict:
    """2단계: 데이터 분석."""
    print("  [Step 2] 데이터 분석 중...")
    result = f"분석 완료: {state['step1_result']}"
    return {"step2_result": result, "current_step": "step2"}


def step3_node(state: ProcessState) -> dict:
    """3단계: 결과 생성."""
    print("  [Step 3] 결과 생성 중...")
    result = f"최종 결과: {state['step2_result']}"
    return {"step3_result": result, "current_step": "step3"}


# 그래프 구성
graph = StateGraph(ProcessState)
graph.add_node("step1", step1_node)
graph.add_node("step2", step2_node)
graph.add_node("step3", step3_node)

graph.set_entry_point("step1")
graph.add_edge("step1", "step2")
graph.add_edge("step2", "step3")
graph.add_edge("step3", END)

# 체크포인터 연결
checkpointer = MemorySaver()
app = graph.compile(checkpointer=checkpointer)

# --- 실행 ---
```


276

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# thread_id: 실행을 식별하는 고유 키 (같은 ID면 이전 상태에서 재개)
config = {"configurable": {"thread_id": "task-001"}}

result = app.invoke(
    {
        "input_data": "사용자 로그 데이터",
        "step1_result": None,
        "step2_result": None,
        "step3_result": None,
        "current_step": "start",
    },
    config=config,
)

print(f"\n최종 결과: {result['step3_result']}")

# 체크포인트 상태 확인
state_snapshot = app.get_state(config)
print(f"마지막 단계: {state_snapshot.values['current_step']}")
```

Step 2: SqliteSaver (로컬 영속화)

277

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# checkpoint_sqlite.py
"""SqliteSaver를 사용한 영속적 체크포인팅.

프로그램을 종료하고 다시 실행해도 마지막 체크포인트에서 재개됩니다.
"""
from typing import TypedDict

from langgraph.graph import StateGraph, END
from langgraph.checkpoint.sqlite import SqliteSaver


class JobState(TypedDict):
    """작업 상태."""
    job_id: str
    data: str
    processed: bool
    result: str | None
    current_step: str


def fetch_data_node(state: JobState) -> dict:
    """데이터를 가져옵니다."""
    print(f"  [Fetch] Job {state['job_id']}: 데이터 수집 중...")
    return {
        "data": f"Raw data for {state['job_id']}",
        "current_step": "fetch",
    }


def process_data_node(state: JobState) -> dict:
    """데이터를 처리합니다."""
    print(f"  [Process] Job {state['job_id']}: 데이터 처리 중...")
    return {
        "processed": True,
        "result": f"Processed: {state['data']}",
        "current_step": "process",
    }


def save_result_node(state: JobState) -> dict:
    """결과를 저장합니다."""
    print(f"  [Save] Job {state['job_id']}: 결과 저장 중...")
    return {"current_step": "save"}


# 그래프 구성
graph = StateGraph(JobState)
graph.add_node("fetch", fetch_data_node)
graph.add_node("process", process_data_node)
graph.add_node("save", save_result_node)

graph.set_entry_point("fetch")
graph.add_edge("fetch", "process")
graph.add_edge("process", "save")
```


278

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
graph.add_edge("save", END)

# SqliteSaver: 파일로 체크포인트 저장
# 프로그램 종료 후에도 체크포인트가 유지됩니다
saver = SqliteSaver.from_conn_string("./checkpoints.db")
app = graph.compile(checkpointer=saver)


def run_job(job_id: str):
    """작업을 실행하거나 재개합니다."""
    config = {"configurable": {"thread_id": f"job-{job_id}"}}

    # 기존 체크포인트 확인
    existing_state = app.get_state(config)

    if existing_state.values:
        print(f"\n기존 체크포인트 발견! (마지막 단계:
        {existing_state.values.get('current_step')})")
        print("마지막 체크포인트에서 재개합니다...")

        # 에러 상태가 있다면 초기화
        if existing_state.values.get("error"):
            app.update_state(config, {"error": None})

        # 재개 (마지막 체크포인트부터)
        result = app.invoke(None, config=config)
    else:
        print(f"\n새로운 작업 시작: {job_id}")
        result = app.invoke(
            {
                "job_id": job_id,
                "data": "",
                "processed": False,
                "result": None,
                "current_step": "start",
            },
            config=config,
        )

    print(f"완료: {result.get('result', 'N/A')}")
    return result


# 실행
if __name__ == "__main__":
    run_job("analysis-001")
```

Step 3: PostgresSaver (프로덕션용)

279

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# checkpoint_postgres.py
"""PostgresSaver를 사용한 프로덕션 체크포인팅.

사전 조건: PostgreSQL이 실행 중이어야 합니다.
pip install langgraph-checkpoint-postgres psycopg[binary]
"""
from langgraph.checkpoint.postgres import PostgresSaver

# PostgreSQL 연결
# 환경변수에서 URI를 읽는 것이 좋습니다
POSTGRES_URI = "postgresql://user:password@localhost:5432/checkpoints_db"

saver = PostgresSaver.from_conn_string(POSTGRES_URI)
saver.setup()  # 체크포인트 테이블 자동 생성

# 이후 사용법은 MemorySaver/SqliteSaver와 동일:
# app = graph.compile(checkpointer=saver)
# result = app.invoke(input_state, config={"configurable": {"thread_id": "..."}})
```

체크포인터 팩토리 패턴 (환경별 자동 선택)

280

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# checkpointer_factory.py
"""환경에 따라 적절한 체크포인터를 생성하는 팩토리."""
import os
import logging

logger = logging.getLogger(__name__)


def create_checkpointer(
    checkpointer_type: str = "memory",
    sqlite_path: str = "./checkpoints.db",
    postgres_uri: str | None = None,
):
    """환경에 따라 적절한 체크포인터를 생성합니다.

    Args:
        checkpointer_type: "memory", "sqlite", 또는 "postgres".
        sqlite_path: SQLite 파일 경로 (sqlite 타입 시).
        postgres_uri: PostgreSQL 연결 URI (postgres 타입 시).

    Returns:
        LangGraph 체크포인터 인스턴스.

    Examples:
        >>> # 테스트용
        >>> saver = create_checkpointer("memory")

        >>> # 로컬 개발용
        >>> saver = create_checkpointer("sqlite", sqlite_path="./dev_checkpoints.db")

        >>> # 프로덕션용
        >>> saver = create_checkpointer("postgres", postgres_uri="postgresql://...")
    """
    if checkpointer_type == "memory":
        from langgraph.checkpoint.memory import MemorySaver
        logger.info("체크포인터: MemorySaver (인메모리)")
        return MemorySaver()

    elif checkpointer_type == "sqlite":
        from langgraph.checkpoint.sqlite import SqliteSaver
        logger.info("체크포인터: SqliteSaver (경로: %s)", sqlite_path)
        return SqliteSaver.from_conn_string(sqlite_path)

    elif checkpointer_type == "postgres":
        if not postgres_uri:
            postgres_uri = os.getenv("CHECKPOINT_POSTGRES_URI")
        if not postgres_uri:
            raise ValueError(
                "PostgresSaver에는 postgres_uri가 필요합니다. "
                "환경변수 CHECKPOINT_POSTGRES_URI를 설정하세요."
            )
        from langgraph.checkpoint.postgres import PostgresSaver
        logger.info("체크포인터: PostgresSaver")
        saver = PostgresSaver.from_conn_string(postgres_uri)
```


281

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
        saver.setup()
        return saver

    else:
        raise ValueError(f"알 수 없는 체크포인터 타입: {checkpointer_type}")
```

thread_id 설계 가이드

```
thread_id는 "실행 세션"을 식별하는 고유 키입니다.
같은 thread_id로 호출하면 → 이전 체크포인트에서 재개
새로운 thread_id로 호출하면 → 처음부터 새로 실행

thread_id 설계 패턴:

  작업 유형 + 고유 식별자
  ──────────────────────
  "analysis-user123-20260323"     ← 사용자 + 날짜
  "review-pr-456"                 ← PR 번호
  "translate-doc-abc123"          ← 문서 ID
  "pipeline-order-789-attempt-2"  ← 재시도 포함
```

2.2 서브그래프 (Subgraph Composition)

왜 필요한가?

그래프가 커지면 관리하기 어려워집니다:

```
문제: 10개 이상 노드가 한 그래프에 → 이해/유지보수 어려움

  [A] → [B] → [C] → [D] → [E] → [F] → [G] → [H] → [I] → [J]
  └──────────── 10개 노드가 하나의 거대한 그래프에 ────────────┘

해결: 서브그래프로 분리 → 각 모듈을 독립적으로 관리

  [메인 그래프]
  [A] → [B] → [서브그래프 1] → [서브그래프 2] → [J]
                    │                  │
              [C] → [D] → [E]   [F] → [G] → [H] → [I]
```

서브그래프의 장점:

282

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

장점	설명
모듈화	각 부분을 독립적으로 개발/테스트 가능
재사용	같은 서브그래프를 여러 메인 그래프에서 사용
상태 격리	서브그래프는 필요한 필드만 접근
가독성	전체 구조를 한눈에 파악 가능

Step 1: 서브그래프 정의

283

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# subgraph_example.py
"""서브그래프 패턴 예제: 문서 분석 파이프라인.

메인 그래프: 입력 → 전처리 → [분석 서브그래프] → 결과 정리 → 출력
분석 서브그래프: 키워드 추출 → 요약 생성 → 감성 분석
"""
from typing import TypedDict

from langgraph.graph import StateGraph, END
from langgraph.checkpoint.memory import MemorySaver


# --- 서브그래프 상태 (분석에 필요한 필드만) ---

class AnalysisSubState(TypedDict):
    """분석 서브그래프의 격리된 상태.

    메인 그래프의 전체 상태가 아닌,
    분석에 필요한 필드만 포함합니다.
    """
    text: str                       # 분석할 텍스트
    keywords: list[str] | None      # 추출된 키워드
    summary: str | None             # 요약
    sentiment: str | None           # 감성 (positive/negative/neutral)


# --- 서브그래프 노드 ---

def extract_keywords_node(state: AnalysisSubState) -> dict:
    """키워드를 추출합니다."""
    text = state["text"]
    # 간단한 키워드 추출 (실제로는 LLM 호출)
    words = text.split()
    keywords = [w for w in words if len(w) > 3][:5]
    print(f"  [서브그래프] 키워드 추출: {keywords}")
    return {"keywords": keywords}


def summarize_node(state: AnalysisSubState) -> dict:
    """텍스트를 요약합니다."""
    text = state["text"]
    # 간단한 요약 (실제로는 LLM 호출)
    summary = text[:100] + "..." if len(text) > 100 else text
    print(f"  [서브그래프] 요약 생성 완료")
    return {"summary": summary}


def analyze_sentiment_node(state: AnalysisSubState) -> dict:
    """감성을 분석합니다."""
    # 간단한 감성 분석 (실제로는 LLM 호출)
    text = state["text"].lower()
    if any(w in text for w in ["좋", "훌륭", "만족", "good", "great"]):
        sentiment = "positive"
    elif any(w in text for w in ["나쁜", "불만", "실망", "bad", "poor"]):
```


284

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
        sentiment = "negative"
    else:
        sentiment = "neutral"
    print(f"  [서브그래프] 감성 분석: {sentiment}")
    return {"sentiment": sentiment}


# --- 서브그래프 빌드 ---

def build_analysis_subgraph():
    """분석 서브그래프를 빌드합니다.

    키워드 추출 → 요약 생성 → 감성 분석
    """
    graph = StateGraph(AnalysisSubState)

    graph.add_node("extract_keywords", extract_keywords_node)
    graph.add_node("summarize", summarize_node)
    graph.add_node("analyze_sentiment", analyze_sentiment_node)

    graph.set_entry_point("extract_keywords")
    graph.add_edge("extract_keywords", "summarize")
    graph.add_edge("summarize", "analyze_sentiment")
    graph.add_edge("analyze_sentiment", END)

    return graph.compile()


# --- 메인 그래프 상태 ---

class MainState(TypedDict):
    """메인 그래프 상태 (전체 필드 포함)."""
    document_id: str
    raw_text: str
    cleaned_text: str | None
    # 서브그래프 결과가 여기에 매핑됨
    text: str                       # 서브그래프 입력용
    keywords: list[str] | None
    summary: str | None
    sentiment: str | None
    # 최종 결과
    report: str | None


# --- 메인 그래프 노드 ---

def preprocess_node(state: MainState) -> dict:
    """텍스트를 전처리합니다."""
    raw = state["raw_text"]
    cleaned = raw.strip().replace("\n", " ")
    print(f"[메인] 전처리 완료: {len(cleaned)}자")
    return {"cleaned_text": cleaned, "text": cleaned}


def generate_report_node(state: MainState) -> dict:
```


285

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
    """최종 리포트를 생성합니다."""
    report = (
        f"문서 분석 리포트 (ID: {state['document_id']})\n"
        f"{'='*50}\n"
        f"키워드: {', '.join(state.get('keywords') or [])}\n"
        f"요약: {state.get('summary', 'N/A')}\n"
        f"감성: {state.get('sentiment', 'N/A')}\n"
    )
    print(f"[메인] 리포트 생성 완료")
    return {"report": report}


# --- 메인 그래프 구성 ---

def build_main_graph():
    """서브그래프가 포함된 메인 그래프를 구성합니다.

    흐름:
        preprocess → [analysis_subgraph] → generate_report → END
    """
    graph = StateGraph(MainState)

    # 서브그래프를 노드로 등록
    analysis_subgraph = build_analysis_subgraph()
    graph.add_node("preprocess", preprocess_node)
    graph.add_node("analyze", analysis_subgraph)  # 서브그래프를 노드처럼 등록!
    graph.add_node("generate_report", generate_report_node)

    graph.set_entry_point("preprocess")
    graph.add_edge("preprocess", "analyze")
    graph.add_edge("analyze", "generate_report")
    graph.add_edge("generate_report", END)

    checkpointer = MemorySaver()
    return graph.compile(checkpointer=checkpointer)


# --- 실행 ---

if __name__ == "__main__":
    app = build_main_graph()

    result = app.invoke(
        {
            "document_id": "DOC-001",
            "raw_text": "이 제품은 정말 훌륭합니다. 성능이 좋고 디자인이 만족스럽습니다.",
            "cleaned_text": None,
            "text": "",
            "keywords": None,
            "summary": None,
            "sentiment": None,
            "report": None,
        },
        config={"configurable": {"thread_id": "doc-analysis-001"}},
    )
```


286

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
    print(f"\n{result['report']}")
```

상태 매핑 다이어그램

```
메인 그래프 상태 (MainState)         서브그래프 상태 (AnalysisSubState)
┌──────────────────────┐            ┌──────────────────────┐
│ document_id          │            │                      │
│ raw_text             │            │                      │
│ cleaned_text         │            │                      │
│ text ────────────────│──입력──→│ text                 │
│ keywords ←───────────│──출력──│ keywords             │
│ summary ←────────────│──출력──│ summary              │
│ sentiment ←──────────│──출력──│ sentiment            │
│ report               │            │                      │
└──────────────────────┘            └──────────────────────┘

* 서브그래프는 자신의 상태 필드만 접근 가능
* 이름이 같은 필드를 통해 자동으로 매핑됨
```

2.3 병렬 실행 (Fan-out / Fan-in)

핵심 개념

```
순차 실행 (기존):
  [문서 A 분석] → [문서 B 분석] → [문서 C 분석] → [결과 합치기]
  총 시간: 30초 + 30초 + 30초 = 90초

병렬 실행 (Send API):
  ┌→ [문서 A 분석] ┐
  ├→ [문서 B 분석] ┼→ [결과 합치기]
  └→ [문서 C 분석] ┘
  총 시간: max(30초, 30초, 30초) = 30초

Fan-out: 하나의 노드에서 여러 병렬 노드로 작업 분배
Fan-in:  여러 병렬 노드의 결과를 하나로 수집
```

Send API + Reducer 구현

287

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```python
# parallel_example.py
"""Send API를 사용한 병렬 실행 예제: 멀티 문서 분석."""
import operator
from typing import Annotated, TypedDict

from langgraph.graph import StateGraph, END
from langgraph.constants import Send
from langchain_anthropic import ChatAnthropic


# --- 상태 정의 ---

class DocumentResult(TypedDict):
    """개별 문서 분석 결과."""
    doc_id: str
    summary: str
    word_count: int


class ParallelState(TypedDict):
    """병렬 처리 상태.

    핵심: Annotated[list, operator.add]
    - 각 병렬 노드가 반환한 리스트가 자동으로 합쳐집니다.
    - 예: 노드 A가 [결과1]을, 노드 B가 [결과2]를 반환하면
          최종 results = [결과1, 결과2]
    """
    documents: list[dict]                                    # 입력 문서 목록
    results: Annotated[list[DocumentResult], operator.add]   # 병렬 결과 수집
    final_report: str | None                                 # 최종 리포트


# --- 노드 정의 ---

def prepare_node(state: ParallelState) -> dict:
    """문서 목록을 확인하고 병렬 처리를 준비합니다."""
    docs = state.get("documents", [])
    print(f"[준비] {len(docs)}개 문서를 병렬 분석합니다.")
    return {}


def fan_out_documents(state: ParallelState) -> list[Send]:
    """각 문서에 대해 독립적인 분석 노드를 생성합니다 (Fan-out).

    Send API 핵심:
    - Send("노드이름", 상태)를 리스트로 반환
    - 각 Send는 독립적으로 병렬 실행됨
    - 결과는 Reducer(operator.add)로 자동 수집됨
    """
    documents = state.get("documents", [])

    if not documents:
        return [Send("generate_report", state)]
```


288

AI Agent 개발 입문: LangGraph로 시작하는 에이전트 개발

```
    sends = []
    for doc in documents:
        # 각 문서에 대해 독립적인 상태를 만들어 Send
        send_state = {
            **state,
            "_current_doc": doc,    # 현재 처리할 문서
            "results": [],          # 각 병렬 노드의 초기값 (빈 리스트)
        }
        sends.append(Send("analyze_document", send_state))

    return sends


def analyze_document_node(state: ParallelState) -> dict:
    """개별 문서를 분석합니다 (병렬 실행됨).

    반환하는 results 리스트는 Reducer(operator.add)에 의해
    다른 병렬 노드의 결과와 자동으로 합쳐집니다.
    """
    doc = state.get("_current_doc", {})
    doc_id = doc.get("id", "unknown")
    content = doc.get("content", "")

    print(f"  [분석] 문서 {doc_id} 분석 중...")

    # 실제로는 LLM을 호출하여 분석
    word_count = len(content.split())
    summary = content[:50] + "..." if len(content) > 50 else content

    return {
        "results": [
            DocumentResult(
                doc_id=doc_id,
                summary=summary,
                word_count=
```


289